ĐẠI HỌC QUỐC GIA TP. HỒ CHÍ MINH

TRƯỜNG ĐẠI HỌC CÔNG NGHỆ THÔNG TIN

KHOA MẠNG MÁY TÍNH VÀ TRUYỀN THÔNG

Nguyễn Xuân Huy

Nguyễn Khang Hưng

KHÓA LUẬN TỐT NGHIỆP

NGHIÊN CỨU KHẢ NĂNG PHÁT HIỆN LỖ HỔNG PHẦN MỀM CỦA CÁC MÔ HÌNH NGÔN NGỮ LỚN DỰA TRÊN HỌC KHẢ DIỄN GIẢI

EXPLORING THE VULNERABILITY DETECTION CAPABILITIES OF LARGE LANGUAGE MODELS THROUGH EXPLAINABLE ARTIFICIAL INTELLIGENCE

CỬ NHÂN NGÀNH AN TOÀN THÔNG TIN

TP. HỒ CHÍ MINH, 2026

ĐẠI HỌC QUỐC GIA TP. HỒ CHÍ MINH

TRƯỜNG ĐẠI HỌC CÔNG NGHỆ THÔNG TIN

KHOA MẠNG MÁY TÍNH VÀ TRUYỀN THÔNG

Nguyễn Xuân Huy – 22520568

Nguyễn Khang Hưng – 22520515

KHÓA LUẬN TỐT NGHIỆP

Nghiên cứu khả năng phát hiện lỗ hổng phần mềm của các mô hình ngôn ngữ lớn dựa trên học khả diễn giải

Exploring the Vulnerability Detection Capabilities of Large Language Models through Explainable Artificial Intelligence

CỬ NHÂN NGÀNH AN TOÀN THÔNG TIN

GIẢNG VIÊN HƯỚNG DẪN

ThS. Tô Trọng Nghĩa

TP. HỒ CHÍ MINH, 2026

THÔNG TIN HỘI ĐỒNG CHẤM KHÓA LUẬN TỐT NGHIỆP

Hội đồng chấm khóa luận tốt nghiệp, thành lập theo Quyết định số 60 ngày 15 tháng 01 năm 2026 của Hiệu trưởng Trường Đại học Công nghệ Thông tin.

Chủ tịch: TS. Lê Văn Tuấn.

Thư ký: ThS. Đỗ Thị Thu Hiền.

Ủy viên: ThS. Trần Mạnh Hùng.

LỜI CẢM ƠN

Khóa luận tốt nghiệp là kết quả của quá trình học tập, rèn luyện và nghiên cứu trong suốt những năm Đại học. Để hoàn thành được khóa luận này, bên cạnh nỗ lực của bản thân, chúng em đã nhận được sự hướng dẫn và hỗ trợ quý báu từ thầy cô và những người xung quanh.

Em xin bày tỏ lòng biết ơn chân thành đến anh Tô Trọng Nghĩa, là người đã hướng dẫn và đồng hành cùng chúng em trong khóa luận này. Anh đã tận tình hướng dẫn, định hướng nghiên cứu và đưa ra những góp ý chuyên môn quan trọng, giúp em từng bước hoàn thiện nội dung và phương pháp thực hiện khóa luận.

Em cũng xin gửi lời cảm ơn đến quý thầy cô Trường Công nghệ Thông tin – Đại học Quốc gia Thành phố Hồ Chí Minh đã trang bị cho em những kiến thức nền tảng trong suốt quá trình học tập.

Cuối cùng, em xin cảm ơn gia đình đã luôn động viên, ủng hộ và tạo điều kiện để em có thể hoàn thành khóa luận tốt nghiệp.

Em xin chân thành cảm ơn!

Nguyễn Xuân Huy **Nguyễn Khang Hưng**

MỤC LỤC

TÓM TẮT ĐỀ TÀI 1

Chương 1. TỔNG QUAN 3

1.1. Giới thiệu vấn đề 3

1.2. Mục tiêu và phạm vi nghiên cứu 5

1.2.1. Mục tiêu nghiên cứu 5

1.2.2. Phạm vi nghiên cứu 5

1.3. Đối tượng nghiên cứu và hướng tiếp cận 6

1.4. Các nghiên cứu liên quan 7

Chương 2. CƠ SỞ LÝ THUYẾT 9

2.1. Các khái niệm trong an toàn phần mềm 9

2.1.1. Lỗ hổng phần mềm (Software Vulnerability) 9

2.1.2. Lỗ hổng phần mềm phổ biến 10

2.2. Phát hiện lỗ hổng từ mã nguồn 11

2.2.1. Bài toán phân loại 11

2.2.2. Mức độ phân tích mã nguồn 12

2.2.3. Phân loại vulnerable / non-vulnerable 13

2.2.4. Vai trò của bộ dữ liệu trong đánh giá mô hình 14

2.3. Mô hình ngôn ngữ lớn cho mã nguồn (LLMs-for-code) 15

2.3.1. Transformer 15

2.3.2. LLMs cho mã nguồn 18

2.3.2.1. Encoder-only 18

2.3.2.2. Encoder–Decoder 20

2.3.2.3. Decoder-only 21
2.3.3. Ưu điểm và hạn chế của LLMs-for-code 24
2.4. Các mô hình LLMs-for-code 25
2.4.1. Mô hình BART và PLBart 25
2.4.2. Mô hình StarCoder2 26
2.4.3. Mô hình Qwen2 26
2.5. Prompting 28
2.5.1. Khái niệm và vai trò của prompting 28
2.5.2. Vanilla Prompt 29
2.5.3. Chain-of-Thought Prompt 31
2.5.4. Prompting trong bài toán phân tích mã nguồn 34
2.6. Prompting và khả năng hỗ trợ giải thích kết quả 35
2.6.1. Explainable AI (XAI) 35
2.6.2. Hạn chế của XAI truyền thống đối với LLMs 37
2.6.3. Chain-of-Thought như một dạng giải thích ngầm 38
Chương 3. PHƯƠNG PHÁP THỰC HIỆN 39
3.1. Kiến trúc tổng quát 39
3.2. Chuẩn bị và tiền xử lý dữ liệu 40
3.2.1. Chuẩn hóa dữ liệu đầu vào 40
3.2.2. Tiền xử lý mã nguồn 41
3.2.3. Tổ chức tập dữ liệu cho thí nghiệm 41
3.3. LLM (Base / Fine-tune) 41
3.3.1. Nhánh mô hình cơ sở (Base model) 42
3.3.2. Nhánh mô hình tinh chỉnh (Fine-tune) 42

3.4. Prompting (Vanilla / Chain-of-Thought) 43
3.4.1. Vanilla Prompt 44
3.4.2. Chain-of-Thought Prompt 44
3.4.3. So sánh hai chiến lược prompting 45
3.5. Prediction 45
3.6. Evaluation 46
3.7. Giải thích mô hình dựa trên SHAP 47
Chương 4. THỰC NGHIỆM VÀ ĐÁNH GIÁ KẾT QUẢ 50
4.1. Câu hỏi nghiên cứu (Research Questions – RQ) 50
4.2. Bộ dữ liệu và tiền xử lý 51
4.2.1. Bộ dữ liệu thực nghiệm 51
4.2.2. Tiền xử lý 52
4.3. Mô hình và cấu hình triển khai 52
4.3.1. Mô hình 52
4.3.1.1. Mô hình Starcoder2 52
4.3.1.1.1 Tạo dataset từ Juliet và gán nhãn good/bad 53
4.3.1.1.2 Chia tập Train/Test 53
4.3.1.1.3 Fine-tune StarCoder2 bằng HuggingFace Trainer 54
4.3.1.1.4 Suy luận và chuẩn hóa dự đoán (inference + parsing) 55
4.3.1.1.5 Giải thích mô hình bằng SHAP (token occlusion) 56
4.3.1.2. Mô hình Qwen2 56
4.3.1.2.1 Chuẩn bị dữ liệu đầu vào cho Qwen2 56
4.3.1.2.2 Thiết kế prompt cho Qwen2 57
4.3.1.2.3 Fine-tune Qwen2 bằng LoRA 57

4.3.1.2.4 Đánh giá định lượng 58
4.3.1.2.5 Giải thích mô hình Qwen2 bằng SHAP 58
4.3.1.3. Mô hình BART 58
4.3.1.3.1 Chuẩn bị và tiền xử lý dữ liệu 58
4.3.1.3.2 Huấn luyện mô hình BART 59
4.3.1.3.3 Suy luận và chuẩn hóa đầu ra 60
4.3.1.3.4 Đánh giá mô hình BART 60
4.3.1.3.5 Giải thích mô hình BART bằng SHAP 60
4.3.1.4. Mô hình PLBART 61
4.3.1.4.1 Chuẩn bị và tiền xử lý dữ liệu 61
4.3.1.4.2 Kiến trúc mô hình PLBART cho phân loại 61
4.3.1.4.3 Tokenization và Data Collator 62
4.3.1.4.4 Huấn luyện mô hình PLBART 62
4.3.2. Cấu hình triển khai 63
4.4. Thiết lập đánh giá 64
4.4.1. Chỉ số đánh giá cho bài toán phân loại 64
4.4.2. Chỉ số đánh giá cho XAI (SHAP-based Metrics) 65
4.4.2.1. Sparsity (Độ thưa của lời giải thích) 65
4.4.2.2. Stability (Độ ổn định) 66
4.4.2.3. Robustness (Độ bền vững) 66
4.4.2.4. Efficiency (Thời gian tính toán) 66
4.4.3. Dữ liệu đánh giá 67
4.4.4. Nhận xét chung 69
4.5. Thiết lập thí nghiệm và phương pháp đánh giá 70

4.5.1. Tập mô hình và trạng thái mô hình 70

4.5.2. Dữ liệu và thiết lập ngôn ngữ.....70

4.5.3. Chiến lược prompting và định dạng đầu ra 71

4.5.4. Cấu hình huấn luyện và suy luận 71

4.5.5. Quy trình đánh giá tự động.....72

4.5.6. Phương pháp đánh giá hiệu năng dự đoán.....72

4.5.7. Phương pháp đánh giá khả diễn giải 72

4.6. Thí nghiệm E0 – Mô hình chưa fine-tune (Baseline) 73

4.6.1. Kịch bản đánh giá 73

4.6.2. Kết quả thực nghiệm 73

4.6.3. Kết luận thí nghiệm 75

4.7. Thí nghiệm E1 – Mô hình fine-tune với từng dataset C/C++ & Java .76

4.7.1. Kịch bản đánh giá 76

4.7.2. Kết quả thực nghiệm 76

4.7.3. Kết luận thí nghiệm 77

4.8. Thí nghiệm E2 – Chuyển giao chéo ngôn ngữ (C/C++ → Java) và ngược lại 78

4.8.1. Kịch bản đánh giá 78

4.8.2. Kết quả thực nghiệm 78

4.8.3. Kết luận thí nghiệm 80

4.9. Thí nghiệm E3 – Ảnh hưởng của Prompting (Vanilla Prompt vs Chain-of-Thought Prompt) 81

4.9.1. Kịch bản đánh giá 81

4.9.2. Kết quả thí nghiệm.....81

4.9.3. Kết luận thí nghiệm 82

4.10. Thí nghiệm E4 – Tác động của ShAP đến hành vi dự đoán của mô hình 83

4.10.1. Kịch bản đánh giá 83

4.10.2. Kết quả thực nghiệm và phân tích 84

4.10.3. Kiểm tra mô hình bằng XAI (SHAP).....86

4.10.4. Đề xuất công thức chung (XAI_score) 87

4.10.5. Kết luận thí nghiệm 90

4.11. Kết quả thực nghiệm tổng của chương 91

Chương 5. KẾT LUẬN 98

TÀI LIỆU THAM KHẢO 100

DANH MỤC HÌNH

Hình 2.1 Minh họa bài toán phát hiện lỗ hổng.....12
Hình 2.2 Confusion Matrix 14
Hình 2.3 Kiến trúc Transformer.....16
Hình 2.4 Self-attention 17
Hình 2.5 Kiến trúc encoder-only.....19
Hình 2.6 Cross-Attention 21
Hình 2.7 Kiến trúc Decoder-only.....22
Hình 2.8 Self-attention một chiều (causal attention) 23
Hình 2.9 Prompting định hướng quá trình phân tích mã nguồn. 29
Hình 2.10 Vanilla prompt 30
Hình 2.11 Vanilla Prompt trong python.....31
Hình 2.12 Chain-of-Thought (CoT) Prompting.....33
Hình 2.13 Chain-of-Thought (CoT) Prompting trong python 33
Hình 2.14 Explainable AI (XAI) giúp các mô hình trở nên dễ hiểu hơn.....37
Hình 3.1 Mô hình tổng quan 40
Hình 3.2 Minh họa cơ chế giải thích dự đoán của mô hình phát hiện lỗ hổng mã nguồn bằng phương pháp SHAP.....48
Hình 4.1 Phân bố tập train/test và nhãn good/bad của bộ dữ liệu Juliet C/C++.....68
Hình 4.2 Phân bố tập train/test và nhãn good/bad của bộ dữ liệu Juliet Java.....69
Hình 4.3 Kết quả khả diễn giải dựa trên SHAP 86
Hình 4.4 Mô hình không được train với dataset và sử dụng Vanila Prompt 92
Hình 4.5 Mô hình đã được train với dataset Juliet c/c++ và sử dụng Vanila Prompt 93
Hình 4.6 Mô hình đã được train với dataset Juliet Java và sử dụng Vanila Prompt.94
Hình 4.7 Mô hình không train với dataset và sử dụng COT Prompt 95
Hình 4.8 Mô hình đã được train với dataset Juliet C/C++ và sử dụng COT Prompt96
Hình 4.9 Mô hình đã được train với dataset Juliet Java và sử dụng COT Prompt ...97

DANH MỤC BẢNG

Bảng 2.1 Nhóm lỗ hổng phần mềm phổ biến trong mã nguồn 11
Bảng 2.2 So sánh các mức độ phân tích mã nguồn.....13
Bảng 2.3 LLMs cho mã nguồn.....24
Bảng 2.4 Các mô hình LLMs-for-code sử dụng trong đề tài 27
Bảng 2.5 Các chiến lược prompting.....35
Bảng 4.1 Cấu hình triển khai các mô hình 64
Bảng 4.2 Bảng kết quả số liệu mô hình chưa fine_tune và sử dụng vanilla prompt 74
Bảng 4.3 Bảng kết quả số liệu mô hình đã được finetune với tập train C/C++ và sử dụng vanilla prompt 74
Bảng 4.4 Bảng kết quả số liệu mô hình đã được finetune với tập train C/C++ và sử dụng vanilla prompt 77
Bảng 4.5 Bảng kết quả số liệu mô hình đã được finetune với tập train Java và sử dụng vanilla prompt 77
Bảng 4.6 Bảng kết quả số liệu mô hình đã được finetune với ngôn ngữ C/C++ và test với Java 79
Bảng 4.7 Bảng kết quả số liệu mô hình đã được finetune với ngôn ngữ Java và test với C/C++.....79
Bảng 4.8 Bảng kết quả mô hình được finetune bởi tập train C/C++ và kết hợp với Vanila Prompt 82
Bảng 4.9 Bảng kết quả mô hình được finetune bởi tập train C/C++ và kết hợp với COT Prompt 82
Bảng 4.10 So sánh các chỉ số XAI của các mô hình LLMs-for-code dựa trên phương pháp SHAP.....84
Bảng 4.11 Kết quả XAI_Score của các mô hình LLMs-for-code 89

DANH MỤC TỪ VIẾT TẮT, CÁC KÝ HIỆU

TP	Số mẫu dự đoán đúng là có lỗ hổng
TN	Số mẫu dự đoán đúng là không có lỗ hổng
FP	Số mẫu dự đoán nhầm là có lỗ hổng
FN	Số mẫu dự đoán nhầm là không có lỗ hổng
$Accuracy$	Độ chính xác tổng thể của mô hình
$Precision$	Độ chính xác của dự đoán lỗ hổng
$Recall$	Khả năng phát hiện đầy đủ các mẫu có lỗ hổng
$F1$	Chỉ số cân bằng giữa Precision và Recall
Q	Vector truy vấn
K	Vector khóa dùng để so khớp mức độ liên quan
V	Vector giá trị chứa thông tin được tổng hợp
QK^T	Tích vô hướng giữa query và key
d_k	Kích thước của vector key/query
Softmax()	Hàm chuẩn hóa xác suất
Attention(Q, K, V)	Hàm tổng hợp thông tin từ các token liên quan
Exact Match	Chỉ số đánh giá mức độ trùng khớp hoàn toàn giữa nhãn dự đoán và nhãn chuẩn.
ϕ	Vector giá trị SHAP biểu diễn mức độ đóng góp của các token trong một mẫu
ϕ_i	Giá trị SHAP của token thứ i
Stability	Chỉ số đánh giá độ ổn định của lời giải thích trước nhiễu nhỏ

Robustness	Chỉ số đo độ bền vững của lời giải thích trước nhiễu
corr()	Hệ số tương quan giữa hai vector
$\phi^{(j)}$	Vector giá trị SHAP của mẫu sau lần perturbation thứ j
t_{SHAP}	Thời gian thực thi của phương pháp SHAP
L	Tổng số token trong chuỗi đầu vào
ϵ	Ngưỡng nhỏ dùng để loại bỏ nhiễu trong tính Sparsity
$\mathrm{h}_{\text{pooled}}$	Vector đặc trưng tổng hợp sau bước mean pooling
h_i	Hidden state của encoder tại token thứ i
n	Tổng số token trong mẫu đầu vào

TÓM TẮT ĐỀ TÀI

Trong đề tài này, nhóm tập trung nghiên cứu khả năng phát hiện lỗ hổng phần mềm của các mô hình ngôn ngữ lớn cho mã nguồn (Large Language Models for Code), đồng thời khảo sát vai trò của prompting trong việc định hướng suy luận và hỗ trợ giải thích kết quả dự đoán của mô hình. Thay vì chỉ đánh giá hiệu năng dự đoán, đề tài hướng đến việc so sánh các cách thiết kế prompt nhằm xem xét mức độ ảnh hưởng của prompting đến độ tin cậy và khả diễn giải trong bài toán phân loại mã nguồn.

Các mô hình được sử dụng trong đề tài bao gồm các mô hình LLMs-for-code và nó đại diện cho những kiến trúc khác nhau, bao gồm StarCoder2, BART, PLBART và Qwen2. Nhóm tiến hành thực nghiệm trên bộ dữ liệu công khai Juliet Test Suite cho hai ngôn ngữ lập trình C/C++ và Java. Trong quá trình nghiên cứu, hai chiến lược prompting chính được áp dụng là Vanilla Prompt, yêu cầu mô hình đưa ra kết luận trực tiếp, và Chain-of-Thought Prompt (CoT), yêu cầu mô hình phân tích mã nguồn theo từng bước trước khi đưa ra kết luận cuối cùng. Các thí nghiệm được thực hiện trên cả hai kịch bản mô hình chưa được huấn luyện với dữ liệu và mô hình đã được huấn luyện nhằm đánh giá riêng biệt ảnh hưởng của prompting.

Kết quả thực nghiệm cho thấy các mô hình LLMs-for-code có khả năng phát hiện lỗ hổng phần mềm ở mức hiệu quả trên bộ dữ liệu Juliet, với các chỉ số Accuracy, Precision, Recall và F1-score đạt mức ổn định. So sánh giữa các mô hình cho thấy các mô hình encoder–decoder, đặc biệt là PLBART, thường đạt Recall và F1-score cao hơn so với các mô hình còn lại. Bên cạnh đó, việc sử dung Chain-of-Thought Prompt giúp cải thiện rõ rệt Precision và Recall so với Vanilla Prompt trong hầu hết các trường hợp, qua đó góp phần giảm số lượng cảnh báo giả và giúp cân bằng tốt hơn giữa Precision và Recall. Đồng thời CoT Prompt cho phép quan sát rõ hơn quá trình suy luận của mô hình thông qua các bước phân tích trung gian.

Ngoài ra, đề tài cũng ghi nhận rằng hiệu quả phát hiện lỗ hổng phụ thuộc đáng kể vào ngôn ngữ lập trình và đặc thù của dữ liệu. Kết quả thực nghiệm trên mã nguồn

C/C++ nhìn chung thì tốt hơn so với Java, cho thấy đặc điểm ngôn ngữ và cấu trúc mã nguồn có ảnh hưởng trực tiếp đến khả năng học và suy luận của mô hình.

Từ các kết quả đạt được, nhóm nhận thấy rằng prompting và đặc biệt là Chain-of-Thought Prompt không chỉ giúp cải thiện hiệu năng phát hiện lỗ hổng mà còn hỗ trợ nâng cao khả năng diễn giải và độ tin cậy của các mô hình LLMs-for-code. Tuy nhiên, việc sử dụng CoT Prompt cũng làm tăng chi phí suy luận và độ dài phản hồi nên cần cân nhắc giữa hiệu quả, khả diễn giải và chi phí tính toán khi áp dụng trong thực tế.

Chương 1. TỔNG QUAN

Chương này trình bày tổng quan về bài toán phát hiện lỗ hổng phần mềm từ mã nguồn trong bối cảnh ứng dụng các mô hình ngôn ngữ lớn cho mã nguồn (Large Language Models for Code – LLMs-for-code), đồng thời làm rõ bối cảnh nghiên cứu, các hướng tiếp cận liên quan và vai trò của prompting trong việc định hướng hành vi cũng như hỗ trợ giải thích kết quả của mô hình. Trên cơ sở tổng hợp và phân tích các công trình nghiên cứu trước, chương xác định những vấn đề còn tồn tại, mục tiêu và phạm vi nghiên cứu của đề tài, làm nền tảng cho các nội dung lý thuyết và thực nghiệm được trình bày ở các chương tiếp theo.

1.1. Giới thiệu vấn đề

Phát hiện lỗ hổng phần mềm là một nhiệm vụ quan trọng trong lĩnh vực an toàn thông tin, nhằm hạn chế các rủi ro bảo mật như rò rỉ dữ liệu, leo thang đặc quyền hoặc bị khai thác trái phép trong quá trình vận hành hệ thống. Các lỗ hổng thường xuất hiện do lỗi lập trình, xử lý bộ nhớ không an toàn hoặc kiểm tra dữ liệu đầu vào chưa đầy đủ và việc phát hiện sớm các lỗ hổng này đóng vai trò then chốt trong vòng đời phát triển phần mềm.

Trong những năm gần đây, các mô hình ngôn ngữ lớn cho mã nguồn (Large Language Models for Code – LLMs-for-code) đã được nghiên cứu và ứng dụng rộng rãi trong nhiều bài toán phân tích mã nguồn như phát hiện lỗi, sinh mã, sửa lỗi và phát hiện lỗ hổng phần mềm. Trong đó tiêu biểu là CodeBERT, mô hình tiền huấn luyện trên dữ liệu song ngữ gồm mã nguồn và ngôn ngữ tự nhiên, cho phép học được cả cú pháp và ngữ nghĩa của chương trình [4]. Nhờ khả năng này, LLMs-for-code có thể đưa ra dự đoán lỗ hổng mà không cần xây dựng thủ công các luật (rule) phân tích như trong các phương pháp phân tích tĩnh truyền thống.

Tuy nhiên, bên cạnh hiệu năng dự đoán, một hạn chế lớn của các mô hình LLMs-for-code là tính chất "hộp đen" của mô hình. Trong nhiều trường hợp, mô hình chỉ đưa ra kết luận một đoạn mã là có lỗ hổng hoặc không có lỗ hổng, nhưng không cung cấp lời giải thích rõ ràng cho quyết định đó. Các nghiên cứu tổng quan gần đây về khả diễn giải của mô hình ngôn ngữ lớn cũng chỉ ra rằng việc thiếu cơ chế giải thích làm giảm độ tin cậy khi áp dụng các mô hình này trong thực tế và đặc biệt là trong lĩnh vực an toàn thông tin [13].

Ngoài ra, nhiều nghiên cứu thực nghiệm trong trong bài toán phát hiện lỗ hổng phần mềm cho thấy các mô hình học sâu (Deep Learning - DL) thường đạt khả năng bao phủ lỗ hổng ở mức tương đối cao, nhưng độ chính xác chưa ổn định dẫn đến số lượng cảnh báo giả còn đáng kể [10]. Trong bối cảnh này, việc xuất hiện quá nhiều cảnh báo giả không chỉ làm tăng chi phí cho việc phân tích mà còn có thể khiến người sử dụng bỏ qua các cảnh báo quan trọng, từ đó làm giảm hiệu quả của hệ thống phát hiện lỗ hổng.

Song song với đó thì một số công trình đã chỉ ra rằng kết quả và hành vi suy luận của các mô hình ngôn ngữ lớn phụ thuộc mạnh vào cách thiết kế đầu vào. Đặc biệt, mô hình encoder–decoder như PLBART cho thấy khả năng thích nghi tốt với nhiều tác vụ phân tích mã nguồn khác nhau, nhờ vào cơ chế tiền huấn luyện thống nhất cho cả hiểu và sinh mã [1]. Điều này gợi ý rằng cách cung cấp thông tin đầu vào, hay còn gọi là prompting, có thể ảnh hưởng đáng kể đến cách mô hình phân tích và đưa ra kết luận.

Từ những vấn đề nêu trên, đề tài nghiên cứu được thực hiện nhằm khảo sát khả năng phát hiện lỗ hổng phần mềm của các mô hình LLMs-for-code, đồng thời phân tích vai trò của prompting trong việc định hướng hành vi suy luận và nâng cao độ tin cậy của kết quả dự đoán. Cách tiếp cận này hướng đến việc đánh giá không chỉ hiệu năng mô hình, mà còn mức độ phù hợp của các loại prompting khi áp dụng LLMs-for-code trong lĩnh vực toàn thông tin.

1.2. Mục tiêu và phạm vi nghiên cứu

1.2.1. Mục tiêu nghiên cứu

Mục tiêu của đề tài là nghiên cứu khả năng phát hiện lỗ hổng phần mềm của các mô hình ngôn ngữ lớn cho mã nguồn (Large Language Models for Code – LLMs-for-code) trong bài toán phân loại mã nguồn [4]. Thông qua việc áp dụng các mô hình trên bộ dữ liệu công khai Juliet Test Suite [2], nhóm đánh giá mức độ hiệu quả và tính ổn định của mô hình thông qua các chỉ số đánh giá phổ biến như Accuracy, Precision, Recall và F1-score.

Bên cạnh việc đánh giá hiệu năng thì đề tài còn tập trung phân tích ảnh hưởng của các chiến lược prompting đến kết quả dự đoán của mô hình. Nhóm so sánh hai cách thiết kế prompt là Vanilla Prompt và Chain-of-Thought Prompt nhằm làm rõ vai trò của prompting trong việc định hướng quá trình suy luận của mô hình khi phân tích mã nguồn và đưa ra kết luận về lỗ hổng [13], [11].

Ngoài ra thì đề tài còn hướng đến việc đánh giá khả năng hỗ trợ giải thích kết quả của các mô hình LLMs-for-code thông qua các bước suy luận được sinh ra từ Chain-of-Thought Prompt. Thông qua đó, nhóm xem xét tiềm năng của prompting như một cơ chế giúp nâng cao độ tin cậy và khả diễn giải của kết quả phát hiện lỗ hổng, mà không cần thay đổi kiến trúc hoặc huấn luyện lại mô hình [5].

1.2.2. Phạm vi nghiên cứu

Trong phạm vi của đề tài, nhóm tập trung vào bài toán phát hiện lỗ hổng phần mềm dưới dạng phân loại mã nguồn ở mức hàm hoặc đoạn mã, trong đó mỗi mẫu được gán nhãn có lỗ hổng (vulnerable) hoặc không có lỗ hổng (non-vulnerable). Các thí nghiệm được thực hiện trên các mô hình ngôn ngữ lớn cho mã nguồn (Large Language Models for Code – LLMs-for-code), với dữ liệu đầu vào là mã nguồn và đầu ra là kết luận về khả năng tồn tại lỗ hổng.

Đề tài sử dụng bộ dữ liệu công khai Juliet Test Suite cho các ngôn ngữ lập trình C/C++ và Java nhằm đánh giá hiệu quả của mô hình thông qua các chỉ số Accuracy, Precision, Recall và F1-score. Về phương pháp, nhóm giới hạn ở việc so sánh hai phương pháp prompting là Vanilla Prompt và Chain-of-Thought Prompt để phân tích ảnh hưởng của cách thiết kế prompt đến kết quả dự đoán.

Đề tài không tập trung vào việc thiết kế kiến trúc mô hình mới, không tối ưu tham số huấn luyện chuyên sâu và không triển khai hệ thống trong môi trường thực tế. Phạm vi nghiên cứu dừng ở mức khảo sát và phân tích thực nghiệm nhằm rút ra nhận xét về vai trò của prompting trong bài toán phát hiện lỗ hổng phần mềm.

1.3. Đối tượng nghiên cứu và hướng tiếp cận

Đối tượng nghiên cứu của đề tài là các mô hình ngôn ngữ lớn cho mã nguồn (Large Language Models for Code – LLMs-for-code), được xây dựng dựa trên kiến trúc Transformer và đã được sử dụng trong nhiều nghiên cứu về phân tích mã nguồn và phát hiện lỗ hổng phần mềm [4], [1]. Bên cạnh mô hình, đề tài còn xem xét vai trò của các chiến lược prompting, cụ thể là Vanilla Prompt và Chain-of-Thought Prompt, nhằm phân tích mức độ ảnh hưởng của cách thiết kế câu lệnh đầu vào đến hành vi suy luận và kết quả dự đoán của mô hình [11].

Về hướng tiếp cận, đề tài được thực hiện theo hướng thực nghiệm, trong đó nhóm áp dụng các mô hình LLMs-for-code vào bài toán phân loại mã nguồn trên bộ dữ liệu công khai Juliet Test Suite cho các ngôn ngữ lập trình C/C++ và Java [2]. Kết quả được đánh giá thông qua các chỉ số Accuracy, Precision, Recall và F1-score để phân tích hiệu quả và độ ổn định của mô hình. Đồng thời, các bước suy luận sinh ra từ Chain-of-Thought Prompt được sử dụng để xem xét khả năng hỗ trợ giải thích kết quả dự đoán, dựa trên các khái niệm về khả diễn giải trong học máy và trí tuệ nhân tạo [5].

1.4. Các nghiên cứu liên quan

Trong những năm gần đây, các mô hình ngôn ngữ lớn cho mã nguồn (Large Language Models for Code – LLMs-for-code) dựa trên kiến trúc Transformer đã trở thành một hướng tiếp cận nổi bật trong lĩnh vực phần mềm. Các mô hình tiền huấn luyện như CodeBERT cho thấy khả năng khai thác đồng thời thông tin từ mã nguồn và ngôn ngữ tự nhiên, từ đó hỗ trợ nhiều tác vụ như phân loại, dịch mã, sinh mã và hiểu ngữ nghĩa mã nguồn [4]. Bên cạnh đó, một số hướng nghiên cứu tập trung xây dựng các mô hình có khả năng vừa hiểu vừa sinh mã, thường dựa trên kiến trúc encoder–decoder, nhằm cải thiện khả năng biểu diễn và suy luận của mô hình trong các bài toán phân tích chương trình [1]. Trong bối cảnh an toàn phần mềm, nhiều công trình coi phát hiện lỗ hổng (vulnerability detection) như một bài toán phân loại dựa trên mã nguồn, nơi dữ liệu huấn luyện và đánh giá thường được chuẩn hóa theo nhãn rõ ràng và trong số đó có Juliet Test Suite là một bộ dữ liệu công khai được sử dụng rộng rãi nhờ có tập mẫu "good/bad" theo kịch bản lỗ hổng điển hình và hỗ trợ nhiều ngôn ngữ như C/C++ và Java [2]. Tuy vậy, một vấn đề thường gặp là mô hình có thể đạt kết quả tốt trên tập dữ liệu quen thuộc nhưng chưa chắc giữ được độ ổn định khi thay đổi đặc thù dự án, ngôn ngữ hoặc bối cảnh sử dụng, đặt ra yêu cầu cần đánh giá và so sánh một cách thận trọng hơn [8].

Một hướng nghiên cứu liên quan trực tiếp đến đề tài là vai trò của prompting trong việc điều khiển hành vi trả lời của mô hình. Prompt không chỉ dùng để cung cấp câu hỏi đầu vào cho mô hình, mà còn định hướng cách mô hình suy luận và đưa ra kết luận. Đặc biệt, Chain-of-Thought (CoT) prompting giúp mô hình thể hiện các bước suy luận trung gian thay vì chỉ trả lời kết quả cuối, từ đó có thể cải thiện cả chất lượng dự đoán và khả năng giải thích [11]. Trong các bài toán về mã nguồn và an toàn phần mềm, đa số nghiên cứu vẫn ưu tiên cải thiện mô hình hoặc dữ liệu, như fine-tuning hoặc thiết kế kiến trúc. Ngược lại, vai trò của prompting như một cơ chế can thiệp nhẹ, nghĩa là không cần thay đổi mô hình vẫn chưa được khảo sát đầy đủ,

đặc biệt về ảnh hưởng đến hiệu năng dự đoán và tính minh bạch của quá trình suy luận [13].

Ở góc độ giải thích mô hình (Explainable AI – XAI), các khảo sát gần đây cho thấy rằng việc mô hình có cung cấp lời giải thích không đồng nghĩa với việc lời giải thích đó thực sự tốt. Do đó, chất lượng giải thích cần được đánh giá dựa trên nhiều tiêu chí khác nhau, chẳng hạn như mức độ hữu ích đối với con người, mức độ phản ánh đúng hành vi của mô hình và chi phí tính toán [5]. Các nghiên cứu về đo lường explainability cho thấy hiện chưa có một thước đo duy nhất phù hợp cho mọi bối cảnh, mà các phương pháp đánh giá thường được lựa chọn tùy theo mục tiêu sử dụng [3]. Trong an toàn phần mềm, việc mô hình chỉ đưa ra kết luận có hoặc không có lỗ hổng mà thiếu lập luận khiến người dùng khó kiểm chứng và tin cậy kết quả. Trên cơ sở đó, đề tài thực hiện đánh giá thực nghiệm trên Juliet Test Suite [2] và so sánh Vanilla Prompt với Chain-of-Thought Prompt nhằm phân tích ảnh hưởng của prompting đến hiệu năng dự đoán và khả năng hỗ trợ giải thích của mô hình [11].

Chương 2. CƠ SỞ LÝ THUYẾT

Các cơ sở lý thuyết liên quan đến đề tài được trình bày trong Chương 2 nhằm làm rõ nền tảng khoa học cho quá trình nghiên cứu và thực nghiệm. Trước hết, chương này giới thiệu các khái niệm cơ bản trong an toàn phần mềm và bài toán phát hiện lỗ hổng từ mã nguồn, qua đó định vị bài toán nghiên cứu dưới góc nhìn phân loại mã nguồn. Tiếp theo, chương trình bày tổng quan về các mô hình ngôn ngữ lớn cho mã nguồn (Large Language Models for Code – LLMs-for-code), bao gồm kiến trúc nền tảng, đặc điểm hoạt động và khả năng ứng dụng trong phân tích mã nguồn. Bên cạnh đó, các khái niệm liên quan đến prompting, đặc biệt là Vanilla Prompt và Chain-of-Thought Prompt, được giới thiệu nhằm làm rõ vai trò của cách thiết kế prompt trong việc định hướng quá trình suy luận của mô hình. Cuối cùng, chương này đề cập đến mối liên hệ giữa prompting và khả năng hỗ trợ giải thích kết quả, tạo cơ sở lý thuyết cho các phân tích và đánh giá thực nghiệm được trình bày ở các chương tiếp theo.

2.1. Các khái niệm trong an toàn phần mềm

2.1.1. Lỗ hổng phần mềm (Software Vulnerability)

Lỗ hổng phần mềm (software vulnerability) là những điểm yếu hoặc sai sót tồn tại trong mã nguồn hoặc quá trình triển khai phần mềm, có thể bị khai thác để gây ra các hành vi không mong muốn như truy cập trái phép, làm sai lệch dữ liệu hoặc làm gián đoạn hoạt động của hệ thống [2]. Các lỗ hổng này thường phát sinh do việc xử lý dữ liệu đầu vào không an toàn, quản lý bộ nhớ không chặt chẽ hoặc sử dụng API và thư viện không đúng cách.

Trong quá trình phân tích mã nguồn, lỗ hổng phần mềm thường được thể hiện trực tiếp thông qua cấu trúc chương trình và các thao tác xử lý dữ liệu bên trong đoạn mã. Trên cơ sở đó, nhiều nghiên cứu tiếp cận phát hiện lỗ hổng như một bài toán học máy, trong đó mô hình học các đặc trưng từ mã nguồn đã được gán nhãn để dự đoán khả năng tồn tại lỗ hổng của một đoạn mã [4], [10].

2.1.2. Lỗ hổng phần mềm phổ biến

Trong thực tế, lỗ hổng phần mềm có thể được phân loại theo nhiều tiêu chí khác nhau (Bảng 2.1), chẳng hạn như nguyên nhân phát sinh, mức độ ảnh hưởng hoặc cách thức khai thác. Tuy nhiên, trong phạm vi đề tài, nhóm không đi sâu vào các chuẩn phân loại chi tiết, mà tập trung vào những nhóm lỗ hổng phổ biến thường xuất hiện trực tiếp trong mã nguồn và có ý nghĩa đối với bài toán phát hiện lỗ hổng tự động [2].

Đối với các ngôn ngữ như C/C++, các lỗ hổng thường liên quan đến quản lý bộ nhớ và thao tác con trỏ, tiêu biểu như buffer overflow, use-after-free hoặc out-of-bounds access. Trong khi đó, ở các ngôn ngữ bậc cao hơn như Java, các lỗ hổng phổ biến thường xuất phát từ việc xử lý dữ liệu đầu vào không an toàn, quản lý ngoại lệ không đầy đủ hoặc sử dụng tài nguyên không hợp lý. Những dạng lỗ hổng này có thể được nhận diện thông qua các mẫu cú pháp, cấu trúc chương trình hoặc luồng xử lý dữ liệu trong mã nguồn [2].

Để thuận tiện cho việc khảo sát và đánh giá trong các thí nghiệm, đề tài xem xét lỗ hổng phần mềm ở mức trừu tượng, tập trung vào việc một đoạn mã có hoặc không chứa lỗ hổng, thay vì phân tích chi tiết từng kiểu khai thác cụ thể. Cách tiếp cận này phù hợp với các bộ dữ liệu gán nhãn sẵn như Juliet Test Suite, nơi mỗi mẫu mã nguồn được xây dựng nhằm đại diện cho một kịch bản lỗ hổng điển hình hoặc một trường hợp an toàn tương ứng [2].

Nhóm lỗ hổng	**Mô tả**	**Ngôn ngữ thường gặp**
Buffer Overflow	Ghi dữ liệu vượt quá kích thước bộ đệm cho phép	C/C++
Use-after-free	Truy cập vùng nhớ đã được giải phóng	C/C++
Out-of-bounds Access	Truy cập ngoài phạm vi mảng hoặc vùng nhớ	C/C++

Improper Input Validation	Không kiểm tra hoặc lọc dữ liệu đầu vào	Java, C/C++
Resource Mismanagement	Quản lý tài nguyên không hợp lý (file, socket, bộ nhớ)	Java, C/C++

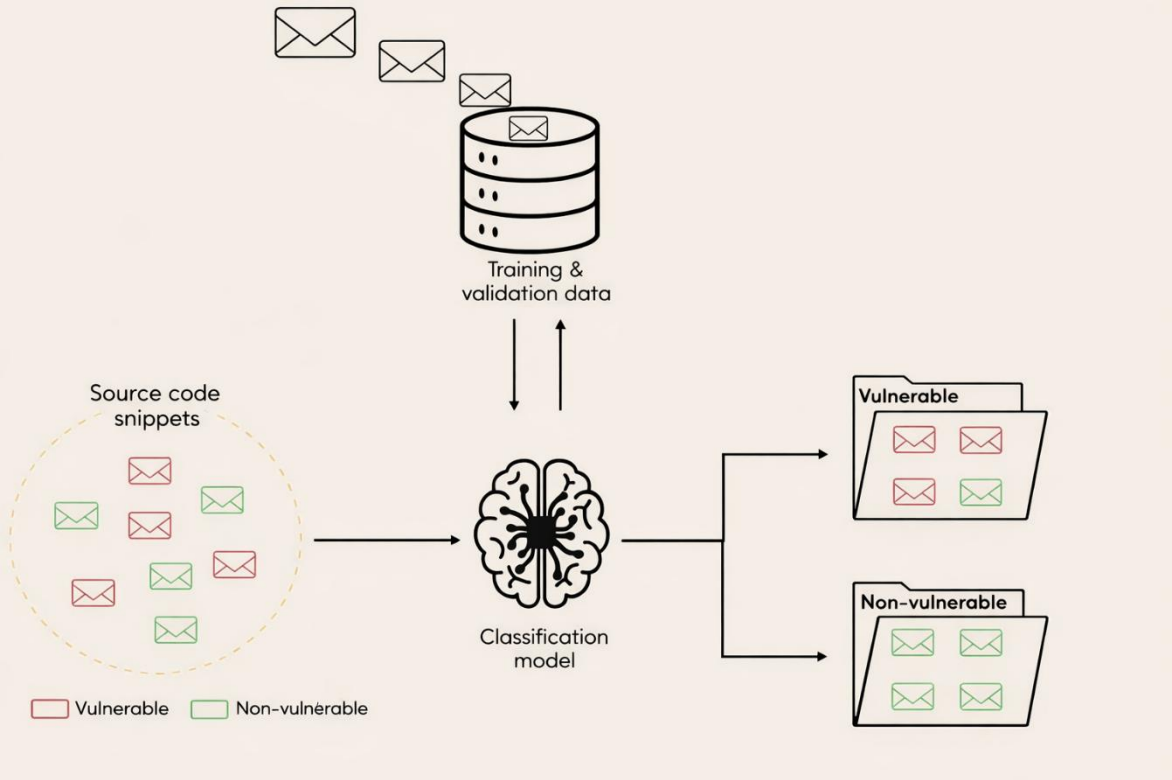
Bảng 2.1 Nhóm lỗ hổng phần mềm phổ biến trong mã nguồn

2.2. Phát hiện lỗ hổng từ mã nguồn

2.2.1. Bài toán phân loại

Trong nhiều nghiên cứu về an toàn phần mềm, bài toán phát hiện lỗ hổng từ mã nguồn thường được mô hình hóa như một bài toán phân loại mã nguồn (code classification), với mục tiêu xác định một đoạn mã có chứa lỗ hổng hay không dựa trên thông tin rút ra trực tiếp từ mã nguồn [10]. Theo cách tiếp cận này thì mã nguồn được xem như dữ liệu đầu vào tương tự văn bản trong xử lý ngôn ngữ tự nhiên, còn đầu ra là nhãn phản ánh trạng thái an toàn của đoạn mã (Hình 2.1).

Các mô hình ngôn ngữ lớn cho mã nguồn dựa trên kiến trúc Transformer đặc biệt phù hợp với bài toán này nhờ khả năng học ngữ cảnh và mối quan hệ giữa các thành phần chương trình [1]. Việc mô hình hóa phát hiện lỗ hổng dưới dạng phân loại không chỉ giúp đơn giản hóa quá trình đánh giá mà còn cho phép sử dụng các chỉ số chuẩn như Accuracy, Precision, Recall và F1-score, đồng thời phù hợp với các bộ dữ liệu công khai như Juliet Test Suite [2].



Hình 2.1 Minh họa bài toán phát hiện lỗ hổng

2.2.2. Mức độ phân tích mã nguồn

Trong bài toán phát hiện lỗ hổng từ mã nguồn, mức độ phân tích mã (granularity) ảnh hưởng trực tiếp đến cách biểu diễn dữ liệu đầu vào và khả năng học của mô hình. Mã nguồn thường được phân tích ở ba mức phổ biến: token, hàm và tệp.

Ở mức token, mã được chia thành các đơn vị cú pháp cơ bản, giúp mô hình học chi tiết cú pháp nhưng thường thiếu ngữ cảnh cần thiết cho các lỗ hổng liên quan đến luồng điều khiển hoặc quan hệ giữa các đoạn mã [4], [1]. Ở mức hàm, mỗi mẫu tương ứng với một hàm hoàn chỉnh, cho phép khai thác đồng thời cú pháp và ngữ nghĩa đồng thời phù hợp với cách gán nhãn của các bộ dữ liệu công khai như Juliet Test Suite [2].

Phân tích ở mức tệp cung cấp ngữ cảnh rộng hơn nhưng làm tăng độ phức tạp do chiều dài đầu vào lớn và chứa nhiều chức năng khác nhau và gây khó khăn cho việc

xác định chính xác lỗ hổng [10]. Vì vậy trong các nghiên cứu phân loại mã nguồn, bao gồm đề tài này, mức hàm hoặc đoạn mã thường được ưu tiên nhằm đảm bảo sự cân bằng giữa độ chi tiết, ngữ cảnh và tính nhất quán trong huấn luyện và đánh giá (Bảng 2.2).

Mức phân tích	**Đơn vị phân tích**	**Ưu điểm**	**Nhược điểm**
Token	Token	Phù hợp với mô hình, ngữ cảnh tốt	Mất thông tin cấu trúc
Hàm	Hàm/đoạn mã	Dễ gán nhãn, phổ biến	Thiếu ngữ cảnh toàn cục
Tệp	File	Bối cảnh đầy đủ	Chi phí tính toán cao

Bảng 2.2 So sánh các mức độ phân tích mã nguồn

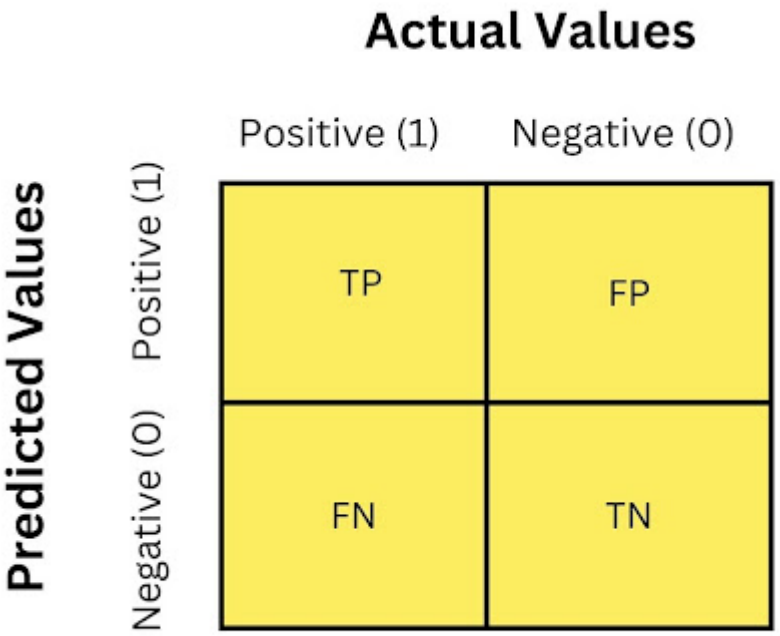
2.2.3. Phân loại vulnerable / non-vulnerable

Trong thực tế nghiên cứu và đánh giá các mô hình phát hiện lỗ hổng phần mềm, nhiều nghiên cứu lựa chọn mô hình hóa bài toán dưới dạng phân loại nhị phân, trong đó mỗi đoạn mã nguồn được gán nhãn có lỗ hổng hoặc không có lỗ hổng. Cách tiếp cận này phản ánh trực tiếp nhu cầu của các kịch bản kiểm thử an toàn, nơi mục tiêu ưu tiên là xác định sự tồn tại của rủi ro trong mã nguồn thay vì phân tích chi tiết từng loại lỗi cụ thể. Việc đơn giản hóa không gian nhãn giúp giảm độ phức tạp của bài toán, đồng thời tạo điều kiện thuận lợi cho huấn luyện và so sánh các mô hình học máy khác nhau.

Mô hình vul/non-vul cũng phù hợp với cách xây dựng của nhiều bộ dữ liệu công khai trong lĩnh vực an toàn phần mềm, tiêu biểu là Juliet Test Suite do NIST phát triển, nơi các đoạn mã được thiết kế theo cặp an toàn và không an toàn tương ứng với cùng một kịch bản lập trình. Nhờ đó, bài toán phát hiện lỗ hổng có thể được triển khai và đánh giá như một bài toán phân loại tiêu chuẩn, sử dụng các chỉ số định lượng phổ biến để đo lường hiệu năng của mô hình [2]. Trong bối cảnh sử dụng các mô hình ngôn ngữ lớn cho mã nguồn, phân loại vul/non-vul đóng vai trò

như một khung nền tảng, cho phép tập trung phân tích khả năng suy luận tổng quát của mô hình trước khi xem xét các yếu tố nâng cao như giải thích kết quả hay hỗ trợ sửa lỗi.

Hiệu năng của mô hình thường được đánh giá thông qua các chỉ số định lượng phổ biến, dựa trên ma trận nhầm lẫn (confusion matrix) (Hình 2.2), ở đó các thước đo được sử dụng rộng rãi trong các nghiên cứu phát hiện lỗ hổng phần mềm nhằm đảm bảo việc đánh giá mô hình không chỉ dựa trên độ chính xác tổng thể mà còn xem xét đến rủi ro bỏ sót lỗ hổng trong các kịch bản an toàn [2].



Hình 2.2 Confusion Matrix

(Nguồn: https://h2o.ai/wiki/confusion-matrix/)

2.2.4. Vai trò của bộ dữ liệu trong đánh giá mô hình

Trong bài toán phát hiện lỗ hổng từ mã nguồn, bộ dữ liệu giữ vai trò then chốt trong việc huấn luyện và đánh giá mô hình, do chất lượng và đặc điểm dữ liệu ảnh hưởng

trực tiếp đến khả năng học mẫu lỗ hổng và độ tin cậy của kết quả. Vì vậy, việc lựa chọn bộ dữ liệu phù hợp là yếu tố quan trọng trong các nghiên cứu về an toàn phần mềm.

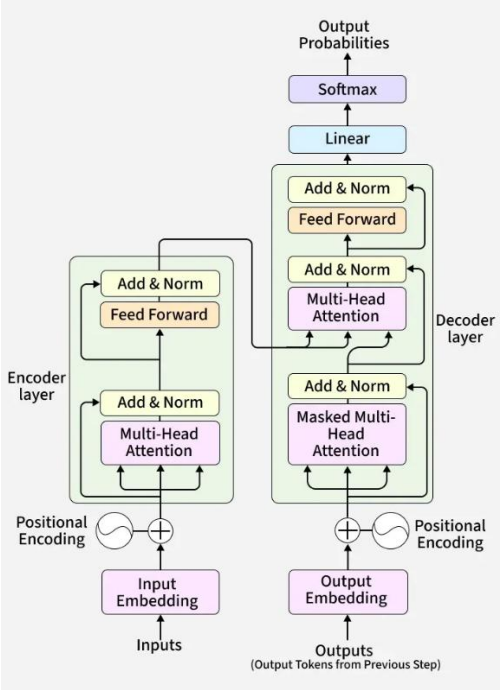
Nhiều nghiên cứu mô hình hóa bài toán dưới dạng phân loại nhị phân dựa trên các bộ dữ liệu có nhãn rõ ràng, trong đó Juliet Test Suite được sử dụng rộng rãi nhờ cấu trúc chuẩn hóa, các kịch bản lỗ hổng điển hình và hỗ trợ nhiều ngôn ngữ như C/C++ và Java [2]. Tuy nhiên, hiệu năng mô hình có thể phụ thuộc mạnh vào đặc thù của bộ dữ liệu và kết quả tốt trên dữ liệu chuẩn hóa chưa chắc phản ánh đầy đủ khả năng tổng quát hóa trong thực tế [10].

Trong phạm vi đề tài, bộ dữ liệu được sử dụng như nền tảng cố định để phân tích tác động của các chiến lược prompting đến kết quả dự đoán. Việc tập trung vào một bộ dữ liệu có nhãn rõ ràng giúp đảm bảo rằng sự khác biệt quan sát được chủ yếu xuất phát từ cách thiết kế prompt, từ đó hỗ trợ phân tích và diễn giải kết quả một cách nhất quán.

2.3. Mô hình ngôn ngữ lớn cho mã nguồn (LLMs-for-code)

2.3.1. Transformer

Kiến trúc Transformer (Hình 2.3) là nền tảng của hầu hết các mô hình ngôn ngữ lớn hiện nay, bao gồm cả các mô hình ngôn ngữ lớn cho mã nguồn (LLMs-for-code). Khác với các mô hình tuần tự truyền thống như RNN hay LSTM, Transformer sử dụng cơ chế self-attention (Hình 2.4) để mô hình hóa quan hệ giữa các phần tử trong toàn bộ chuỗi đầu vào, cho phép xử lý song song và nắm bắt các phụ thuộc dài hạn hiệu quả hơn [1].



Hình 2.3 Kiến trúc Transformer

(Nguồn: https://www.geeksforgeeks.org/deep-learning/architecture-and-working-of-transformers-in-deep-learning/)

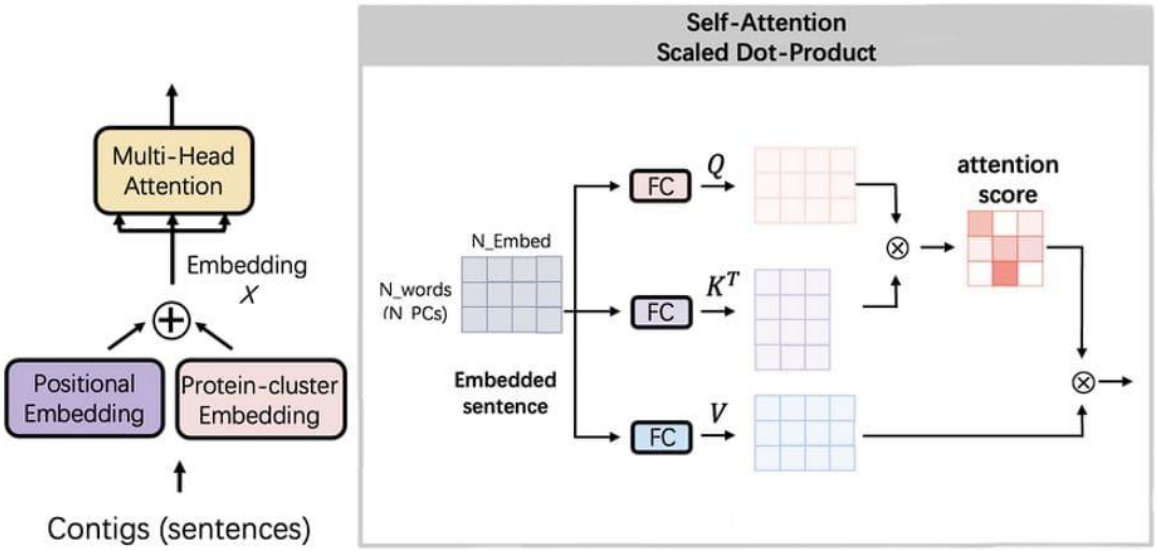
Trong ngữ cảnh mã nguồn, Transformer không chỉ học các mẫu cú pháp bề mặt mà còn có khả năng biểu diễn thông tin ngữ nghĩa thông qua mối quan hệ giữa các token. Mã nguồn được chuyển thành chuỗi token và ánh xạ sang các vector nhúng để đưa vào mô hình và nhờ cơ chế attention thì mô hình có thể học được các quan hệ như giữa biến và phép gán, điều kiện và khối lệnh, hoặc lời gọi hàm và định nghĩa tương ứng [4].

Transformer cũng có khả năng mở rộng linh hoạt sang nhiều kiến trúc khác nhau tùy theo mục tiêu sử dụng, như encoder-only cho các tác vụ phân tích và phân loại mã nguồn, hoặc encoder–decoder cho các tác vụ vừa hiểu vừa sinh mã [1]. Nhờ đó, LLMs-for-code có thể được áp dụng cho nhiều bài toán, bao gồm cả phát hiện lỗ hổng phần mềm.

Tuy nhiên, việc biểu diễn mã nguồn chủ yếu dựa trên chuỗi token tuyến tính trong khi chương trình mang cấu trúc chặt chẽ hơn (ví dụ cây cú pháp hoặc dòng chảy dữ

liệu) khiến khả năng "hiểu" logic chương trình của mô hình vẫn còn hạn chế, đặc biệt trong các bài toán an toàn phần mềm [10]. Nhận thức được điểm này là cơ sở để đề tài xem xét vai trò của prompting như một cách hỗ trợ mô hình tổ chức quá trình suy luận hiệu quả hơn.

Trong cơ chế self-attention, mỗi token được ánh xạ thành các vector query (Q), key (K) và value (V); mức độ liên quan giữa các token được xác định thông qua phép so khớp giữa Q và K, từ đó gán trọng số cho V tương ứng. Cơ chế này cho phép mô hình tập trung vào các thành phần quan trọng của chuỗi khi biểu diễn mỗi token, thay vì chỉ dựa vào vị trí tuần tự.



Hình 2.4 Self-attention

(Nguồn: https://viso.ai/deep-learning/sequential-models/)

Cụ thể, cơ chế scaled dot-product attention được tính như sau:

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V$$

Trong đó, mỗi token trong chuỗi đầu vào được ánh xạ thành ba vector gồm query (Q), key (K) và value (V) thông qua các phép biến đổi tuyến tính. Tích QK^T biểu

diễn mức độ tương đồng giữa các token, phản ánh mức độ liên quan giữa query và các key tương ứng. Giá trị này được chia cho $\sqrt{d_k}$ nhằm chuẩn hóa, giúp ổn định quá trình huấn luyện khi số chiều của vector tăng lên. Hàm softmax được sử dụng để chuyển các giá trị tương đồng thành các trọng số attention, từ đó tổng hợp các vector value và tạo ra biểu diễn đầu ra cho mỗi token.

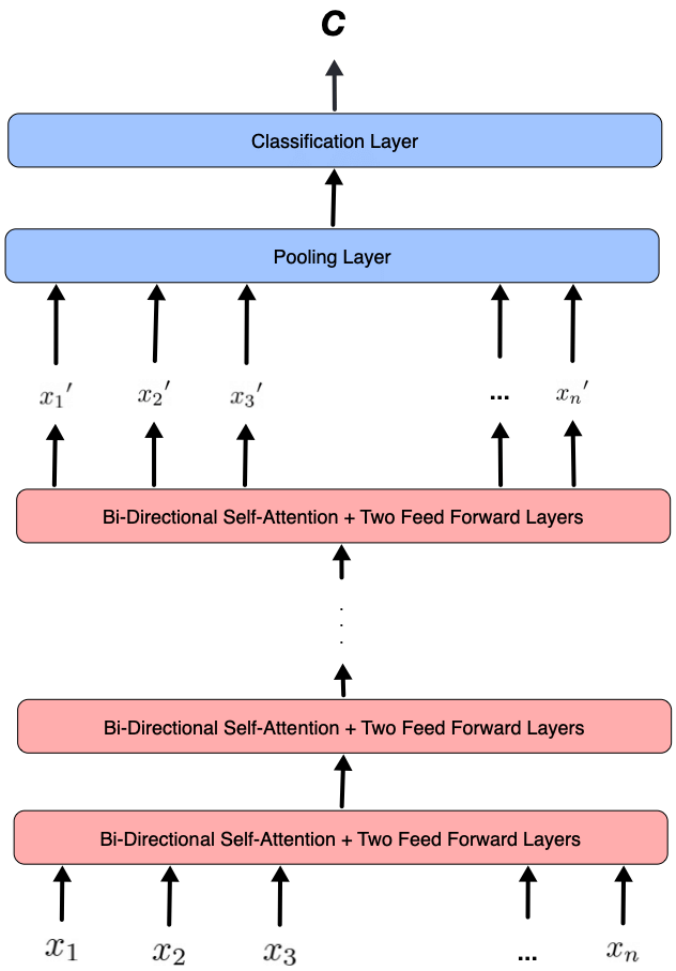
2.3.2. LLMs cho mã nguồn

2.3.2.1. Encoder-only

Các mô hình encoder-only (Hình 2.5) là một nhánh quan trọng của mô hình ngôn ngữ lớn cho mã nguồn, trong đó kiến trúc chỉ bao gồm khối encoder của Transformer. Nhờ khả năng học biểu diễn ngữ cảnh hai chiều, nhóm mô hình này đặc biệt phù hợp với các tác vụ phân tích mã nguồn như phân loại và phát hiện lỗ hổng, nơi đầu ra thường là một nhãn hoặc điểm số [4].

Trong lĩnh vực an toàn phần mềm, encoder-only thường được tiền huấn luyện trên tập lớn mã nguồn và ngôn ngữ tự nhiên, sau đó fine-tuning cho bài toán phân loại vulnerable / non-vulnerable, giúp tận dụng tri thức ngữ nghĩa đã học trong khi vẫn giữ kiến trúc đơn giản và ổn định [4], [10].

Tuy nhiên, encoder-only chủ yếu sinh ra biểu diễn cố định cho đầu vào và không trực tiếp hỗ trợ sinh lập luận bằng ngôn ngữ tự nhiên. Do đó, trong các kịch bản yêu cầu giải thích kết quả, mô hình thường cần kết hợp với các cơ chế XAI (Explainable artificial intelligence) hậu xử lý, đây cũng là cơ sở để đề tài so sánh encoder-only với các kiến trúc khác khi phân tích vai trò của prompting.



Hình 2.5 Kiến trúc encoder-only

(Nguồn: Farahani et al., Leveraging ParsBERT and Pretrained mT5 for Persian Abstractive Text Summarization, 2020)

Đầu vào của mô hình là chuỗi token $x_1, x_2, \dots, x_n$đại diện cho mã nguồn, sau khi được ánh xạ sang không gian vector nhúng. Chuỗi này được đưa qua nhiều lớp encoder, mỗi lớp bao gồm cơ chế self-attention hai chiều kết hợp với các tầng feed-forward, cho phép mô hình học được mối quan hệ ngữ cảnh giữa các token trong toàn bộ đoạn mã.

Sau khi đi qua các lớp encoder, mỗi token được biểu diễn bởi một vector ngữ cảnh $x_1', x_2', \dots, x_n'$. Các vector này được tổng hợp thông qua một tầng gộp (pooling layer),

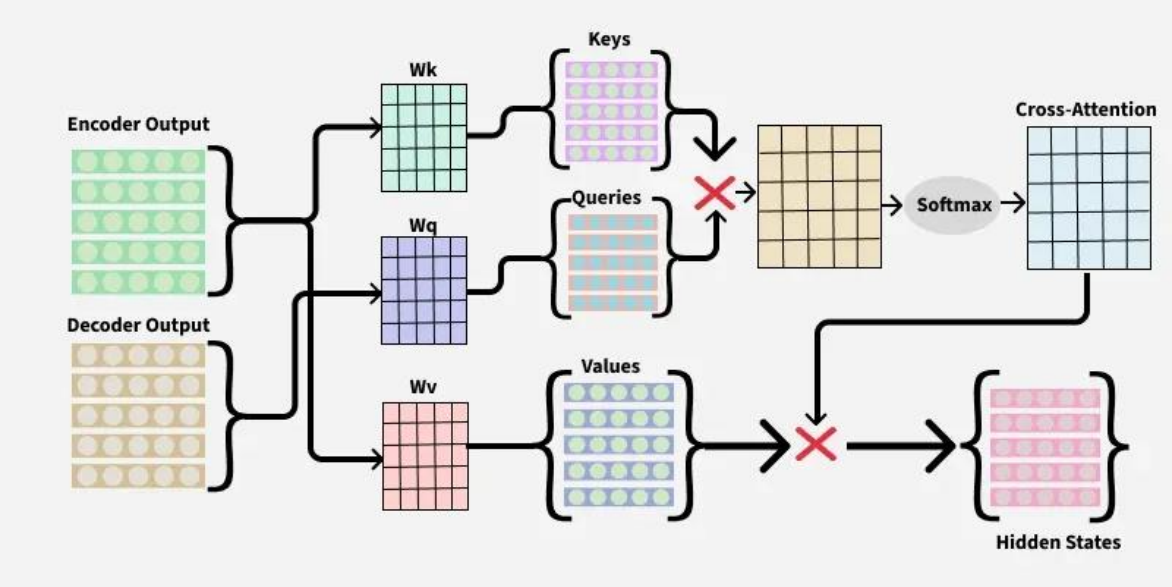
thường bằng cách sử dụng token đặc biệt hoặc phép gộp thống kê, nhằm tạo ra biểu diễn toàn cục cho đoạn mã. Biểu diễn này sau đó được đưa vào tầng phân loại (classification layer) để dự đoán nhãn đầu ra C, tương ứng với các lớp như vulnerable hoặc non-vulnerable. Cách tiếp cận này cho phép mô hình tập trung vào việc học biểu diễn và ngữ nghĩa của mã nguồn, phù hợp với các tác vụ phân loại trong an toàn phần mềm.

2.3.2.2. Encoder–Decoder

Kiến trúc encoder–decoder (Hình 2.3) là một hướng tiếp cận quan trọng trong các mô hình ngôn ngữ lớn cho mã nguồn, đặc biệt phù hợp với các bài toán mà đầu ra không chỉ là một nhãn mà còn là một chuỗi văn bản hoặc mã nguồn. Trong kiến trúc này, encoder mã hóa đoạn mã nguồn đầu vào thành các biểu diễn ngữ cảnh thông qua cơ chế self-attention hai chiều, trong khi decoder sinh chuỗi đầu ra theo từng bước dựa trên các biểu diễn đó.

Sự tương tác giữa encoder và decoder được thực hiện thông qua cross-attention (Hình 2.6), trong đó decoder sử dụng trạng thái hiện tại để truy vấn (Q) các biểu diễn đầu vào do encoder tạo ra (K, V). Nhờ cơ chế này, decoder có thể truy cập có chọn lọc đến thông tin từ mã nguồn, giúp duy trì mối liên kết chặt chẽ giữa đầu vào và đầu ra trong các tác vụ yêu cầu vừa hiểu vừa sinh nội dung.

Trong lĩnh vực phân tích mã nguồn, kiến trúc encoder–decoder cho phép mô hình thực hiện các tác vụ phức tạp hơn so với encoder-only, như sinh mô tả chương trình, giải thích lỗi hoặc đề xuất chỉnh sửa mã, và đã được chứng minh hiệu quả trong các nghiên cứu về tiền huấn luyện thống nhất cho hiểu và sinh mã [10], [1]. Tuy nhiên, do cơ chế sinh chuỗi tự hồi quy, các mô hình encoder–decoder thường có chi phí suy luận cao hơn, đặc biệt trong các bài toán phân loại nhị phân. Vì vậy, ưu thế chính của kiến trúc này không nằm ở tốc độ, mà ở khả năng sinh lập luận hoặc nội dung trung gian, khiến nó đặc biệt phù hợp khi kết hợp với các chiến lược prompting để phân tích và diễn giải quá trình suy luận của mô hình.

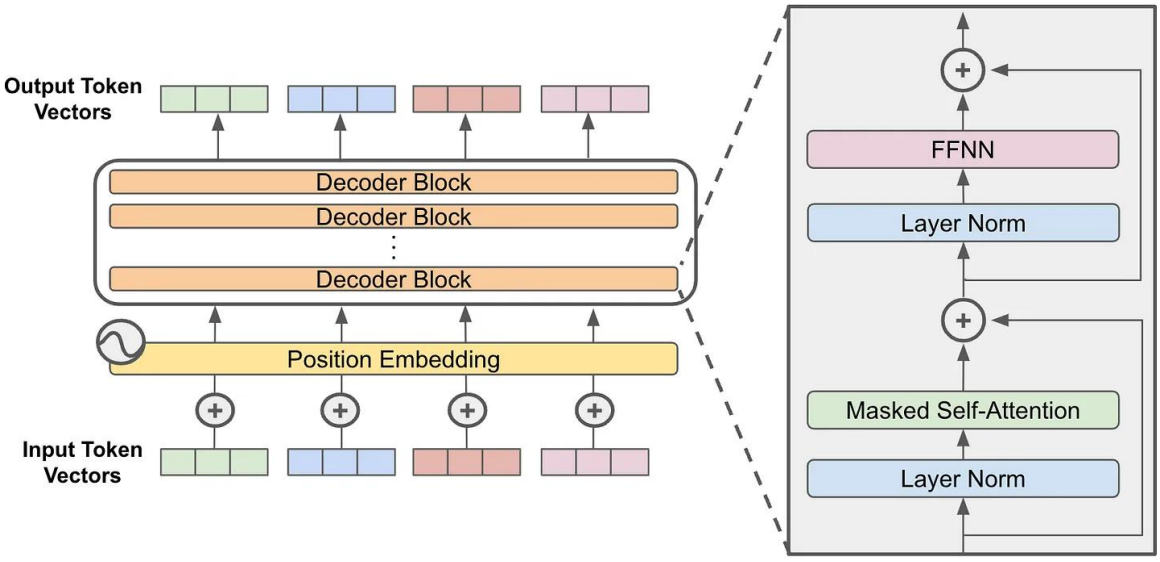


Hình 2.6 Cross-Attention

(Nguồn: https://www.geeksforgeeks.org/nlp/cross-attention-mechanism-in-transformers/)

2.3.2.3. Decoder-only

Các mô hình decoder-only là một nhánh quan trọng của mô hình ngôn ngữ lớn, trong đó kiến trúc chỉ bao gồm khối decoder của Transformer và không sử dụng encoder riêng biệt (Hình 2.7). Đặc trưng cốt lõi của nhóm mô hình này là cơ chế sinh chuỗi theo kiểu auto-regressive, tức là mô hình tạo đầu ra từng bước và trong đó mỗi token mới được dự đoán dựa trên các token đã sinh trước đó trong cùng chuỗi đầu vào/đầu ra. Nhờ cách tiếp cận này, decoder-only có khả năng mô hình hóa phân bố xác suất của chuỗi một cách tự nhiên và linh hoạt.



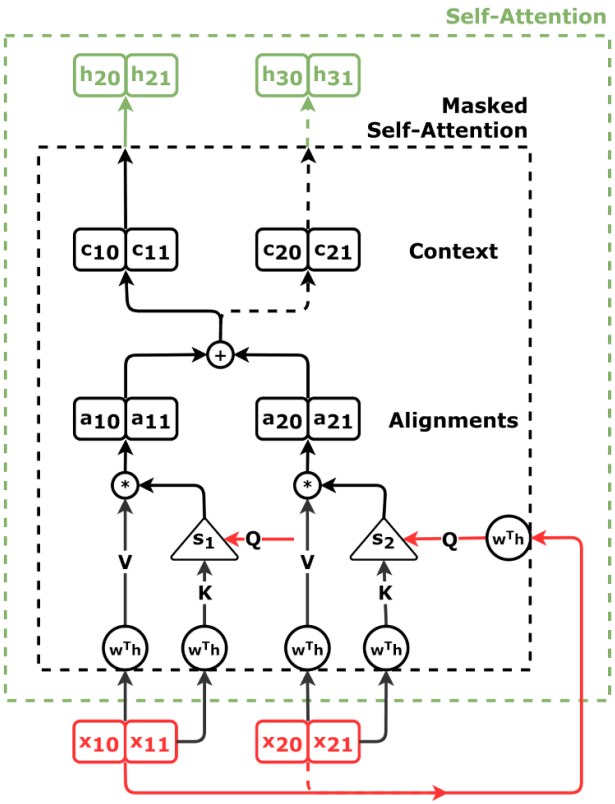
Hình 2.7 Kiến trúc Decoder-only

(Nguồn: https://cameronrwolfe.substack.com/p/decoder-only-transformers-the-workhorse)

Trong bối cảnh mã nguồn, mô hình decoder-only tiếp nhận chuỗi token (có thể là mã nguồn thuần hoặc kết hợp với câu lệnh – prompt) và học cách dự đoán token tiếp theo dựa trên ngữ cảnh trước đó. Cơ chế self-attention một chiều (causal attention) (Hình 2.8) được sử dụng để đảm bảo rằng tại mỗi thời điểm, mô hình chỉ truy cập được các token ở quá khứ, qua đó duy trì tính nhất quán của quá trình sinh chuỗi. Nhờ đặc điểm này, decoder-only đặc biệt phù hợp với các tác vụ sinh mã, hoàn thành mã hoặc sinh lời giải thích dạng văn bản cho mã nguồn.

Gần đây, nhiều mô hình decoder-only đã được phát triển chuyên biệt cho miền mã nguồn, tiêu biểu như StarCoder2 và Qwen2. Các mô hình này thường được huấn luyện trên tập dữ liệu lớn gồm mã nguồn đa ngôn ngữ, kết hợp với văn bản mô tả, giúp nâng cao khả năng hiểu cú pháp, ngữ nghĩa và bối cảnh lập trình. Các nghiên cứu thực nghiệm cho thấy mô hình decoder-only có năng lực mạnh trong việc nắm bắt ngữ nghĩa mã và phản hồi linh hoạt theo prompt, đặc biệt trong các kịch bản yêu cầu tương tác hoặc suy luận từng bước khi phân tích mã nguồn [8].

Tuy nhiên, so với encoder-only và encoder–decoder, mô hình decoder-only cũng tồn tại một số hạn chế. Do quá trình suy luận diễn ra theo cơ chế auto-regressive, chi phí tính toán trong giai đoạn suy luận thường cao hơn, đặc biệt khi chuỗi đầu ra dài. Ngoài ra, trong các bài toán phân loại nhị phân như phát hiện lỗ hổng vulnerable / non-vulnerable, decoder-only không khai thác trực tiếp ưu thế kiến trúc của mình nếu chỉ yêu cầu dự đoán nhãn cuối cùng. Thay vào đó, điểm mạnh của nhóm mô hình này nằm ở khả năng kết hợp với các chiến lược prompting để sinh lập luận hoặc giải thích bằng ngôn ngữ tự nhiên, từ đó hỗ trợ người dùng hiểu rõ hơn quá trình suy luận dẫn đến kết luận của mô hình [13], [8]. Bảng 2.3 giúp phân loại các mô hình dựa trên cách xử lý, đầu ra điển hình, đặc trưng nổi bật.



Hình 2.8 Self-attention một chiều (causal attention)

(Nguồn: https://commons.wikimedia.org/wiki/File%3ADecoder_self-attention_with_causal_masking%2C_detailed_diagram.png)

Loại mô hình	Cách xử lý	Đầu ra điển hình	Đặc trưng nổi bật
Encoder-only	Mã hóa toàn bộ chuỗi đầu vào	Nhãn / điểm số	Phù hợp cho bài toán phân loại
Encoder–Decoder	Mã hóa → sinh chuỗi	Chuỗi văn bản / nhãn	Hỗ trợ hiểu và sinh nội dung
Decoder-only	Sinh token theo prompt	Chuỗi văn bản	Linh hoạt, mạnh với prompting

Bảng 2.3 LLMs cho mã nguồn

2.3.3. Ưu điểm và hạn chế của LLMs-for-code

Các mô hình ngôn ngữ lớn cho mã nguồn (LLMs-for-code) mang lại nhiều lợi thế trong bài toán an toàn phần mềm nhờ khả năng học biểu diễn ngữ cảnh và ngữ nghĩa của mã nguồn ở quy mô lớn. Thông qua tiền huấn luyện trên dữ liệu đa dạng gồm mã nguồn và văn bản mô tả, các mô hình này có thể nắm bắt các mẫu lập trình phổ biến và mối quan hệ giữa các thành phần chương trình, từ đó đạt hiệu quả tốt trong các tác vụ phân tích mã nguồn, bao gồm phát hiện lỗ hổng dưới dạng phân loại vulnerable / non-vulnerable [4].

Một ưu điểm quan trọng khác của LLMs-for-code là tính linh hoạt trong sử dụng. Cùng một mô hình có thể được áp dụng cho nhiều tác vụ khác nhau thông qua fine-tuning hoặc điều chỉnh prompting, mà không cần thay đổi kiến trúc. Đặc biệt, các chiến lược prompting cho phép khai thác khả năng suy luận và diễn đạt của mô hình, hỗ trợ không chỉ dự đoán nhãn mà còn sinh ra các lập luận hoặc mô tả trung gian, góp phần nâng cao tính minh bạch của kết quả [11].

Tuy nhiên, LLMs-for-code cũng tồn tại một số hạn chế trong bối cảnh an toàn phần mềm. Việc biểu diễn mã nguồn chủ yếu dựa trên chuỗi token tuyến tính khiến mô hình có thể bỏ sót các quan hệ cấu trúc quan trọng như dòng điều khiển hoặc dòng

dữ liệu [10]. Ngoài ra, hiệu năng của mô hình phụ thuộc mạnh vào đặc thù bộ dữ liệu, và kết quả trên dữ liệu chuẩn hóa chưa chắc phản ánh đầy đủ khả năng tổng quát hóa trong thực tế [2]. Bên cạnh đó, dù mô hình có thể sinh lời giải thích bằng ngôn ngữ tự nhiên, chất lượng và độ tin cậy của các giải thích này vẫn cần được đánh giá cẩn trọng, do việc "trình bày" lập luận không luôn phản ánh đúng cơ chế nội tại của mô hình [5].

2.4. Các mô hình LLMs-for-code

2.4.1. Mô hình BART và PLBart

BART là mô hình ngôn ngữ dựa trên kiến trúc Transformer theo hướng encoder–decoder (Hình 2.3), ban đầu được đề xuất cho các tác vụ xử lý ngôn ngữ tự nhiên như sinh văn bản, tóm tắt và dịch máy [6]. Thông qua cơ chế tiền huấn luyện dạng denoising autoencoder, BART kết hợp khả năng mã hóa ngữ cảnh hai chiều của encoder với khả năng sinh chuỗi của decoder, giúp học được các biểu diễn ngữ nghĩa linh hoạt [1].

PLBart là phiên bản mở rộng của BART, được thiết kế riêng cho dữ liệu mã nguồn và các tác vụ phần mềm. Nhờ được tiền huấn luyện trên tập lớn gồm mã nguồn và thông tin liên quan như thông báo commit, PLBart thể hiện khả năng thích nghi tốt hơn với các tác vụ hiểu mã, sinh mã và phân tích thay đổi chương trình [10].

Trong bài toán phát hiện lỗ hổng phần mềm, kiến trúc encoder–decoder của PLBart cho phép mô hình sinh đầu ra có cấu trúc, như nhãn phân loại kèm theo lập luận trung gian. Điều này phù hợp với hướng tiếp cận của đề tài khi phân tích vai trò của prompting và Chain-of-Thought, giúp kết hợp giữa dự đoán và khả năng giải thích [11].

2.4.2. Mô hình StarCoder2

StarCoder2 là mô hình ngôn ngữ lớn cho mã nguồn thuộc nhóm decoder-only (Hình 2.7), được phát triển trong khuôn khổ dự án BigCode, một sáng kiến nghiên cứu mở nhằm xây dựng các mô hình cho mã nguồn có tính minh bạch và khả tái lập. Mô hình dựa trên kiến trúc Transformer sinh chuỗi tự hồi quy, trong đó mỗi token đầu ra được dự đoán dựa trên toàn bộ chuỗi token trước đó, phù hợp với các tác vụ sinh và phân tích mã nguồn theo trình tự [8].

Khác với các mô hình encoder–decoder, StarCoder2 không sử dụng pha mã hóa riêng biệt mà trực tiếp mô hình hóa mã nguồn như một chuỗi liên tục, cho phép duy trì ngữ cảnh dài và hỗ trợ tốt các kịch bản yêu cầu suy luận tuần tự. Mô hình được huấn luyện trên tập dữ liệu mã nguồn đa ngôn ngữ quy mô lớn, giúp cải thiện khả năng tổng quát hóa và hiểu cú pháp, ngữ nghĩa chương trình.

Trong phạm vi đề tài, StarCoder2 được lựa chọn như đại diện tiêu biểu cho nhóm mô hình decoder-only, đóng vai trò đối chứng với các kiến trúc encoder-based và encoder–decoder. Việc so sánh này giúp đánh giá ảnh hưởng của kiến trúc mô hình đến khả năng phát hiện lỗ hổng và mức độ đáp ứng đối với các chiến lược prompting khác nhau.

2.4.3. Mô hình Qwen2

Qwen2 là mô hình ngôn ngữ lớn được phát triển bởi nhóm nghiên cứu Qwen (thuộc Alibaba Cloud), trong đó có các phiên bản được huấn luyện và tối ưu cho cả dữ liệu ngôn ngữ tự nhiên và mã nguồn. Tương tự StarCoder2, Qwen2 thuộc nhóm mô hình decoder-only (Hình 2.7), sử dụng kiến trúc Transformer sinh chuỗi tự hồi quy (autoregressive), phù hợp với các tác vụ sinh văn bản, dịch mã và phân tích mã nguồn theo ngữ cảnh.

Một đặc điểm đáng chú ý của Qwen2 là khả năng xử lý chuỗi đầu vào có độ dài lớn, cho phép mô hình duy trì ngữ cảnh trong các đoạn mã hoặc chương trình có cấu trúc phức tạp. Điều này có ý nghĩa trong bài toán phân tích mã nguồn, khi thông tin liên quan đến lỗ hổng không chỉ nằm ở một dòng lệnh riêng lẻ mà có thể phụ thuộc vào mối quan hệ giữa nhiều thành phần trong chương trình. Nhờ cơ chế sinh chuỗi tuần tự, mô hình có thể kết hợp thông tin từ các token trước đó để đưa ra dự đoán nhất quán hơn.

Trong phạm vi đề tài, Qwen2 được sử dụng như một mô hình decoder-only đa năng, đại diện cho xu hướng các LLMs hiện đại được huấn luyện trên tập dữ liệu lớn và đa dạng. Việc đưa Qwen2 vào thí nghiệm nhằm mục đích so sánh ảnh hưởng của kiến trúc decoder-only đối với bài toán phát hiện lỗ hổng phần mềm, đồng thời đánh giá mức độ phản hồi của mô hình trước các chiến lược prompting khác nhau, đặc biệt là khi yêu cầu mô hình trình bày quá trình suy luận.

Mô hình	**Nhóm kiến trúc**	**Cơ chế xử lý**	**Vai trò trong đề tài**
CodeBERT	Encoder-only	Mã hóa ngữ nghĩa mã	Phân loại mã nguồn, baseline so sánh
PLBart	Encoder–Decoder	Hiểu và sinh mã nguồn	Đại diện encoder–decoder cho phân tích mã
BART	Encoder–Decoder	Sinh chuỗi từ văn bản	Mô hình nền tảng cho hướng encoder–decoder
StarCoder2	Decoder-only	Sinh mã tự hồi quy	Đánh giá khả năng suy luận theo prompt
Qwen2	Decoder-only	Sinh mã, xử lý ngữ cảnh dài	So sánh decoder-only đa năng

Bảng 2.4 Các mô hình LLMs-for-code sử dụng trong đề tài

Bảng 2.4 so sánh cho thấy sự khác biệt rõ ràng giữa các nhóm kiến trúc LLMs-for-code. Trong khi CodeBERT phù hợp cho các tác vụ phân loại thuần túy, các mô hình encoder–decoder và decoder-only thể hiện lợi thế khi kết hợp sinh chuỗi và lập luận, tạo tiền đề cho việc khảo sát ảnh hưởng của prompting trong các thí nghiệm ở chương tiếp theo.

2.5. Prompting

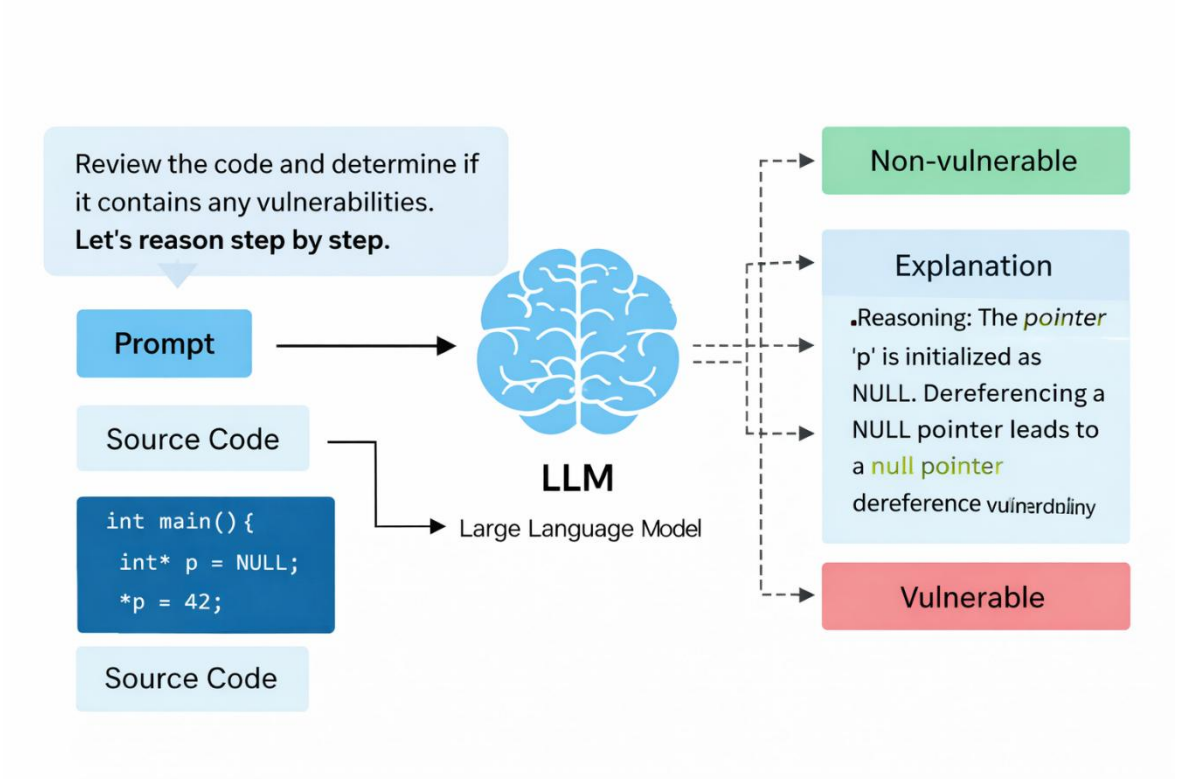
2.5.1. Khái niệm và vai trò của prompting

Trong các mô hình ngôn ngữ lớn, prompting được hiểu là quá trình thiết kế chuỗi đầu vào nhằm định hướng cách mô hình tiếp nhận thông tin, tổ chức suy luận và sinh kết quả (Hình 2.9). Khác với huấn luyện hoặc fine-tuning, prompting cho phép điều chỉnh hành vi của mô hình mà không cần can thiệp vào tham số, nhờ đó trở thành một cơ chế linh hoạt và chi phí thấp khi sử dụng LLMs.

Đối với các mô hình ngôn ngữ lớn cho mã nguồn (LLMs-for-code), prompting đóng vai trò đặc biệt quan trọng do mã nguồn vừa mang tính cú pháp vừa chứa ngữ nghĩa và ý đồ lập trình. Cách thiết kế prompt có thể ảnh hưởng trực tiếp đến việc mô hình tập trung vào các khía cạnh khác nhau của đoạn mã, dẫn đến sự khác biệt trong kết luận về khả năng tồn tại lỗ hổng.

Các nghiên cứu gần đây cho thấy prompting không chỉ tác động đến chất lượng dự đoán mà còn ảnh hưởng đến quá trình suy luận nội tại của mô hình, đặc biệt với các mô hình sinh chuỗi tự hồi quy. Việc thiết kế prompt phù hợp có thể khuyến khích mô hình suy luận theo từng bước, từ đó cải thiện khả năng giải thích kết quả [11].

Trong bối cảnh Explainable AI (XAI), prompting được xem là một hướng tiếp cận bổ sung cho các phương pháp giải thích truyền thống, cho phép mô hình trình bày lập luận dưới dạng ngôn ngữ tự nhiên. Tuy nhiên, hiệu quả của prompting phụ thuộc mạnh vào cách thiết kế prompt và vẫn chưa được khảo sát một cách hệ thống trong các bài toán an toàn phần mềm [13], đây cũng là động lực chính của đề tài.



Hình 2.9 Prompting định hướng quá trình phân tích mã nguồn.

2.5.2. Vanilla Prompt

Vanilla Prompt là hình thức prompting cơ bản nhất, trong đó mô hình được cung cấp trực tiếp yêu cầu cần thực hiện mà không kèm theo chỉ dẫn về cách suy luận hay các bước trung gian [7] (Hình 2.10). Đối với bài toán phát hiện lỗ hổng phần mềm, vanilla prompt thường được thiết kế dưới dạng câu hỏi ngắn gọn, yêu cầu mô hình xác định đoạn mã có tồn tại lỗ hổng hay không, đôi khi kèm theo yêu cầu nêu kết luận ngắn gọn (Hình 2.11). Cách tiếp cận này phản ánh cách sử dụng phổ biến của người dùng khi tương tác với các mô hình ngôn ngữ lớn trong thực tế.

Ưu điểm chính của vanilla prompt là tính đơn giản và dễ triển khai. Do không yêu cầu mô hình sinh ra quá trình suy luận chi tiết, thời gian phản hồi thường ngắn và chi phí tính toán thấp hơn so với các chiến lược prompting phức tạp. Trong nhiều nghiên cứu, vanilla prompt được sử dụng như một đường cơ sở (baseline) để đánh

giá hiệu năng ban đầu của mô hình trong các tác vụ phân loại hoặc dự đoán, bao gồm cả các bài toán liên quan đến mã nguồn và an toàn phần mềm [10].

Tuy nhiên, hạn chế của vanilla prompt là mô hình chỉ cung cấp kết quả cuối cùng mà không làm rõ quá trình lập luận dẫn đến kết luận đó. Điều này khiến người sử dụng khó đánh giá mức độ tin cậy của dự đoán, đặc biệt trong các kịch bản nhạy cảm như phát hiện lỗ hổng, nơi việc hiểu nguyên nhân cảnh báo quan trọng không kém bản thân kết quả. Ngoài ra, do thiếu định hướng về cách suy luận, mô hình có thể dựa nhiều vào các mẫu bề mặt trong dữ liệu huấn luyện, dẫn đến kết quả thiếu ổn định khi thay đổi ngữ cảnh hoặc đặc thù mã nguồn [13].

Trong phạm vi đề tài, vanilla prompt được sử dụng làm phương án tham chiếu để so sánh với các chiến lược prompting nâng cao hơn. Việc đánh giá hiệu năng của mô hình dưới vanilla prompt giúp làm rõ mức độ cải thiện khi áp dụng các cách thiết kế prompt có định hướng suy luận, từ đó nhấn mạnh vai trò của prompting trong bài toán phát hiện lỗ hổng phần mềm.

Hình 2.10 Vanilla prompt

Using Vanilla Prompt for Vulnerability Analysis

```python
from transformers import AutoTokenizer, AutoModelForCausalLM
import torch

# Load model and tokinizer
model_name = "bigcode/starcocder2-3b"
tokenizer = AutoTokenizer.from_pretrained(model_name)
model = AutoModelForCausalLM.from_pretrained(
        model_name,torch_dtype=toorc.float16,device_map="auto")

# Define the code snippet and vanilla prompt
code_snippet = "\\··
  int* p = NULL;
  *p = 42;
}
prompt = f[ Determine whether the following code is vulnerable.
  Answer with ONLY: Vulnerable or Non-vulnerable.
ʃ[
  # Run the model to generate output
  inputs = tokinzer(prompt, return_tensors="pt").to(model.device)
  with torch_no_grad():
  print(tokinzer.decode(output_ids[0], skip_special_tokens=True))
```

Hình 2.11 Vanilla Prompt trong python

2.5.3. Chain-of-Thought Prompt

Chain-of-Thought Prompt (CoT Prompt) là một chiến lược thiết kế prompt trong đó mô hình được khuyến khích trình bày các bước suy luận trung gian trước khi đưa ra câu trả lời cuối cùng (Hình 2.12). Thay vì chỉ sinh trực tiếp kết luận, CoT Prompt yêu cầu mô hình mô tả quá trình phân tích từng bước, từ đó làm rõ cách mô hình tiếp cận và giải quyết vấn đề (Hình 2.13).

Khái niệm này được giới thiệu và phân tích có hệ thống trong nghiên cứu của Wei et al., cho thấy việc bổ sung hướng dẫn suy luận từng bước có thể cải thiện đáng kể hiệu năng của các mô hình ngôn ngữ lớn trong những bài toán đòi hỏi lập luận phức tạp, như suy luận logic, toán học hoặc phân tích có cấu trúc [11]. Về bản chất, CoT Prompt không thay đổi kiến trúc hay tham số mô hình, mà chỉ tác động ở mức đầu vào, giúp mô hình khai thác tốt hơn năng lực suy luận đã có từ giai đoạn tiền huấn luyện.

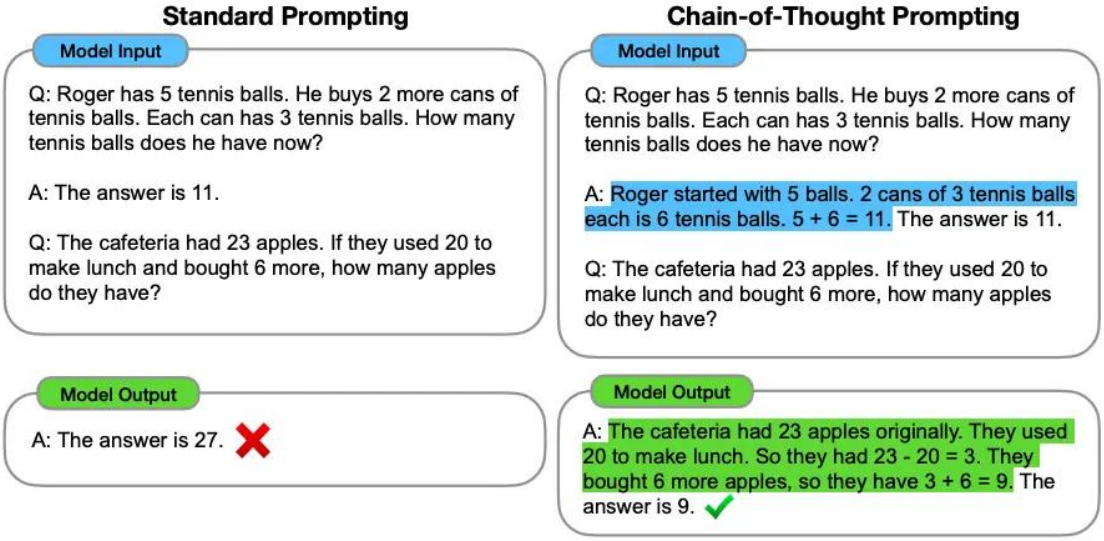
Trong bài toán phân tích mã nguồn và phát hiện lỗ hổng phần mềm, Chain-of-Thought Prompt thường được xây dựng dưới dạng yêu cầu mô hình lần lượt:

- Xác định các thao tác nhạy cảm trong mã.
- Kiểm tra việc sử dụng bộ nhớ, con trỏ hoặc dữ liệu đầu vào.
- Đưa ra kết luận về khả năng tồn tại lỗ hổng.

Cách tiếp cận này giúp mô hình không chỉ trả lời "có" hoặc "không", mà còn thể hiện được chuỗi lập luận dẫn tới kết luận, qua đó hỗ trợ người đọc hiểu rõ hơn lý do mô hình đưa ra dự đoán.

Dưới góc độ khả diễn giải, các bước suy luận được sinh ra từ CoT Prompt có thể được xem như một dạng giải thích ngầm (implicit explainability), giúp tăng mức độ minh bạch và độ tin cậy của kết quả mà không cần áp dụng thêm các kỹ thuật XAI hậu xử lý phức tạp [5]. Tuy nhiên, cần lưu ý rằng các bước suy luận này không phải lúc nào cũng phản ánh chính xác toàn bộ cơ chế nội tại của mô hình, mà chủ yếu đóng vai trò hỗ trợ con người trong việc theo dõi và đánh giá kết quả dự đoán.

Trong phạm vi đề tài, Chain-of-Thought Prompt được sử dụng như một phương pháp so sánh trực tiếp với Vanilla Prompt nhằm đánh giá ảnh hưởng của chiến lược prompting đến cả hiệu năng phân loại lỗ hổng và khả năng hỗ trợ giải thích kết quả của các mô hình LLMs-for-code.



Hình 2.12 Chain-of-Thought (CoT) Prompting

(Nguồn: https://www.promptingguide.ai/techniques/cot)

```
Your task is to complete the following code. You should first write a
rough problem-solving process then output the final code.

Here are some demonstration examples:

def sum_Of_Primes(n):
    """
    Write a python function to find sum of prime numbers between 1 to n.
    """
    # Let's think step by step
    # 1. prime <- [True] * (n + 1)
    # 2. p <- 2
    # 3. while p * p <= n:
    # 4.   if prime[p] == True:
    # 5.     for each i in range(p * 2, n + 1, p):
    # 6.       prime[i] <- False
    # 7.   p += 1
    # 8. sum <- 0
    # 9. for each i in range(2, n + 1):
    # 10.   if prime[i]:
    # 11.     sum += i
    # 12. return sum
    prime = [True] * (n + 1)
     (more lines…)
(more examples…)

Input code:
def text_lowercase_underscore(text):
    """
    Write a function to find sequences of lowercase letters joined with
an underscore.
    """
    # Let's think step by step
```

Hình 2.13 Chain-of-Thought (CoT) Prompting trong python

(Nguồn: https://dl.acm.org/doi/10.1145/3690635)

2.5.4. Prompting trong bài toán phân tích mã nguồn

Trong bài toán phân tích mã nguồn, prompt không chỉ đóng vai trò cung cấp dữ liệu đầu vào cho mô hình, mà còn quyết định cách mô hình tiếp cận và suy luận về đoạn mã cần phân tích. Khác với các tác vụ xử lý ngôn ngữ tự nhiên thông thường, mã nguồn mang tính cấu trúc chặt chẽ, phụ thuộc mạnh vào ngữ cảnh và logic thực thi. Do đó, cách thiết kế prompt có ảnh hưởng trực tiếp đến việc mô hình có hiểu đúng ý nghĩa của đoạn mã và đưa ra kết luận phù hợp hay không.

Đối với các mô hình ngôn ngữ lớn cho mã nguồn (LLMs-for-code), prompt thường bao gồm ba thành phần chính:

- đoạn mã nguồn cần phân tích.
- mô tả nhiệm vụ (ví dụ: xác định khả năng tồn tại lỗ hổng).
- yêu cầu về định dạng hoặc cách trả lời.

Trong cách tiếp cận Vanilla Prompt, mô hình thường được yêu cầu trả lời trực tiếp câu hỏi phân loại, dẫn đến kết quả ngắn gọn nhưng thiếu thông tin về quá trình suy luận. Ngược lại, Chain-of-Thought Prompt bổ sung các chỉ dẫn nhằm khuyến khích mô hình phân tích từng bước, từ đó tạo ra chuỗi lập luận trung gian trước khi đưa ra kết luận cuối cùng [11].

Trong bối cảnh phát hiện lỗ hổng phần mềm, prompting ảnh hưởng không chỉ đến hiệu năng dự đoán mà còn đến khả năng hỗ trợ người dùng hiểu kết quả của mô hình. Khi mô hình được dẫn dắt phân tích các yếu tố như xử lý dữ liệu đầu vào, truy cập bộ nhớ hoặc lời gọi hàm nhạy cảm, các bước suy luận sinh ra có thể đóng vai trò như một dạng giải thích ngầm, giúp người đọc theo dõi logic ra quyết định của mô hình mà không cần áp dụng thêm phương pháp XAI chuyên biệt [13].

Để làm rõ sự khác biệt giữa các chiến lược prompting trong bài toán phân tích mã nguồn, Bảng 2.5 trình bày so sánh tổng quan giữa Vanilla Prompt và Chain-of-Thought Prompt theo các khía cạnh liên quan trực tiếp đến đề tài.

Tiêu chí	**Vanilla Prompt**	**Chain-of-Thought Prompt**
Mục tiêu chính	Trả lời trực tiếp kết quả	Khuyến khích suy luận từng bước
Cách mô hình xử lý mã nguồn	Phân tích ngầm, không hiển thị lập luận	Phân tích có cấu trúc theo các bước
Dạng đầu ra	Nhãn hoặc kết luận ngắn	Kết luận kèm chuỗi lập luận
Khả năng hỗ trợ giải thích	Thấp	Cao
Chi phí suy luận	Thấp hơn	Cao hơn do sinh thêm token
Phù hợp cho phát hiện lỗ hổng	Đánh giá nhanh	Phân tích, kiểm chứng và giải thích

Bảng 2.5 Các chiến lược prompting

2.6. Prompting và khả năng hỗ trợ giải thích kết quả

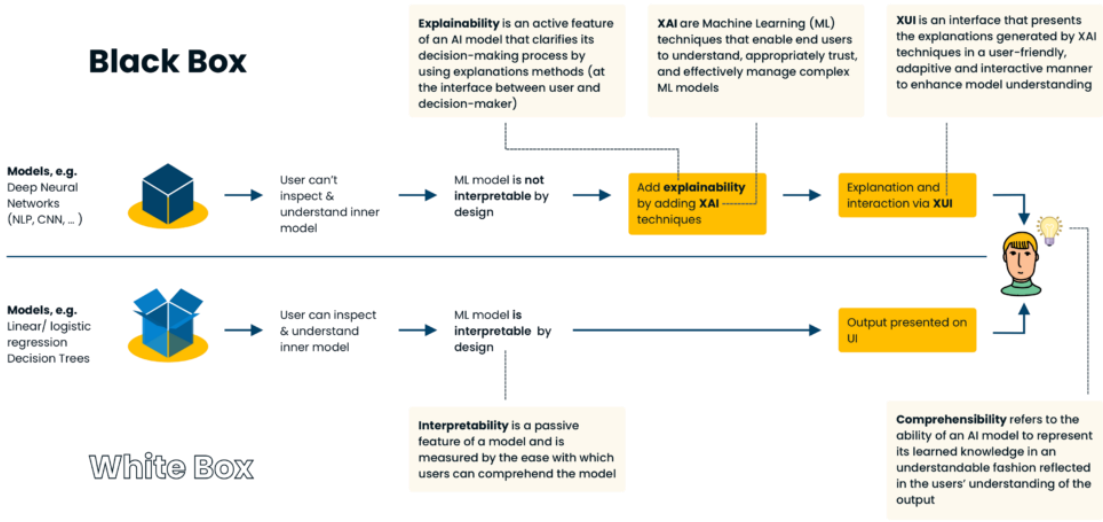
2.6.1. Explainable AI (XAI)

Explainable AI (XAI) là một hướng nghiên cứu nhằm làm rõ cách thức và lý do mà mô hình học máy đưa ra dự đoán hoặc quyết định (Hình 2.14). Mục tiêu cốt lõi của XAI không chỉ dừng ở việc cải thiện độ chính xác, mà còn hướng đến tăng tính minh bạch, khả kiểm chứng và mức độ tin cậy của hệ thống, đặc biệt trong các lĩnh vực nhạy cảm như y tế, tài chính và an toàn phần mềm [5].

Các nghiên cứu tổng quan thường phân loại XAI theo hai hướng chính. Thứ nhất là giải thích nội tại (intrinsic explainability), trong đó bản thân mô hình được thiết kế sao cho cấu trúc và cơ chế hoạt động của nó có thể được con người hiểu trực tiếp. Thứ hai là giải thích hậu xử lý (post-hoc explainability), áp dụng các phương pháp bên ngoài để phân tích hoặc xấp xỉ hành vi của mô hình đã huấn luyện, tiêu biểu như LIME, SHAP hay Integrated Gradients [5].

Trong bối cảnh các mô hình ngôn ngữ lớn, đặc biệt là LLMs-for-code, vấn đề giải thích trở nên phức tạp hơn do kiến trúc nhiều tầng, số lượng tham số rất lớn và đầu ra mang tính chuỗi. Các khảo sát gần đây chỉ ra rằng mặc dù nhiều phương pháp XAI đã được đề xuất, chưa tồn tại một cách tiếp cận thống nhất có thể vừa phản ánh trung thực hành vi mô hình, vừa dễ hiểu và hiệu quả về mặt chi phí tính toán [13]. Do đó, việc lựa chọn và đánh giá phương pháp giải thích thường phụ thuộc mạnh vào ngữ cảnh sử dụng và mục tiêu của bài toán.

Ngoài ra, một số nghiên cứu nhấn mạnh rằng việc đánh giá XAI không chỉ dựa trên hình thức giải thích, mà còn cần xem xét các tiêu chí như mức độ hợp lý đối với con người, mức độ trung thực với mô hình và khả năng hỗ trợ ra quyết định trong thực tế [3]. Những hạn chế này đặt ra yêu cầu phải tìm kiếm các hướng tiếp cận mới, phù hợp hơn với đặc thù của LLMs, đặc biệt trong các bài toán phân tích và phát hiện lỗ hổng phần mềm.



Hình 2.14 Explainable AI (XAI) giúp các mô hình trở nên dễ hiểu hơn

(Nguồn: https://papermaker.ai/explainable-ai/)

2.6.2. Hạn chế của XAI truyền thống đối với LLMs

Mặc dù Explainable AI (XAI) đã được nghiên cứu rộng rãi, các phương pháp XAI truyền thống vẫn gặp nhiều hạn chế khi áp dụng cho các mô hình ngôn ngữ lớn (LLMs), đặc biệt trong phân tích mã nguồn. Phần lớn các kỹ thuật như LIME và SHAP thuộc nhóm giải thích hậu xử lý (post-hoc), tức là tạo lời giải thích sau khi mô hình đã đưa ra dự đoán, thay vì phản ánh trực tiếp quá trình suy luận nội tại của mô hình [9], [12].

Với các LLMs có quy mô tham số lớn, XAI hậu xử lý thường tốn kém chi phí tính toán và khó mở rộng, nhất là khi xử lý mã nguồn dài. Bên cạnh đó, các lời giải thích dưới dạng điểm số hoặc bản đồ đóng góp thường khó diễn giải và chưa đảm bảo tính trung thực, khi chúng không nhất thiết phản ánh đúng các yếu tố mà mô hình thực sự sử dụng để ra quyết định [5], [3]. Ngoài ra, việc triển khai XAI như một thành phần bên ngoài mô hình cũng làm tăng độ phức tạp hệ thống. Những hạn chế này cho thấy nhu cầu về các cách tiếp cận giải thích gắn trực tiếp với quá trình sinh kết quả của mô hình, thay vì chỉ dựa vào giải thích hậu xử lý.

2.6.3. Chain-of-Thought như một dạng giải thích ngầm

Chain-of-Thought (CoT) prompting là cơ chế khuyến khích mô hình sinh ra các bước suy luận trung gian trước khi đưa ra kết quả cuối cùng, thay vì chỉ trả lời trực tiếp nhãn dự đoán [11]. Khác với các phương pháp Explainable AI hậu xử lý, CoT gắn trực tiếp việc "giải thích" vào quá trình sinh đầu ra của mô hình.

Trong các mô hình ngôn ngữ lớn cho mã nguồn, CoT có thể được xem như một dạng giải thích ngầm (implicit explainability), khi các bước lập luận là sản phẩm trực tiếp của quá trình sinh chuỗi, giúp người đọc theo dõi cách mô hình phân tích mã và đi đến kết luận về lỗ hổng. Cách tiếp cận này không yêu cầu thay đổi kiến trúc mô hình, chỉ cần điều chỉnh prompt, đồng thời giảm chi phí tính toán so với XAI hậu xử lý [13], [5].

Tuy nhiên, các bước suy luận sinh ra vẫn không đảm bảo phản ánh đầy đủ cơ chế nội tại của mô hình. Do đó, trong phạm vi đề tài, CoT được sử dụng như một công cụ hỗ trợ phân tích hành vi mô hình, và việc so sánh giữa Vanilla Prompt và Chain-of-Thought Prompt nhằm đánh giá tiềm năng của prompting trong việc nâng cao khả diễn giải của kết quả phát hiện lỗ hổng.

Chương 3. PHƯƠNG PHÁP THỰC HIỆN

3.1. Kiến trúc tổng quát

Trong đề tài này, nhóm đề xuất một phương pháp khảo sát khả năng phát hiện lỗ hổng phần mềm của các mô hình ngôn ngữ lớn cho mã nguồn (LLMs-for-code) thông qua việc kết hợp giữa dữ liệu mã nguồn, chiến lược prompting và các chỉ số đánh giá định lượng. Phương pháp được thiết kế theo hướng mô-đun, cho phép phân tích riêng rẽ vai trò của từng thành phần trong toàn bộ quy trình phát hiện lỗ hổng, đồng thời đảm bảo tính linh hoạt khi áp dụng cho các mô hình và kịch bản thực nghiệm khác nhau.

Quy trình tổng thể của phương pháp gồm các bước chính:

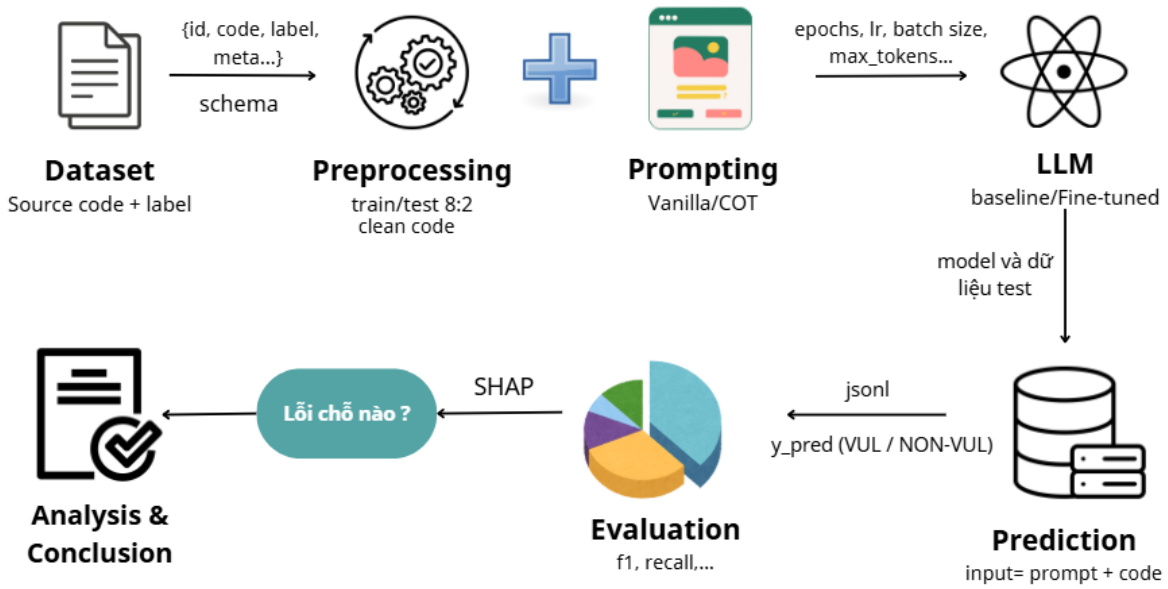
- Thu thập và chuẩn hóa dữ liệu mã nguồn
- Tiền xử lý và chia tập dữ liệu phục vụ huấn luyện và đánh giá
- Sử dụng mô hình ngôn ngữ lớn ở hai thiết lập là mô hình gốc (base) và mô hình được tinh chỉnh (fine-tuned)
- Áp dụng các chiến lược prompting khác nhau để sinh dự đoán
- Đánh giá kết quả dự đoán bằng các chỉ số định lượng
- Phân tích, giải thích kết quả thông qua các cơ chế hỗ trợ giải thích.

Toàn bộ luồng xử lý được minh họa trong Hình 3.1, thể hiện mối quan hệ và thứ tự thực hiện giữa các thành phần trong phương pháp đề xuất.

Điểm trọng tâm của phương pháp nằm ở việc giữ nguyên kiến trúc và tham số của mô hình trong từng kịch bản thí nghiệm, đồng thời chỉ thay đổi cách thiết kế prompt đầu vào. Cách tiếp cận này cho phép nhóm đánh giá trực tiếp ảnh hưởng của chiến lược prompting—cụ thể là Vanilla Prompt và Chain-of-Thought Prompt—đến hiệu năng phát hiện lỗ hổng và khả năng diễn giải kết quả của mô hình. Nhờ đó, đề tài tập trung làm rõ vai trò của prompting như một cơ chế điều khiển hành vi suy luận

của LLMs-for-code, thay vì phụ thuộc vào việc thiết kế mô hình mới hay tối ưu hóa huấn luyện chuyên sâu.

Cuối cùng, kết quả dự đoán và các thông tin giải thích thu được từ mô hình được tổng hợp và phân tích nhằm rút ra các nhận xét về mức độ hiệu quả, tính ổn định cũng như tiềm năng ứng dụng của phương pháp trong bài toán phát hiện lỗ hổng phần mềm từ mã nguồn. Phần phân tích này đóng vai trò cầu nối giữa phương pháp đề xuất và các kết quả thực nghiệm được trình bày ở chương tiếp theo.



Hình 3.1 Mô hình tổng quan

3.2. Chuẩn bị và tiền xử lý dữ liệu

3.2.1. Chuẩn hóa dữ liệu đầu vào

Trong phương pháp đề xuất, dữ liệu đầu vào được giả định là các mẫu mã nguồn đã được gán nhãn phục vụ cho bài toán phân loại. Để đảm bảo tính nhất quán trong toàn bộ pipeline, nhóm chuẩn hóa cấu trúc dữ liệu về một schema chung, trong đó mỗi mẫu bao gồm tối thiểu các trường: id, code và label. Một số metadata bổ sung (ví dụ: ngôn ngữ lập trình hoặc loại lỗ hổng) được giữ lại để phục vụ phân tích mở rộng ở các bước sau.

3.2.2. Tiền xử lý mã nguồn

Trước khi đưa vào mô hình, dữ liệu được tiền xử lý nhằm giảm nhiễu và cải thiện độ tin cậy của đầu vào. Các bước bao gồm loại bỏ các mẫu thiếu mã nguồn hoặc nhãn, loại bỏ mã rỗng và phát hiện các mẫu trùng lặp dựa trên nội dung mã. Mã nguồn sau đó được chuẩn hóa ở mức cú pháp cơ bản (xử lý khoảng trắng và định dạng dòng) để đảm bảo tính nhất quán giữa các mẫu.

Đối với các đoạn mã có độ dài lớn, nhóm áp dụng giới hạn độ dài đầu vào nhằm phù hợp với khả năng xử lý của mô hình, đồng thời ưu tiên giữ lại phần thân hàm và các đoạn mã liên quan đến xử lý dữ liệu hoặc thao tác nhạy cảm.

3.2.3. Tổ chức tập dữ liệu cho thí nghiệm

Sau khi tiền xử lý, dữ liệu được chia thành các tập phục vụ huấn luyện và đánh giá theo tỉ lệ cố định. Việc chia tập được thực hiện sau bước tiền xử lý và đảm bảo phân bố nhãn giữa các tập là tương đồng, nhằm hạn chế hiện tượng rò rỉ dữ liệu và đảm bảo tính nhất quán trong đánh giá mô hình. Quy trình này được áp dụng độc lập cho từng ngôn ngữ lập trình để phản ánh đúng khả năng học và tổng quát hóa của mô hình trong từng kịch bản.

3.3. LLM (Base / Fine-tune)

Trong đề tài này, nhóm sử dụng các mô hình ngôn ngữ lớn cho mã nguồn (LLMs-for-code) theo hai nhánh xử lý khác nhau, bao gồm nhánh sử dụng mô hình cơ sở (base model) và nhánh sử dụng mô hình được tinh chỉnh (fine-tuned). Việc tách hai nhánh này nhằm mục đích so sánh hiệu quả phát hiện lỗ hổng giữa mô hình chưa được huấn luyện chuyên biệt và mô hình đã được điều chỉnh theo dữ liệu của bài toán.

3.3.1. Nhánh mô hình cơ sở (Base model)

Ở nhánh base model, đầu vào của hệ thống bao gồm mô hình LLMs-for-code ở trạng thái tiền huấn luyện ban đầu và các prompt template được thiết kế cho bài toán phát hiện lỗ hổng. Trong thiết lập này, mô hình không trải qua quá trình huấn luyện bổ sung trên dữ liệu của đề tài mà được sử dụng trực tiếp để suy luận trên tập dữ liệu kiểm thử hoặc tập đánh giá.

Quá trình xử lý của nhánh này bao gồm việc đưa từng mẫu mã nguồn vào mô hình thông qua prompt tương ứng và thực hiện suy luận để sinh ra kết quả dự đoán. Cách tiếp cận này tương ứng với các kịch bản zero-shot hoặc few-shot, trong đó mô hình dựa hoàn toàn vào kiến thức đã học trong giai đoạn tiền huấn luyện và thông tin được cung cấp trong prompt.

Đầu ra của nhánh base model là các nhãn dự đoán cho từng mẫu mã nguồn, được sử dụng làm kết quả baseline để so sánh với các mô hình đã được tinh chỉnh ở nhánh tiếp theo.

3.3.2. Nhánh mô hình tinh chỉnh (Fine-tune)

Ở nhánh fine-tune, đầu vào bao gồm mô hình LLMs-for-code ở trạng thái ban đầu cùng với tập dữ liệu huấn luyện và đánh giá. Mô hình được tinh chỉnh dựa trên các tham số huấn luyện như số vòng lặp, tốc độ học, kích thước batch và giới hạn độ dài đầu vào, nhằm điều chỉnh tham số của mô hình cho phù hợp với nhiệm vụ phát hiện lỗ hổng phần mềm.

Quá trình fine-tuning được thực hiện theo nhiệm vụ phân loại mã nguồn. Đối với các mô hình theo hướng sinh chuỗi (seq2seq), mô hình được huấn luyện để sinh ra nhãn tương ứng với mỗi đoạn mã nguồn. Trong trường hợp mô hình hỗ trợ lớp phân loại, đầu ra của mô hình là xác suất thuộc về từng lớp, từ đó suy ra nhãn dự đoán cuối cùng.

Để phản ánh sự khác biệt về đặc thù ngôn ngữ, quá trình fine-tuning được thực hiện riêng biệt cho từng tập dữ liệu theo ngôn ngữ, cụ thể là C/C++ và Java. Cách tiếp cận này cho phép đánh giá chính xác hơn khả năng thích nghi của mô hình đối với từng ngôn ngữ lập trình.

Đầu ra của nhánh fine-tune bao gồm các checkpoint mô hình đã được tinh chỉnh cùng với các thông tin đánh giá trong quá trình huấn luyện và kiểm thử, được sử dụng cho các bước sinh dự đoán và đánh giá hiệu năng ở các mục tiếp theo.

3.4. Prompting (Vanilla / Chain-of-Thought)

Trong phương pháp đề xuất, prompting đóng vai trò là cơ chế trung gian giữa mô hình ngôn ngữ lớn và bài toán phát hiện lỗ hổng phần mềm. Thay vì thay đổi kiến trúc hoặc tham số của mô hình, nhóm tập trung khảo sát ảnh hưởng của cách thiết kế prompt đầu vào đến hành vi suy luận và kết quả dự đoán của mô hình. Cùng một mô hình và cùng một tập dữ liệu, việc thay đổi chiến lược prompting cho phép đánh giá trực tiếp mức độ nhạy cảm của mô hình đối với cách đặt câu hỏi và hướng dẫn phân tích mã nguồn.

Các cấu trúc prompting được sử dụng trong khóa luận này được xây dựng dựa trên các nghiên cứu trước về prompting cho mô hình ngôn ngữ lớn, đặc biệt là chiến lược Chain-of-Thought Prompting do Wei et al. đề xuất, trong đó mô hình được khuyến khích trình bày các bước suy luận trước khi đưa ra kết luận cuối cùng [11]. Bên cạnh đó, cách tiếp cận prompting trong nghiên cứu này cũng tuân theo khung tổng quan về prompting được hệ thống hóa trong các khảo sát gần đây [7]. Trên cơ sở đó, prompt được điều chỉnh về nội dung và định dạng nhằm phù hợp với bài toán phát hiện lỗ hổng phần mềm từ mã nguồn và phục vụ mục tiêu đánh giá định lượng trong các thí nghiệm.

Trong đề tài này, hai chiến lược prompting được sử dụng và so sánh bao gồm Vanilla Prompt và Chain-of-Thought Prompt. Hai chiến lược này được áp dụng

nhất quán cho cả nhánh base model và nhánh fine-tuned nhằm đảm bảo tính công bằng trong quá trình đánh giá.

3.4.1. Vanilla Prompt

Vanilla Prompt là chiến lược prompting đơn giản và trực tiếp, trong đó mô hình được yêu cầu đưa ra kết luận phân loại mà không kèm theo hướng dẫn suy luận chi tiết. Prompt thường bao gồm một mô tả ngắn gọn về nhiệm vụ, kèm theo đoạn mã nguồn cần phân tích, và yêu cầu mô hình trả về nhãn tương ứng với khả năng tồn tại lỗ hổng.

Trong thiết lập này, đầu vào của mô hình là prompt đã được ghép với mã nguồn, và đầu ra mong đợi là một nhãn duy nhất, chẳng hạn vulnerable hoặc non-vulnerable. Vanilla Prompt phản ánh cách sử dụng phổ biến của các mô hình ngôn ngữ lớn trong các tác vụ phân loại, nơi mô hình được xem như một bộ phân loại hộp đen và chỉ quan tâm đến kết quả cuối cùng.

Mục đích của việc sử dụng Vanilla Prompt là xây dựng một mốc so sánh cơ bản để đánh giá hiệu năng của mô hình trong điều kiện không có sự can thiệp rõ ràng vào quá trình suy luận nội bộ.

3.4.2. Chain-of-Thought Prompt

Chain-of-Thought Prompt là chiến lược prompting mở rộng, trong đó mô hình được hướng dẫn phân tích mã nguồn theo từng bước trước khi đưa ra kết luận cuối cùng. Thay vì chỉ yêu cầu dự đoán nhãn, prompt gợi ý mô hình xem xét các khía cạnh liên quan đến an toàn mã nguồn, chẳng hạn như kiểm tra đầu vào, quản lý bộ nhớ, hoặc việc sử dụng các hàm và thao tác tiềm ẩn rủi ro.

Trong chiến lược này, prompt có thể yêu cầu mô hình trình bày quá trình suy luận một cách tường minh hoặc chỉ định rằng việc suy luận được thực hiện nội bộ, trong khi đầu ra cuối cùng vẫn là nhãn phân loại. Việc hướng dẫn mô hình theo một trình

tự phân tích nhất định nhằm khuyến khích mô hình khai thác sâu hơn các mối quan hệ ngữ nghĩa trong mã nguồn, từ đó có thể cải thiện chất lượng dự đoán.

Chain-of-Thought Prompt được áp dụng trong cùng điều kiện mô hình và dữ liệu như Vanilla Prompt, nhằm đảm bảo rằng mọi sự khác biệt về kết quả đều xuất phát từ chiến lược prompting chứ không phải từ các yếu tố khác.

3.4.3. So sánh hai chiến lược prompting

Việc sử dụng song song Vanilla Prompt và Chain-of-Thought Prompt cho phép nhóm phân tích vai trò của prompting trong bài toán phát hiện lỗ hổng phần mềm. Trong khi Vanilla Prompt cung cấp một cách tiếp cận đơn giản và hiệu quả về mặt tính toán, Chain-of-Thought Prompt hướng đến việc cải thiện khả năng suy luận và làm rõ quá trình đưa ra quyết định của mô hình.

Thông qua việc so sánh kết quả dự đoán và các chỉ số đánh giá thu được từ hai chiến lược prompting, đề tài hướng đến việc trả lời câu hỏi liệu việc thay đổi cách đặt câu hỏi có thể giúp nâng cao hiệu năng phát hiện lỗ hổng và hỗ trợ tốt hơn cho việc phân tích, giải thích kết quả của các mô hình LLMs-for-code hay không.

3.5. Prediction

Sau khi mô hình và chiến lược prompting đã được xác định, bước sinh dự đoán được thực hiện nhằm thu thập kết quả phân loại của mô hình trên tập dữ liệu kiểm thử. Đầu vào của bước này bao gồm mô hình được lựa chọn (base model hoặc fine-tuned model), tập dữ liệu kiểm thử đã qua tiền xử lý, cùng với prompt template tương ứng với từng chiến lược prompting. Ngoài ra, các tham số sinh chuỗi như temperature, số lượng token sinh tối đa và các tham số lấy mẫu được thiết lập thống nhất để đảm bảo tính nhất quán giữa các kịch bản thí nghiệm.

Quá trình sinh dự đoán được thực hiện theo từng mẫu dữ liệu trong tập kiểm thử. Đối với mỗi mẫu, đầu vào của mô hình được xây dựng bằng cách kết hợp prompt

template với đoạn mã nguồn cần phân tích. Mô hình sau đó thực hiện suy luận và sinh ra đầu ra tương ứng. Đầu ra này có thể là một nhãn trực tiếp hoặc một chuỗi văn bản chứa thông tin dự đoán, tùy thuộc vào kiến trúc và cách sử dụng mô hình.

Sau khi nhận được đầu ra từ mô hình, nhóm tiến hành trích xuất và chuẩn hóa kết quả về một không gian nhãn thống nhất, tương ứng với hai lớp phân loại. Việc chuẩn hóa này giúp đảm bảo rằng kết quả dự đoán từ các mô hình và chiến lược prompting khác nhau có thể được so sánh trực tiếp trong các bước đánh giá tiếp theo.

Trong suốt quá trình sinh dự đoán, các thông tin liên quan đến từng mẫu được lưu lại, bao gồm định danh mẫu, nhãn thực tế, nhãn dự đoán, và đầu ra thô của mô hình. Trong trường hợp mô hình cung cấp thêm các thông tin phụ trợ như điểm số hoặc xác suất dự đoán, các giá trị này cũng được ghi nhận để phục vụ cho việc phân tích chi tiết hơn.

Đầu ra của bước này là một tập kết quả dự đoán hoàn chỉnh, được lưu trữ dưới dạng tệp dữ liệu có cấu trúc, chẳng hạn như định dạng jsonl hoặc csv. Tập dữ liệu này đóng vai trò là đầu vào cho bước đánh giá hiệu năng và phân tích kết quả của phương pháp đề xuất.

3.6. Evaluation

Bước đánh giá được thực hiện sau khi hoàn tất quá trình sinh dự đoán, nhằm tổng hợp và phân tích kết quả dự đoán của mô hình trong bài toán phát hiện lỗ hổng phần mềm. Đầu vào của bước này bao gồm nhãn thực tế y_{true}và nhãn dự đoán y_{pred}tương ứng với từng mẫu mã nguồn trong tập kiểm thử, được thu thập từ bước sinh dự đoán.

Trên cơ sở y_{true}và y_{pred}, nhóm xây dựng ma trận nhầm lẫn (confusion matrix) (hình 2.2), bao gồm bốn thành phần: true positive (TP), false positive (FP), true

negative (TN) *và* false negative (FN). Ma trận này cung cấp cái nhìn tổng quan về hành vi dự đoán của mô hình, đặc biệt trong việc phân biệt giữa các mẫu có lỗ hổng và không có lỗ hổng.

Kết quả đánh giá được tổng hợp và báo cáo theo nhiều trục so sánh khác nhau nhằm phản ánh tác động của các thành phần trong phương pháp đề xuất. Cụ thể, nhóm tiến hành so sánh giữa mô hình cơ sở và mô hình được tinh chỉnh, giữa các mô hình được tinh chỉnh trên dữ liệu C/C++ và Java, cũng như giữa hai chiến lược prompting là Vanilla Prompt và Chain-of-Thought Prompt.

Đầu ra của bước đánh giá bao gồm các bảng tổng hợp kết quả và các biểu đồ trực quan phục vụ cho việc so sánh và phân tích. Việc đo lường cụ thể bằng các chỉ số định lượng và phân tích chi tiết dựa trên các kết quả này sẽ được trình bày ở chương thực nghiệm.

3.7. Giải thích mô hình dựa trên SHAP

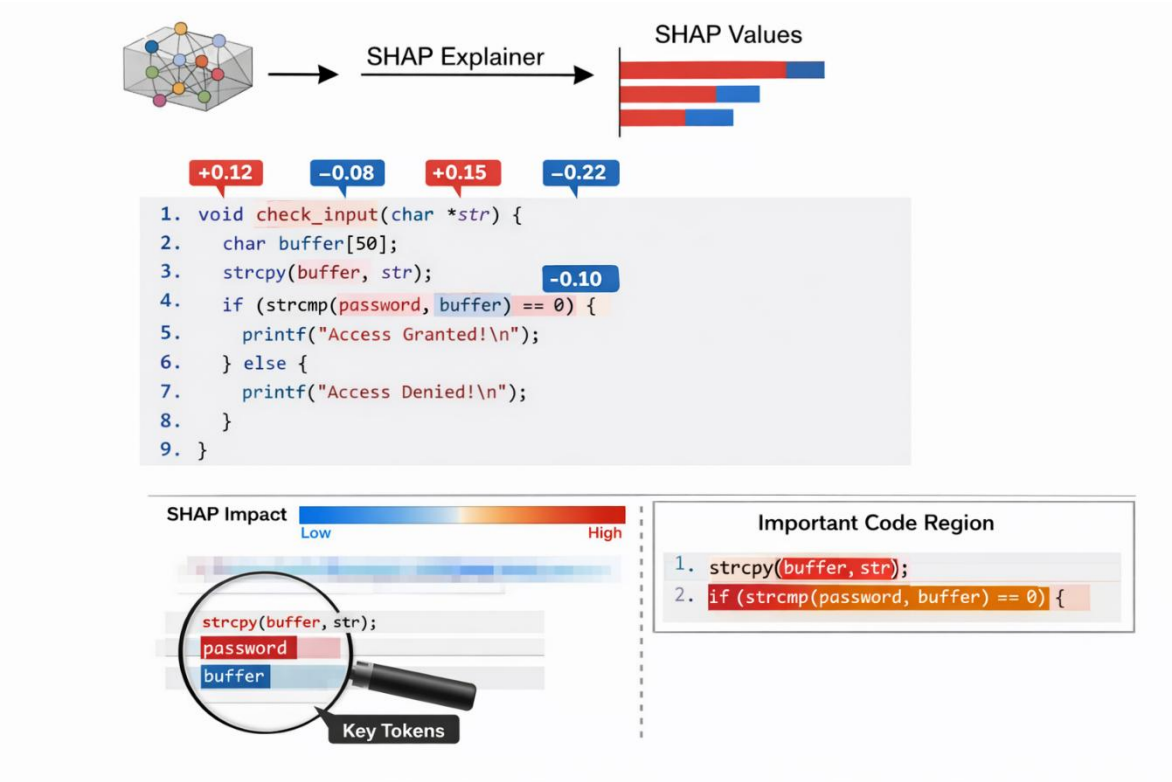
Bước giải thích mô hình được thực hiện nhằm làm rõ cơ sở mà mô hình sử dụng để đưa ra dự đoán trong bài toán phát hiện lỗ hổng phần mềm. Trong đề tài, nhóm kết hợp hai hướng tiếp cận giải thích gồm phương pháp dựa trên đặc trưng đầu vào (SHAP) và giải thích dựa trên lập luận ngôn ngữ do mô hình sinh ra thông qua Chain-of-Thought (CoT).

Đối với phương pháp SHAP, đầu vào của bước giải thích bao gồm mô hình đã được huấn luyện (base hoặc fine-tuned), hàm dự đoán xác suất của mô hình và một tập con các mẫu mã nguồn đại diện được trích từ tập kiểm thử. Việc lựa chọn tập con giúp giảm chi phí tính toán trong khi vẫn đảm bảo tính đại diện của dữ liệu.

Quá trình xử lý bao gồm việc áp dụng SHAP explainer để ước lượng giá trị đóng góp của từng token hoặc nhóm token trong mã nguồn đối với xác suất dự đoán nhãn vulnerable. Các giá trị SHAP phản ánh mức độ ảnh hưởng của từng thành phần đầu

vào đến quyết định cuối cùng của mô hình, cho phép xác định những đoạn mã làm tăng hoặc giảm khả năng dự đoán có lỗ hổng.

Kết quả đầu ra của bước này bao gồm các biểu đồ trực quan như summary plot, bar plot hoặc waterfall plot cho từng mẫu, thể hiện các token hoặc vùng mã nguồn quan trọng nhất. Những kết quả này giúp đánh giá mức độ hợp lý của mô hình từ góc nhìn an toàn phần mềm, chẳng hạn mô hình có tập trung vào các hàm nguy hiểm, vùng xử lý dữ liệu đầu vào hay các thao tác bộ nhớ nhạy cảm hay không.



Hình 3.2 Minh họa cơ chế giải thích dự đoán của mô hình phát hiện lỗ hổng mã nguồn bằng phương pháp SHAP.

Hình 3.2 minh họa cách phương pháp SHAP (SHapley Additive exPlanations) được sử dụng để giải thích quyết định của mô hình trong bài toán phát hiện lỗ hổng phần mềm từ mã nguồn. Mô hình nhận đầu vào là đoạn mã nguồn và đưa ra dự đoán

nhãn vulnerable hoặc non-vulnerable, trong khi SHAP đóng vai trò phân tích mức độ đóng góp của từng thành phần đầu vào vào kết quả dự đoán.

Các giá trị SHAP được biểu diễn thông qua việc tô màu trực tiếp lên mã nguồn, trong đó các token hoặc câu lệnh có màu đỏ thể hiện tác động làm tăng khả năng mô hình kết luận đoạn mã có lỗ hổng, còn các vùng màu xanh biểu thị tác động ngược lại. Ngoài ra, biểu đồ tổng hợp ở phía dưới cho thấy mức độ ảnh hưởng của các token quan trọng nhất, giúp người phân tích nhanh chóng xác định những vùng mã nguồn có vai trò lớn trong quyết định của mô hình.

Thông qua cách biểu diễn trực quan này, SHAP hỗ trợ việc kiểm tra và diễn giải kết quả dự đoán của mô hình, góp phần nâng cao độ tin cậy khi áp dụng các mô hình học máy trong lĩnh vực an toàn phần mềm.

Chương 4. THỰC NGHIỆM VÀ ĐÁNH GIÁ KẾT QUẢ

Chương này nhằm đánh giá hiệu quả của các mô hình ngôn ngữ lớn (LLM/Seq2Seq) trong bài toán nhận diện lỗ hổng bảo mật từ mã nguồn, đồng thời khảo sát tác động của fine-tune theo ngôn ngữ, prompting và khả năng chuyển giao chéo ngôn ngữ. Ngoài ra, chương cũng trình bày cách áp dụng XAI với SHAP để giải thích quyết định dự đoán của mô hình.

4.1. Câu hỏi nghiên cứu (Research Questions – RQ)

Các câu hỏi nghiên cứu đặt ra:

- **RQ1:** Việc fine-tune mô hình trên dữ liệu mã nguồn theo từng ngôn ngữ lập trình (C/C++ hoặc Java) ảnh hưởng như thế nào đến hiệu năng phát hiện lỗ hổng khi mô hình được đánh giá trên cùng ngôn ngữ đó, xét theo các chỉ số Precision, Recall và F1-score?

  Kịch bản khảo sát: So sánh hiệu năng của mô hình ở trạng thái chưa fine-tune và đã fine-tune theo từng ngôn ngữ, khi đánh giá trên tập dữ liệu test tương ứng, nhằm xác định mức độ cải thiện do quá trình fine-tuning mang lại (4.7).

- **RQ2:** Mức độ khái quát hóa chéo ngôn ngữ của mô hình được fine-tune trên dữ liệu C/C++ thể hiện ra sao khi áp dụng trực tiếp để phát hiện lỗ hổng trên mã nguồn Java mà không thực hiện fine-tune bổ sung, xét theo các chỉ số Precision, Recall và F1-score?

  Kịch bản khảo sát: Huấn luyện mô hình trên dữ liệu của một ngôn ngữ và đánh giá trực tiếp trên dữ liệu của ngôn ngữ còn lại, không thực hiện fine-tune bổ sung, nhằm phân tích khả năng chuyển giao liên ngôn ngữ (4.8).

- **RQ3:** Các chiến lược prompting khác nhau, bao gồm vanilla prompt và Chain-of-Thought prompt, ảnh hưởng như thế nào đến hiệu năng của mô hình trong bài toán nhận diện lỗ hổng phần mềm, xét theo các chỉ số Precision, Recall và F1-score?

Kịch bản khảo sát: Giữ nguyên mô hình và dữ liệu đánh giá, chỉ thay đổi chiến lược prompting giữa Vanilla Prompt và CoT Prompt để phân tích riêng tác động của prompting đến hiệu năng dự đoán (4.9).

- **RQ4:** Các phương pháp diễn giải mô hình dựa trên SHAP phản ánh đặc điểm hành vi dự đoán của mô hình nhận diện lỗ hổng phần mềm như thế nào, xét theo các tiêu chí Sparsity, Stability, Robustness và Efficiency?

  Kịch bản khảo sát: Áp dụng phương pháp SHAP lên các mô hình và cấu hình tiêu biểu, sau đó đánh giá chất lượng lời giải thích dựa trên các tiêu chí định lượng đã đề ra (4.10).

4.2. Bộ dữ liệu và tiền xử lý

4.2.1. Bộ dữ liệu thực nghiệm

Các thí nghiệm trong đề tài được thực hiện trên bộ dữ liệu công khai Juliet Test Suite, bao gồm các mẫu mã nguồn được thiết kế cho bài toán phát hiện lỗ hổng phần mềm. Bộ dữ liệu này cung cấp các đoạn mã nguồn có nhãn rõ ràng, phản ánh các kịch bản lập trình an toàn và không an toàn, và được sử dụng rộng rãi trong các nghiên cứu về an toàn phần mềm.

Trong phạm vi thực nghiệm, đề tài sử dụng các mẫu mã nguồn thuộc hai ngôn ngữ C/C++ và Java. Dữ liệu được tổ chức dưới dạng các tệp JSONL (ví dụ: juliet/test.jsonl), trong đó mỗi dòng tương ứng với một mẫu mã nguồn độc lập kèm theo nhãn phân loại. Đối với mỗi mẫu, dữ liệu được chuẩn hóa theo một schema thống nhất, bao gồm tối thiểu các trường: id, code và label. Trường đầu vào có thể là code, func hoặc input. Nhãn (0/1) nằm ở ID/Label. Ngoài ra, một số metadata như language hoặc cwe_id được giữ lại để phục vụ cho các phân tích bổ sung.

4.2.2. Tiền xử lý

Trước khi đưa vào mô hình để tiến hành thực nghiệm, dữ liệu được tiền xử lý nhằm đảm bảo tính nhất quán và độ tin cậy của đầu vào. Các mẫu thiếu mã nguồn hoặc nhãn hợp lệ, các đoạn mã rỗng, cũng như các mẫu trùng lặp được loại bỏ khỏi tập dữ liệu. Mã nguồn sau đó được chuẩn hóa ở mức cú pháp cơ bản, bao gồm xử lý khoảng trắng và định dạng dòng.

Đối với các đoạn mã có độ dài lớn, dữ liệu được giới hạn độ dài đầu vào để phù hợp với khả năng xử lý của mô hình, đồng thời ưu tiên giữ lại phần thân hàm và các đoạn mã liên quan đến xử lý dữ liệu hoặc thao tác nhạy cảm. Cụ thể quy trình:

- Đọc lần lượt từng dòng JSONL.
- Lấy nội dung code từ các key ưu tiên: code → func → input.
- Loại bỏ mẫu thiếu code.
- Chuẩn hóa nhãn về 2 lớp (good/bad) theo định nghĩa bộ dữ liệu.

Sau khi hoàn tất tiền xử lý, dữ liệu được chia thành tập huấn luyện và tập kiểm thử theo tỉ lệ 80% – 20%. Việc chia tập được thực hiện sau bước tiền xử lý và đảm bảo phân bố nhãn giữa các tập là tương đồng. Quy trình chia tập được áp dụng độc lập cho từng ngôn ngữ lập trình nhằm phản ánh đúng khả năng học và tổng quát hóa của mô hình trong từng kịch bản thực nghiệm.

4.3. Mô hình và cấu hình triển khai

4.3.1. Mô hình

4.3.1.1. Mô hình Starcoder2

Trong KLTN, mô hình được sử dụng là StarCoder2 (bigcode/starcoder2-3b) theo hướng Causal Language Model (Decoder-only LLM), được fine-tune để thực hiện nhiệm vụ phân loại lỗ hổng bảo mật từ đoạn mã nguồn. Toàn bộ hệ thống được hiện thực theo pipeline gồm 5 khối: (1) tạo dataset từ Juliet, (2) chia tập, (3) xây dựng

prompt + định dạng dữ liệu huấn luyện, (4) fine-tune, (5) suy luận & giải thích bằng SHAP.

4.3.1.1.1 Tạo dataset từ Juliet và gán nhãn good/bad

Nhóm sử dụng bộ dữ liệu Juliet Test Suite cho hai ngôn ngữ C/C++ và Java và tiến hành xây dựng tập dữ liệu ở mức hàm nhằm phục vụ bài toán phân loại mã nguồn vulnerable / non-vulnerable. Quá trình trích xuất hàm được thực hiện bằng cách kết hợp biểu thức chính quy và phân tích tuyến tính mã nguồn, trong đó biểu thức chính quy được sử dụng để nhận diện phần header của hàm (bao gồm kiểu trả về, tên hàm, danh sách tham số và ký tự mở ngoặc "{"), sau đó thân hàm được trích xuất bằng cách quét tuần tự và đếm cặp ngoặc nhọn {}. Trong quá trình này, các trường hợp đặc biệt như comment một dòng (//), comment nhiều dòng (/* … */), chuỗi ký tự ("...") và ký tự đơn ('...') được xử lý nhằm tránh việc đếm ngoặc sai trong các ngữ cảnh không thuộc mã thực thi. Sau khi trích xuất, các hàm được gán nhãn dựa trên quy ước đặt tên chuẩn của bộ Juliet, theo đó các hàm có tên chứa từ khóa *"bad"* được gán nhãn 1 (vulnerable), trong khi các hàm có tên chứa từ khóa *"good"* được gán nhãn 0 (non-vulnerable); các hàm phụ trợ không liên quan được loại bỏ để giảm nhiễu dữ liệu và đảm bảo tính nhất quán của tập huấn luyện. Bên cạnh thân hàm và nhãn phân loại, nhóm trích xuất thêm các metadata từ tên tệp nguồn theo quy ước của Juliet, bao gồm thông tin về CWE, source, sink và variant (ví dụ: CWE114_…__w32_…_socket_01.c), phục vụ cho các phân tích và đánh giá ở các thí nghiệm tiếp theo. Kết quả của bước này là một tệp dữ liệu định dạng JSONL, trong đó mỗi mẫu bao gồm thân hàm mã nguồn, nhãn phân loại nhị phân (0/1) và các metadata liên quan, cho phép thuận tiện trong quá trình huấn luyện, suy luận và đánh giá tự động trên các mô hình LLMs-for-code.

4.3.1.1.2 Chia tập Train/Test

Sau khi thu được tập dữ liệu ở định dạng JSONL, nhóm tiến hành bước chia dữ liệu để phục vụ cho quá trình huấn luyện và đánh giá mô hình. Toàn bộ các mẫu dữ liệu được đọc vào, xáo trộn ngẫu nhiên nhằm loại bỏ ảnh hưởng của thứ tự ban đầu, sau đó được chia thành các tập con theo tỉ lệ xác định cho huấn luyện, xác thực và kiểm

thử. Các tập dữ liệu này được lưu thành các file riêng biệt để sử dụng trong các thí nghiệm tiếp theo.

Trong đề tài, prompt đầu vào được chuẩn hóa theo hai dạng tương ứng với hai mục tiêu sử dụng khác nhau. Thứ nhất, prompt dạng phân loại đơn giản (vanilla prompt) được dùng cho các thí nghiệm mà mô hình chỉ cần trả lời trực tiếp nhãn good/bad. Ở bước này, mã nguồn được lấy từ trường func trong dataset, nhãn được ánh xạ từ 0 → good và 1 → bad. Prompt có cấu trúc ngắn gọn, chỉ bao gồm đoạn mã và câu hỏi xác định liệu hàm có tồn tại lỗ hổng hay không. Mỗi mẫu dữ liệu được lưu dưới dạng một dòng JSON chứa prompt và nhãn tương ứng. Cách thiết kế này phù hợp cho các thí nghiệm baseline, tập trung đánh giá khả năng phân loại trực tiếp của mô hình. Thứ hai, prompt dạng Chain-of-Thought kết hợp câu trả lời JSON được sử dụng trong quá trình huấn luyện fine-tune. Prompt được thiết kế theo hướng một hệ thống phát hiện lỗ hổng, trong đó mã nguồn được đặt trong khối code, kèm theo các thông tin bổ trợ (nếu có) như chuỗi suy luận gợi ý (chain), ý đồ của mã (intent) và thông tin dòng dữ liệu (dfg). Mô hình được yêu cầu sinh ra đầu ra cuối cùng dưới dạng một đối tượng JSON chuẩn hóa, bao gồm nhãn, trạng thái lỗ hổng, thành phần ảnh hưởng và lý do.

4.3.1.1.3 Fine-tune StarCoder2 bằng HuggingFace Trainer

Quá trình fine-tune mô hình StarCoder2-3B được triển khai bằng HuggingFace Trainer API với cấu hình huấn luyện được thiết lập nhằm cân bằng giữa hiệu năng mô hình và giới hạn tài nguyên tính toán. Mô hình và tokenizer được tải từ bigcode/starcoder2-3b, trong đó pad_token được gán bằng eos_token để đảm bảo tương thích với kiến trúc causal language model. Dữ liệu huấn luyện và kiểm chứng được đọc từ các tệp train.jsonl và val.jsonl, mỗi mẫu đầu vào được xây dựng dưới dạng chuỗi văn bản gồm prompt và phần đầu ra mục tiêu ở định dạng JSON nhằm đảm bảo mô hình sinh kết quả có cấu trúc thống nhất. Ở bước tiền xử lý, dữ liệu được token hóa với độ dài tối đa 1024 token nhằm giảm chi phí bộ nhớ so với cấu hình mặc định 2048 token, đồng thời vẫn đảm bảo khả năng biểu diễn ngữ cảnh của đoạn mã.

Trong quá trình huấn luyện, mô hình được thiết lập ở độ chính xác bán phần (float16), kết hợp với cơ chế phân bổ thiết bị tự động và kỹ thuật gradient checkpointing để tiết kiệm VRAM. Do giới hạn bộ nhớ, batch size trên mỗi thiết bị được đặt bằng 1 và sử dụng chiến lược tích lũy gradient với 4 bước, tương đương batch hiệu dụng bằng 4. Các tham số huấn luyện chính được lựa chọn theo thực tiễn fine-tuning mô hình ngôn ngữ lớn cho mã nguồn, bao gồm learning rate 2×10^{-5} và 3 epoch huấn luyện, nhằm đảm bảo mô hình hội tụ mà không gây hiện tượng overfitting. Trong quá trình huấn luyện, mô hình được đánh giá và lưu định kỳ theo số bước, đồng thời mô hình tốt nhất được lựa chọn dựa trên giá trị eval_loss trên tập kiểm chứng. Kết quả fine-tuning được lưu lại và sử dụng cho các thí nghiệm đánh giá tiếp theo, trong đó mô hình có khả năng sinh đầu ra ở định dạng JSON đúng yêu cầu, phục vụ trực tiếp cho việc trích xuất nhãn và tính toán các chỉ số Precision, Recall và F1-score.

4.3.1.1.4 Suy luận và chuẩn hóa dự đoán (inference + parsing)

Sau khi hoàn tất quá trình fine-tuning, nhóm triển khai một chương trình suy luận để đánh giá mô hình trên từng hàm mã nguồn cụ thể. Mô hình được nạp bằng AutoModelForCausalLM.from_pretrained, trong đó kiểu dữ liệu được lựa chọn tự động nhằm tối ưu hiệu năng tính toán (ưu tiên bfloat16 khi có GPU hỗ trợ, ngược lại sử dụng float32), kết hợp với cơ chế phân bổ thiết bị tự động (device_map="auto"). Trong giai đoạn suy luận, prompt được thiết kế theo chiến lược Chain-of-Thought, yêu cầu mô hình phân tích mã nguồn theo các khía cạnh liên quan đến an toàn bộ nhớ và bảo mật (như buffer usage, pointer safety, input validation, memory access và các lời gọi hàm tiềm ẩn rủi ro) trước khi đưa ra kết luận cuối cùng. Để đảm bảo tính ổn định và khả năng tái lập của kết quả, quá trình sinh đầu ra được cấu hình theo hướng xác định, với số token sinh mới giới hạn ở mức thấp, không sử dụng sampling và đặt nhiệt độ bằng 0. Sau khi mô hình sinh kết quả, đầu ra được chuẩn hóa thông qua một bước parsing tự động nhằm trích xuất nhãn dự đoán. Cụ thể, hệ thống ưu tiên nhận diện nhãn theo mẫu định dạng đã quy ước trong prompt; trong trường hợp không tìm thấy định dạng này, một cơ chế dự

phòng được áp dụng bằng cách quét toàn bộ đầu ra để phát hiện các từ khóa tương ứng với hai nhãn GOOD và BAD. Cách tiếp cận này giúp đảm bảo tính nhất quán của nhãn dự đoán, đồng thời hạn chế lỗi phát sinh do sự biến thiên trong cách sinh văn bản của mô hình.

4.3.1.1.5 Giải thích mô hình bằng SHAP (token occlusion)

Phương pháp SHAP được hiện thực theo hướng token occlusion nhằm đánh giá mức độ quan trọng của từng token trong mã nguồn. Với mỗi mẫu code, hệ thống tạo prompt theo định dạng chuẩn, sau đó chạy mô hình để lấy dự đoán gốc (base_class). Prompt được tokenize và chỉ xét tối đa MAX_TOKENS_SHAP token đầu để giảm chi phí tính toán. Tiếp theo, từng token được lần lượt thay thế bằng eos_token (coi như bị loại bỏ) và mô hình được chạy lại; nếu nhãn dự đoán thay đổi so với base_class thì token đó được gán shap_score = 1, ngược lại là 0. Kết quả thu được gồm danh sách token, điểm SHAP tương ứng và đầu ra gốc của mô hình, được lưu lại để phục vụ các phân tích XAI ở các mục sau.

4.3.1.2. Mô hình Qwen2

Bên cạnh StarCoder2, khóa luận tiếp tục triển khai và đánh giá mô hình Qwen2.5-Coder-1.5B, một mô hình ngôn ngữ lớn chuyên biệt cho mã nguồn, nhằm so sánh khả năng nhận diện lỗ hổng bảo mật và khả năng giải thích mô hình giữa các kiến trúc LLM khác nhau.

Qwen2 được triển khai theo cùng triết lý LLM wrapper + prompt-based inference + fine-tune (LoRA) + XAI, đảm bảo tính nhất quán trong toàn bộ pipeline thực nghiệm.

4.3.1.2.1 Chuẩn bị dữ liệu đầu vào cho Qwen2

Ở bước này nhóm sẽ duyệt toàn bộ thư mục juliet/testcases, lọc các file mã nguồn C/C++ (.c, .cpp), đọc toàn bộ nội dung file làm trường code, trích xuất nhãn CWE từ đường dẫn thư mục hoặc tên file (ví dụ CWE36_Absolute_Path_Traversal), ghi kết quả ra file juliet.csv với hai cột chính: code: toàn bộ mã nguồn, label: nhãn

CWE tương ứng. cho phép mô hình Qwen2 tiếp cận bài toán dưới dạng nhận diện và giải thích lỗ hổng, thay vì chỉ phân loại nhị phân đơn thuần.

4.3.1.2.2 Thiết kế prompt cho Qwen2

Đề tài sử dụng hai dạng prompt: vanilla prompt và Chain-of-Thought (CoT) prompt. Với vanilla prompt, prompt chỉ cung cấp đoạn mã và yêu cầu mô hình đưa ra kết luận vulnerable / non-vulnerable (hoặc GOOD/BAD) mà không yêu cầu trình bày các bước suy luận. Ngược lại, CoT prompt bổ sung chỉ dẫn để mô hình phân tích và lập luận theo từng bước trước khi đưa ra kết luận, nhằm hỗ trợ khả năng giải thích.

4.3.1.2.3 Fine-tune Qwen2 bằng LoRA

Phương pháp LoRA (Low-Rank Adaptation) được sử dụng nhằm giảm chi phí huấn luyện.

a) Mô hình và tokenizer

Mô hình gốc: Qwen/Qwen2.5-Coder-1.5B, Tokenizer được nạp với trust_remote_code=True. Trong trường hợp tokenizer không có pad_token, token eos_token được sử dụng thay thế để đảm bảo tương thích với Causal LM.

b) Cấu hình LoRA

Mô hình được gắn LoRA với các tham số: r = 16, lora_alpha = 16, lora_dropout = 0.05, target_modules = {q_proj, k_proj, v_proj, o_proj}, việc chỉ fine-tune các projection layer giúp: giảm đáng kể số tham số cần huấn luyện, giữ nguyên kiến thức nền của mô hình gốc, phù hợp với tài nguyên phần cứng hạn chế.

c) Chuẩn hóa dữ liệu huấn luyện

Mỗi mẫu huấn luyện được chuyển thành chuỗi văn bản gồm: Prompt (code + task), nhãn ở phần ### Answer:. Dữ liệu được tokenize với: max_length = 512, truncation = True, không padding trong preprocessing (padding được xử lý bởi data collator).

d) Cấu hình huấn luyện

Một số tham số huấn luyện chính: per_device_train_batch_size = 1, gradient_accumulation_steps = 8, learning_rate = 2e-4, num_train_epochs = 1, fp16 = True, lưu checkpoint mỗi 500 step, giữ tối đa 2 checkpoint. Sau huấn luyện, mô hình LoRA được lưu.

4.3.1.2.4 Đánh giá định lượng

Thực hiện đánh giá mô hình theo hướng prompt-based classification. Với mỗi mẫu: đưa code vào prompt, tính điểm (score) cho từng nhãn khả dĩ, chọn nhãn có score cao nhất làm dự đoán. Dựa trên nhãn thật và nhãn dự đoán, script tính: TP, FP, TN, FN, cách đánh giá này tương thích với các chỉ số được trình bày ở mục 4.5, cho phép so sánh trực tiếp với StarCoder2.

4.3.1.2.5 Giải thích mô hình Qwen2 bằng SHAP

Triển khai SHAP cho Qwen2 theo hướng token-level explanation: hàm dự đoán f(tokens) trả về logits của token cuối, SHAP được áp dụng trên các token đầu vào, kết quả thể hiện mức độ đóng góp của từng token (ví dụ gets, buf) vào quyết định mô hình. Kết quả SHAP được trực quan hóa, phục vụ phân tích: token nào mang tính quyết định, mức độ hợp lý của lời giải thích, so sánh với StarCoder2 ở cùng kịch bản.

4.3.1.3. Mô hình BART

Bên cạnh các mô hình Coder LLM (StarCoder2, Qwen2), khóa luận sử dụng BART (Bidirectional and Auto-Regressive Transformer) như một mô hình Seq2Seq nền tảng để làm đối chứng và đánh giá khả năng áp dụng của các mô hình ngôn ngữ tổng quát trong bài toán nhận diện lỗ hổng bảo mật.

Không giống các Coder LLM thuần decoder, BART sử dụng kiến trúc encoder–decoder, phù hợp với các nhiệm vụ sinh chuỗi có cấu trúc, đặc biệt khi kết hợp với Chain-of-Thought (CoT) và đầu ra dạng văn bản có giải thích.

4.3.1.3.1 Chuẩn bị và tiền xử lý dữ liệu

Việc xây dựng dữ liệu huấn luyện cho BART được hiện thực với các bước chính như sau:

a) Trích xuất hàm từ bộ Juliet

Duyệt toàn bộ thư mục juliet/testcases, Đọc các file dataset C/C++ hoặc Java

Sử dụng biểu thức chính quy để trích xuất các hàm có tên chứa hậu tố _bad hoặc _good. Loại bỏ comment, macro và các chỉ thị tiền xử lý không cần thiết. Chuẩn hóa nội dung mã nguồn để giảm nhiễu. Mỗi hàm sau khi trích xuất được gán:

- code: thân hàm đã làm sạch.
- label: VULNERABLE hoặc NOT VULNERABLE,
- cwe: mã CWE tương ứng (nếu có).

b) Sinh dữ liệu huấn luyện dạng Chain-of-Thought

Khác với StarCoder2 (trả nhãn trực tiếp hoặc JSON), pipeline BART chuyển bài toán phân loại thành bài toán sinh chuỗi có lập luận. Hàm add_explanation() trong sinh đầu ra (output) theo dạng:

+ Với mẫu VULNERABLE: mô tả hành vi nguy hiểm,chỉ ra thao tác không an toàn,liên hệ với CWE,kết luận bằng Answer: VULNERABLE.

+ Với mẫu NOT VULNERABLE: Mô tả thực hành lập trình an toàn, phủ định các pattern nguy hiểm,kết luận bằng Answer: NOT VULNERABLE.

Thiết kế này giúp BART học suy luận từng bước → kết luận, thay vì chỉ học ánh xạ trực tiếp từ code sang nhãn.

4.3.1.3.2 Huấn luyện mô hình BART

Quá trình fine-tune được hiện thực:

a) Mô hình và tokenizer

Mô hình nền: facebook/bart-base, tokenizer tương ứng của BART, kiến trúc encoder–decoder cho phép xử lý code dài ở encoder và sinh giải thích ở decoder.

b) Chuẩn hóa dữ liệu đầu vào/đầu ra

Encoder input: mã nguồn (code), giới hạn max_length = 512.

Decoder target: chuỗi giải thích + nhãn (output), giới hạn max_length = 128, Padding và truncation được áp dụng để đảm bảo batch thống nhất. Các mẫu không sinh được nhãn hợp lệ sẽ bị loại bỏ để tránh gây nhiễu khi huấn luyện.

c) Cấu hình huấn luyện

Một số tham số huấn luyện chính: per_device_train_batch_size = 4, num_train_epochs = 5, learning_rate = 3e-5, evaluation_strategy = "epoch", save_strategy = "epoch", fp16 được bật nếu có GPU, chỉ giữ tối đa 2 checkpoint.

Mô hình sau huấn luyện được lưu lại.

4.3.1.3.3 Suy luận và chuẩn hóa đầu ra

Quá trình suy luận được triển khai như sau:

a) Sinh chuỗi dự đoán

Với mỗi đoạn mã nguồn: mã được đưa vào encoder, decoder sinh chuỗi giải thích với: max_new_tokens = 128, num_beams = 5, do_sample = False (suy luận xác định).

b) Trích xuất nhãn dự đoán

Do BART sinh văn bản tự do, nhãn cuối cùng được trích xuất bằng cách: tìm các từ khóa như VULNERABLE, NOT VULNERABLE, SAFE, ánh xạ về nhãn chuẩn. Cách làm này cho phép đánh giá định lượng đồng thời vẫn giữ được lời giải thích trung gian.

4.3.1.3.4 Đánh giá mô hình BART

Thực hiện đánh giá mô hình BART trên tập test: sinh dự đoán theo batch, chuẩn hóa nhãn dự đoán, tính các chỉ số:Accuracy, Precision, Recall, F1-score.

4.3.1.3.5 Giải thích mô hình BART bằng SHAP

Triển khai SHAP Gradient Explainer cho BART: mô hình được xem như một hàm ánh xạ từ token đầu vào → logits, SHAP tính mức độ đóng góp của từng token trong mã nguồn, kết quả được trực quan hóa bằng shap.plots.text. Điều này cho

phép phân tích: token nào ảnh hưởng mạnh đến quyết định VULNERABLE ,mức độ hợp lý của lời giải thích so với trực giác chuyên gia.

4.3.1.4. Mô hình PLBART

Bên cạnh các mô hình LLM tổng quát và coder LLM, khóa luận triển khai PLBART – một mô hình encoder–decoder được tiền huấn luyện chuyên biệt cho mã nguồn, nhằm đánh giá hiệu quả của các mô hình code-aware pretraining trong bài toán nhận diện lỗ hổng bảo mật.

PLBART được xây dựng dựa trên kiến trúc BART nhưng được tiền huấn luyện trên dữ liệu mã nguồn đa ngôn ngữ, giúp mô hình nắm bắt tốt hơn cấu trúc cú pháp và ngữ nghĩa của code so với BART thông thường.

4.3.1.4.1 Chuẩn bị và tiền xử lý dữ liệu

Việc chuẩn bị dữ liệu cho PLBART được hiện thực trong đề tài, với quy trình như sau:

a) Thu thập dữ liệu từ bộ Juliet

Duyệt toàn bộ thư mục juliet/testcases, Lọc các file mã nguồn (.c, .cpp, .h, .hpp, …), trích xuất toàn bộ nội dung file làm đầu vào, gán nhãn dựa trên thư mục CWE trong đường dẫn (ví dụ CWE369_Divide_By_Zero), loại bỏ các lớp CWE quá hiếm (xuất hiện < 2 mẫu), cắt bớt mã nguồn quá dài để tránh vượt giới hạn token.

Sau bước này, dữ liệu được chia theo tỉ lệ 80% train – 20% test, đồng thời đảm bảo phân bố nhãn được giữ nguyên (stratified split).

b) Định dạng dữ liệu huấn luyện

Mỗi mẫu dữ liệu được biểu diễn dưới dạng JSONL với các trường chính: code: đoạn mã nguồn, label: nhãn CWE tương ứng. Khác với BART (sinh chuỗi giải thích), PLBART trong khóa luận được sử dụng cho bài toán phân loại đa lớp (multi-class classification), trong đó mỗi lớp tương ứng với một CWE.

4.3.1.4.2 Kiến trúc mô hình PLBART cho phân loại

Kiến trúc mô hình được định nghĩa :

a) Backbone PLBART

Backbone sử dụng uclanlp/plbart-base, Encoder của PLBART được khai thác để trích xuất biểu diễn ngữ nghĩa của mã nguồn, Decoder không được sử dụng trực tiếp trong bước phân loại.

b) Chiến lược pooling và classification head

Do PLBART không có sẵn token [CLS], khóa luận áp dụng mean pooling có mask attention trên toàn bộ hidden states của encoder:

$$\mathbf{h}_{\text{pooled}} = \frac{1}{\sum m_i} \sum_{i=1}^{L} m_i \cdot \mathbf{h}_i$$

Trong đó, h_i là hidden state tại token thứ i, m_i là attention mask. Vector pooling này được đưa qua: tầng Dropout, tầng Linear để sinh logits cho từng lớp CWE. Hàm mất mát sử dụng Cross-Entropy Loss, phù hợp cho bài toán phân loại đa lớp.

4.3.1.4.3 Tokenization và Data Collator

Quá trình tokenization được thực hiện trong dataset_loader.py: Sử dụng tokenizer của PLBART, max_length = 512, truncation được bật để tránh vượt ngưỡng, nhãn được ánh xạ sang chỉ số số nguyên thông qua label2id. Data collator (DataCollatorForCode) đảm nhiệm: padding động theo batch, tối ưu bộ nhớ GPU bằng cách pad theo bội số của 8.

4.3.1.4.4 Huấn luyện mô hình PLBART

a) Cấu hình huấn luyện

Một số tham số chính: learning_rate = 2e-5, per_device_train_batch_size = 4, num_train_epochs = 3, weight_decay = 0.01, fp16 = True, không thực hiện evaluation trong quá trình train (evaluation_strategy = "no"). Việc không eval trong train giúp tập trung tài nguyên cho huấn luyện, trong khi đánh giá được thực hiện riêng ở bước sau.

b) Lưu mô hình và ánh xạ nhãn

Sau huấn luyện: trọng số mô hình được lưu tại outputs/plbart_cls, file label_map.json được tạo để lưu ánh xạ label2id và id2label, tokenizer được lưu đồng bộ với mô hình.

4.3.2. Cấu hình triển khai

Bảng 4.1 mô tả cấu hình kiến trúc và yêu cầu tài nguyên của các mô hình được sử dụng trong thực nghiệm, nhằm làm rõ bối cảnh triển khai và các ràng buộc tính toán trong quá trình đánh giá.

Tên mô hình / Tiêu chí	StarCoder2	BART	PLBART	Qwen2
Loại mô hình	Decoder-only	Encoder-Decoder	Encoder-Decoder	Decoder-only
Mục tiêu thiết kế	Sinh mã nguồn	Sinh văn bản / Seq2Seq	Sinh & phân tích mã nguồn	LLM đa năng
Dung lượng VRAM	~16–40 GB	~8–16 GB	~12–24 GB	~16–48 GB
Tốc độ inference	Trung bình	Nhanh	Trung bình	Trung bình-Chậm
Hidden_size	3072	768	768	2048
Số lớp Transformer	30	6 encoder + 6 decoder	6 encoder + 6 decoder	24
Attention heads	24	12	12	16
FFN Dimension	12288	3072	3072	8192
Max context	16,384	1,024	1,024	>16,384

length				
Vocabulary size	49,152	~50k	~50k	~150k

Bảng 4.1 Cấu hình triển khai các mô hình

4.4. Thiết lập đánh giá

4.4.1. Chỉ số đánh giá cho bài toán phân loại

Đối với bài toán nhận diện lỗ hổng bảo mật nhị phân (good / bad), các chỉ số đánh giá được tính dựa trên ma trận nhầm lẫn (Confusion Matrix) gồm bốn thành phần:

- TP (True Positive): mẫu *bad* được dự đoán đúng là bad
- FP (False Positive): mẫu *good* bị dự đoán nhầm là bad
- FN (False Negative): mẫu *bad* bị dự đoán nhầm là good
- TN (True Negative): mẫu *good* được dự đoán đúng là good

Từ đó, các chỉ số được xác định như sau:

- Precision (Độ chính xác):

$$\text{Precision} = \frac{TP}{TP + FP}$$

Chỉ số này phản ánh mức độ tin cậy của dự đoán "bad", tức là trong số các mẫu được mô hình dự đoán là có lỗ hổng, bao nhiêu mẫu là đúng.

- Recall (Độ bao phủ):

$$\text{Recall} = \frac{TP}{TP + FN}$$

Recall thể hiện khả năng phát hiện đầy đủ các lỗ hổng, tức là mô hình tìm được bao nhiêu phần trăm các mẫu *bad* thực sự tồn tại.

- F1-score:

$$\text{F1} = 2 \times \frac{\text{Precision} \times \text{Recall}}{\text{Precision} + \text{Recall}}$$

F1-score là trung bình điều hòa giữa Precision và Recall, thường được sử dụng khi dữ liệu mất cân bằng giữa lớp good và bad, như trong bài toán phát hiện lỗ hổng.

- Accuracy (Độ chính xác tổng thể):

$$\text{Accuracy} = \frac{TP + TN}{TP + TN + FP + FN}$$

Accuracy phản ánh tỷ lệ dự đoán đúng trên toàn bộ tập dữ liệu, tuy nhiên chỉ số này không phản ánh đầy đủ chất lượng mô hình khi dữ liệu mất cân bằng.

Exact Match (EM): được sử dụng khi đầu ra của mô hình là chuỗi nhãn sinh ra, được coi là đúng nếu trùng khớp hoàn toàn với nhãn chuẩn.

$$\text{Exact Match} = \begin{cases} 1, & \text{nếu output = nhãn chuẩn} \\ 0, & \text{ngược lại} \end{cases}$$

Trong thực nghiệm, mô hình facebook/bart-large chưa fine-tune cho kết quả Exact Match = 0 trên 21037 mẫu Juliet, cho thấy mô hình chưa học được ánh xạ từ mã nguồn sang nhãn "good/bad", từ đó làm rõ vai trò của fine-tuning và prompt engineering (RQ1).

4.4.2. Chỉ số đánh giá cho XAI (SHAP-based Metrics)

Ngoài hiệu năng dự đoán, khóa luận còn đánh giá chất lượng lời giải thích của mô hình thông qua các chỉ số XAI, được tính từ giá trị SHAP trên từng mẫu.

Giả sử với mỗi mẫu, SHAP sinh ra một vector giá trị:

$$\phi = (\phi_1, \phi_2, \ldots, \phi_n)$$

Trong đó $\phi_i$là mức độ đóng góp của token thứ i.

4.4.2.1. Sparsity (Độ thưa của lời giải thích)

Sparsity đo lường mức độ tập trung của lời giải thích vào một số ít token quan trọng.

$$\text{Sparsity} = \frac{|\{i \mid |\phi_i| > \epsilon\}|}{n}$$

Trong đó: n là tổng số token, ϵ là ngưỡng nhỏ để loại bỏ nhiễu, giá trị Sparsity càng thấp cho thấy mô hình chỉ dựa vào một số token chính, giúp lời giải thích dễ hiểu hơn đối với con người.

4.4.2.2. Stability (Độ ổn định)

Stability đánh giá mức độ nhất quán của SHAP khi đầu vào bị nhiễu nhẹ (ví dụ thay đổi token không quan trọng).

$$\text{Stability} = 1 - \frac{1}{k}\sum_{j=1}^{k} \| \phi - \phi^{(j)} \|_1$$

Trong đó: $\phi^{(j)}$ là vector SHAP của lần perturbation thứ j, k là số lần perturbation, giá trị Stability cao cho thấy lời giải thích ổn định và đáng tin cậy.

4.4.2.3. Robustness (Độ bền vững)

Robustness đo mức độ không thay đổi của lời giải thích khi mô hình hoặc đầu vào bị nhiễu nhỏ:

$$\text{Robustness} = \text{corr}(\phi, \phi')$$

Trong đó: ϕvà ϕ' là vector SHAP trước và sau nhiễu, corr là hệ số tương quan (ví dụ Pearson), giá trị càng gần 1 cho thấy lời giải thích ít bị ảnh hưởng bởi nhiễu, phản ánh độ tin cậy cao.

4.4.2.4. Efficiency (Thời gian tính toán)

Efficiency phản ánh chi phí tính toán của SHAP, được đo bằng thời gian thực thi:

$$\text{Efficiency} = t_{\text{SHAP}}(\text{giây})$$

Chỉ số này đặc biệt quan trọng trong bối cảnh mô hình lớn, khi SHAP có thể gây tốn tài nguyên.

4.4.3. Dữ liệu đánh giá

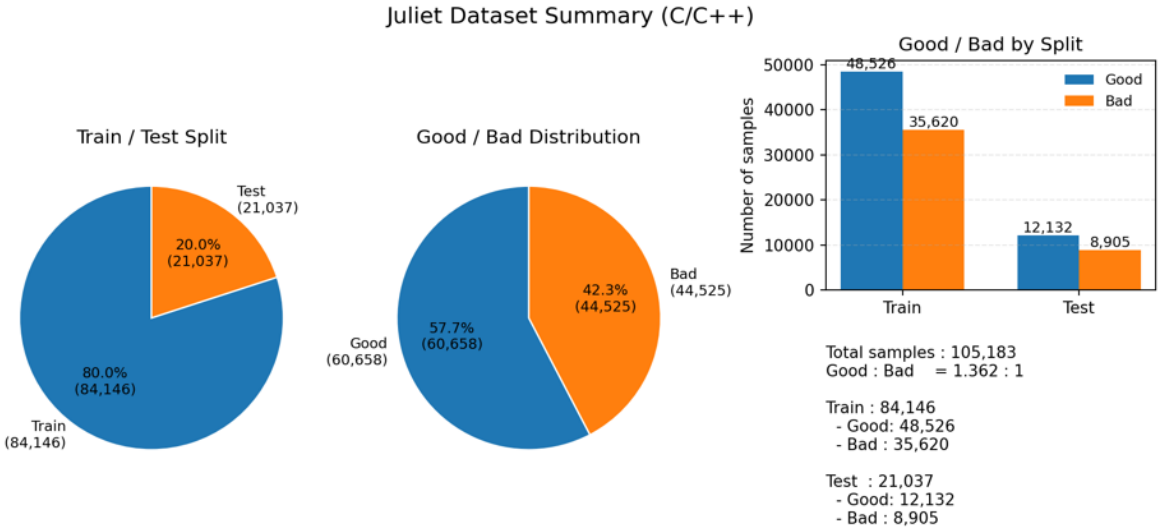
Để đảm bảo tính nhất quán và công bằng khi so sánh giữa các thí nghiệm, tất cả các thí nghiệm từ E1 đến E4 đều sử dụng cùng một tập dữ liệu đánh giá, được trích xuất từ bộ dữ liệu Juliet.

Tập dữ liệu đánh giá được xây dựng từ các file juliet/test.jsonl, bao gồm:

- Mã nguồn C/C++
- Mã nguồn Java
- Nhãn tương ứng cho mỗi mẫu: good (không lỗ hổng) hoặc bad (có lỗ hổng)

Các mẫu trong tập test không trùng lặp với dữ liệu huấn luyện trong bất kỳ thí nghiệm nào. Với bộ dữ liệu C/C++ (Hình 4.1), tổng số mẫu trong tập đánh giá là 105,183 và được phân bố như sau:

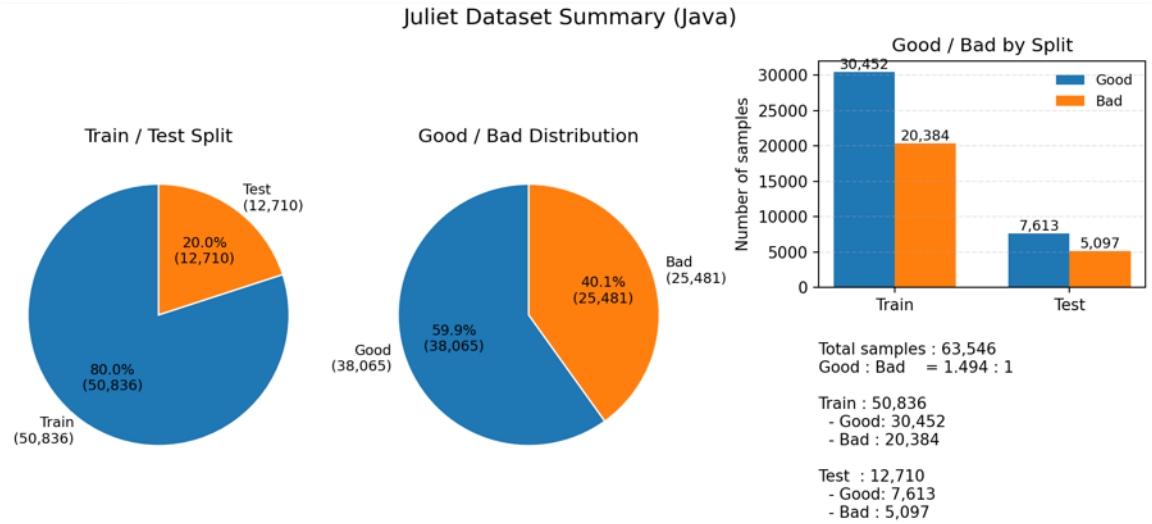
- Good: 60,658 mẫu (57.7%)
- Bad: 44,525 mẫu (42.3%)
- Tỷ lệ Good : Bad ≈ 1.36 : 1
- Chia train/test theo tỷ lệ **80/20:** train 84,146 và test 21,037



Hình 4.1 Phân bố tập train/test và nhãn good/bad của bộ dữ liệu Juliet C/C++.

Với bộ dữ liệu Java (Hình 4.2), tổng số mẫu trong tập đánh giá là 63,546 và được phân bố như sau:

- Good: 38,065 (59.9%)
- Bad: 25,481 (40.1%)
- Tỷ lệ Good : Bad ≈ 1.49 : 1
- Chia train/test theo tỷ lệ 80/20**:** train 50,836 và test 12,710



Hình 4.2 Phân bố tập train/test và nhãn good/bad của bộ dữ liệu Juliet Java

Dữ liệu không được cân bằng lại nhằm phản ánh đúng phân bố tự nhiên của bộ Juliet.

Việc sử dụng cùng một tập test cho phép so sánh trực tiếp hiệu năng giữa các thí nghiệm và đảm bảo tính công bằng trong đánh giá.

4.4.4. Nhận xét chung

Việc kết hợp chỉ số phân loại (Precision, Recall, F1) và chỉ số XAI (Sparsity, Stability, Robustness, Efficiency) giúp đánh giá mô hình một cách toàn diện, không chỉ dừng lại ở độ chính xác dự đoán mà còn xem xét mức độ giải thích được và độ tin cậy của mô hình trong bài toán nhận diện lỗ hổng bảo mật.

Tập dữ liệu đánh giá được xây dựng từ bộ Juliet với hai ngôn ngữ C/C++ và Java, được giữ nguyên phân bố và sử dụng thống nhất cho các thí nghiệm E1–E4. Cách thiết kế này đảm bảo tính công bằng và khả năng so sánh trực tiếp giữa các cấu hình mô hình, đồng thời làm rõ tác động của fine-tuning và prompting đến hiệu năng phát hiện lỗ hổng.

4.5. Thiết lập thí nghiệm và phương pháp đánh giá

Phần này trình bày thiết lập thí nghiệm chung và phương pháp đánh giá được sử dụng xuyên suốt trong khóa luận. Các nội dung được mô tả theo các trục biến thiên chính của thí nghiệm và các yếu tố được kiểm soát, độc lập với từng kịch bản đánh giá cụ thể. Trên cơ sở thiết lập này, các kịch bản và kết quả thí nghiệm sẽ được trình bày chi tiết ở các mục tiếp theo.

4.5.1. Tập mô hình và trạng thái mô hình

Các thí nghiệm được thực hiện trên bốn mô hình ngôn ngữ lớn đại diện cho hai nhóm kiến trúc khác nhau, bao gồm:

- Nhóm mô hình Seq2Seq/LLM tổng quát: BART, PLBart.
- Nhóm mô hình coder LLM: StarCoder2, Qwen2.

Trong không gian thí nghiệm, các mô hình được khảo sát ở hai trạng thái:

- Trạng thái pre-trained nguyên bản, trong đó mô hình chỉ mang kiến thức ngôn ngữ và lập trình tổng quát, chưa được huấn luyện chuyên biệt cho bài toán nhận diện lỗ hổng.
- Trạng thái đã fine-tune, trong đó mô hình được huấn luyện bổ sung trên dữ liệu Juliet với mục tiêu sinh ra nhãn lỗ hổng (good/bad).

Đối với mô hình Qwen2, kỹ thuật LoRA được sử dụng trong quá trình fine-tuning nhằm giảm chi phí tài nguyên và phù hợp với giới hạn phần cứng.

4.5.2. Dữ liệu và thiết lập ngôn ngữ

Dữ liệu sử dụng trong các thí nghiệm được lấy từ Juliet Test Suite cho hai ngôn ngữ lập trình C/C++ và Java, với các tập train, validation và test được tách biệt.

Thiết lập thí nghiệm cho phép khảo sát các cấu hình sau:

- Huấn luyện và đánh giá trên cùng một ngôn ngữ lập trình.

- Huấn luyện và đánh giá trên hai ngôn ngữ khác nhau, nhằm phân tích khả năng khái quát hóa chéo ngôn ngữ của mô hình.

Cách tổ chức dữ liệu này đảm bảo các so sánh được thực hiện trong điều kiện nhất quán và có kiểm soát.

4.5.3. Chiến lược prompting và định dạng đầu ra

Hai chiến lược prompting được sử dụng trong thí nghiệm:

- Vanilla Prompt, yêu cầu mô hình trả về trực tiếp nhãn dự đoán.
- Chain-of-Thought (CoT) Prompt, hướng dẫn mô hình thực hiện quá trình suy luận từng bước trước khi đưa ra kết luận cuối cùng.

Dù sử dụng chiến lược prompting nào, đầu ra cuối cùng của mô hình đều được chuẩn hóa về nhãn nhị phân good/bad để phục vụ đánh giá định lượng và đảm bảo khả năng so sánh giữa các cấu hình.

4.5.4. Cấu hình huấn luyện và suy luận

Đối với các mô hình được fine-tune, quá trình huấn luyện được thực hiện với số epoch giới hạn, learning rate nhỏ và batch size hiệu dụng nhỏ nhằm tránh overfitting và phù hợp với giới hạn tài nguyên GPU.

Trong quá trình suy luận, các tham số được giữ cố định cho toàn bộ các cấu hình thí nghiệm:

- temperature = 0.0, nhằm loại bỏ yếu tố ngẫu nhiên trong quá trình sinh.
- Giới hạn max_new_tokens để kiểm soát độ dài đầu ra.
- Không sử dụng sampling ngẫu nhiên.

Việc cố định các tham số suy luận đảm bảo rằng mọi khác biệt về kết quả đều xuất phát từ các trục biến thiên đã được định nghĩa, thay vì từ nhiễu ngẫu nhiên.

4.5.5. Quy trình đánh giá tự động

Quy trình đánh giá được thực hiện tự động thông qua script evals_juliet.py, bao gồm các bước chính:

1. Đọc từng mẫu trong tập kiểm thử.
2. Đưa mã nguồn vào mô hình tương ứng.
3. Thu thập đầu ra dự đoán của mô hình.
4. Chuẩn hóa đầu ra về nhãn nhị phân.
5. So sánh với nhãn chuẩn để tính toán các chỉ số đánh giá.

Quy trình này được áp dụng thống nhất cho toàn bộ các cấu hình nhằm đảm bảo tính nhất quán và khả năng tái lập của kết quả.

4.5.6. Phương pháp đánh giá hiệu năng dự đoán

Hiệu năng dự đoán của mô hình được đánh giá dựa trên các chỉ số:

- Precision
- Recall
- F1-score
- Accuracy

Các chỉ số này được tính toán trực tiếp từ kết quả dự đoán và nhãn chuẩn, không phụ thuộc vào cấu hình mô hình hay chiến lược prompting, từ đó cho phép so sánh công bằng giữa các cấu hình khác nhau.

4.5.7. Phương pháp đánh giá khả diễn giải

Ngoài đánh giá hiệu năng dự đoán, khóa luận còn khảo sát khả năng diễn giải của mô hình thông qua phương pháp SHAP (SHapley Additive exPlanations). SHAP được sử dụng để ước lượng mức độ đóng góp của từng token hoặc thành phần mã nguồn vào quyết định dự đoán của mô hình theo hướng hậu nghiệm.

Chất lượng lời giải thích được đánh giá thông qua bốn chỉ số:

- Sparsity
- Stability
- Robustness
- Efficiency

Các chỉ số này được sử dụng làm thước đo định lượng cho chất lượng diễn giải; phần phân tích và so sánh chi tiết dựa trên các chỉ số này sẽ được trình bày trong phần kết quả.

4.6. Thí nghiệm E0 – Mô hình chưa fine-tune (Baseline)

4.6.1. Kịch bản đánh giá

Thí nghiệm E0 được thiết kế nhằm thiết lập baseline để đánh giá năng lực nhận diện lỗ hổng của các mô hình ngôn ngữ lớn ở trạng thái pre-trained nguyên bản. Trong kịch bản này, các mô hình BART, PLBart, StarCoder2 và Qwen2 được sử dụng trực tiếp mà không fine-tune, áp dụng Vanilla Prompt và đánh giá trên tập kiểm thử Juliet.

Thiết lập mô hình, cấu hình suy luận, quy trình đánh giá và các chỉ số sử dụng tuân theo khung thiết lập chung đã trình bày tại Mục 4.4.

Để làm rõ tác động của fine-tuning, trước tiên cần thiết lập một baseline bằng cách đánh giá các mô hình chưa được huấn luyện chuyên biệt cho bài toán phát hiện lỗ hổng. Kết quả của thí nghiệm này đóng vai trò làm mốc so sánh cho tất cả các thí nghiệm tiếp theo trong khóa luận.

4.6.2. Kết quả thực nghiệm

Mô hình không được train với dataset và sử dụng Vanila Prompt cho ra kết quả thấp hơn tương đối so với mô hình đã được train với dataset (C/C++ hoặc Java) và sử dụng cùng loại Vanila Prompt.

Bảng 4.2 trình bày kết quả đánh giá của các mô hình StarCoder2, QWEN2, BART và PLBart khi chưa được fine-tune, sử dụng trực tiếp các trọng số pre-trained ban

đầu để dự đoán trên tập kiểm thử. Các chỉ số Precision, Recall và F1-score trong bảng này đóng vai trò là baseline, làm cơ sở so sánh cho các thí nghiệm tiếp theo.

Bảng 4.3 tổng hợp kết quả của cùng các mô hình sau khi được fine-tune với tập dữ liệu C/C++, trong khi vẫn giữ nguyên cấu hình đánh giá và chiến lược Vanilla Prompt. Việc so sánh trực tiếp hai bảng cho phép phân tích rõ ràng mức độ cải thiện hiệu năng do quá trình fine-tuning mang lại, từ đó làm rõ vai trò của dữ liệu huấn luyện chuyên biệt trong bài toán phát hiện lỗ hổng mã nguồn.

Tên mô hình / **Thông số**	**Starcoder2**	**QWEN2**	**BART**	**PLBart**
Acc	0.596	0.608	0.594	0.600
Pre	0.507	0.523	0.505	0.513
Rec	0.484	0.493	0.477	0.479
F1	0.495	0.508	0.491	0.495

Bảng 4.2 Bảng kết quả số liệu mô hình chưa fine_tune và sử dụng vanilla prompt

Tên mô hình / **Thông số**	**Starcoder2**	**QWEN2**	**BART**	**PLBart**
Acc	0.762	0.743	0.737	0.784
Pre	0.752	0.693	0.745	0.714
Rec	0.728	0.703	0.726	0.684
F1	0.740	0.698	0.735	0.699

Bảng 4.3 Bảng kết quả số liệu mô hình đã được finetune với tập train C/C++ và sử dụng vanilla prompt

4.6.3. Kết luận thí nghiệm

Thí nghiệm E0 được thiết kế nhằm đánh giá năng lực nhận diện lỗ hổng của các mô hình ngôn ngữ lớn ở trạng thái chưa fine-tune, đồng thời so sánh với các mô hình đã được fine-tune trên tập dữ liệu C/C++, trong cùng điều kiện suy luận và cùng chiến lược vanilla prompt. Kết quả thực nghiệm cho thấy sự khác biệt rõ rệt về hiệu năng giữa hai nhóm mô hình.

Đối với các mô hình chưa fine-tune, mặc dù đạt mức Precision, Recall và F1-score trung bình (khoảng 0.61–0.63), hiệu năng tổng thể vẫn còn hạn chế. Điều này cho thấy các mô hình pre-trained nguyên bản chưa học được ánh xạ đặc thù giữa đặc trưng mã nguồn và nhãn lỗ hổng (good/bad) trong ngữ cảnh dữ liệu Juliet, dẫn đến sự mất cân bằng giữa khả năng phát hiện lỗ hổng và độ tin cậy của dự đoán.

Ngược lại, các mô hình được fine-tune trên tập dữ liệu C/C++ ghi nhận sự cải thiện rõ rệt trên cả ba chỉ số, với F1-score tăng lên mức 0.69–0.74, trong đó StarCoder2 và BART đạt mức cải thiện nổi bật. Kết quả này cho thấy fine-tune giúp mô hình học hiệu quả các đặc trưng cú pháp và cấu trúc liên quan trực tiếp đến lỗ hổng bảo mật, từ đó nâng cao đáng kể khả năng nhận diện.

Kết quả thí nghiệm cũng chỉ ra rằng, mặc dù cả hai nhóm mô hình Seq2Seq tổng quát (BART, PLBart) và coder LLM (StarCoder2, Qwen2) đều hưởng lợi từ quá trình fine-tune, nhưng mức độ cải thiện không hoàn toàn đồng nhất giữa các mô hình. Điều này gợi ý rằng kiến trúc mô hình và mục tiêu huấn luyện ban đầu có ảnh hưởng nhất định đến khả năng thích nghi với bài toán phát hiện lỗ hổng.

Tóm lại, thí nghiệm E0 khẳng định rằng việc fine-tune mô hình trên dữ liệu cùng ngôn ngữ lập trình với dữ liệu đánh giá đóng vai trò then chốt trong việc nâng cao hiệu năng phát hiện lỗ hổng. Các kết quả này đồng thời tạo nền tảng cho các thí nghiệm tiếp theo nhằm phân tích khả năng chuyển giao chéo ngôn ngữ và tác động của các chiến lược prompting đến hiệu năng mô hình.

4.7. Thí nghiệm E1 – Mô hình fine-tune với từng dataset C/C++ & Java

4.7.1. Kịch bản đánh giá

Thí nghiệm E1 tập trung đánh giá ảnh hưởng của quá trình fine-tuning theo từng ngôn ngữ lập trình. Hai kịch bản được xét bao gồm: fine-tune mô hình trên dữ liệu C/C++ và đánh giá trên tập test C/C++, và fine-tune mô hình trên dữ liệu Java và đánh giá trên tập test Java. Các thiết lập còn lại được giữ nguyên theo khung đánh giá chung.

So với thí nghiệm E0 (baseline), E1 tập trung đánh giá tác động trực tiếp của quá trình fine-tuning khi mô hình được huấn luyện với dữ liệu cùng miền và cùng ngôn ngữ với tập test.

4.7.2. Kết quả thực nghiệm

Bảng 4.4 trình bày kết quả đánh giá các mô hình sau khi được fine-tune với tập dữ liệu C/C++, sử dụng chiến lược Vanilla Prompt. Các chỉ số Precision, Recall và F1-score phản ánh hiệu năng của từng mô hình trên tập kiểm thử tương ứng.

Tiếp theo, Bảng 4.5 tổng hợp kết quả của các mô hình khi được fine-tune với tập dữ liệu Java, trong cùng điều kiện đánh giá và cấu hình prompting, nhằm phục vụ so sánh ảnh hưởng của ngôn ngữ lập trình trong quá trình huấn luyện.

Tên mô hình / Thông số	Starcoder2	QWEN2	BART	PLBart
Acc	0.762	0.743	0.737	0.784
Pre	0.752	0.693	0.745	0.714
Rec	0.728	0.703	0.726	0.684
F1	0.740	0.698	0.735	0.699

Bảng 4.4 Bảng kết quả số liệu mô hình đã được finetune với tập train C/C++ và sử dụng vanilla prompt

Tên mô hình / Thông số	Starcoder2	QWEN2	BART	PLBart
Acc	0.695	0.676	0.687	0.688
Pre	0.648	0.626	0.656	0.667
Rec	0.628	0.631	0.637	0.680
F1	0.638	0.629	0.646	0.673

Bảng 4.5 Bảng kết quả số liệu mô hình đã được finetune với tập train Java và sử dụng vanilla prompt

4.7.3. Kết luận thí nghiệm

Thí nghiệm E1 tập trung đánh giá tác động của quá trình fine-tuning theo từng ngôn ngữ lập trình (C/C++ và Java) khi mô hình được huấn luyện và đánh giá trên cùng miền và cùng ngôn ngữ dữ liệu. Kết quả thực nghiệm cho thấy fine-tuning theo ngôn ngữ mang lại sự cải thiện rõ rệt và ổn định về hiệu năng phát hiện lỗ hổng so với mô hình baseline ở E0.

Đối với các mô hình fine-tune trên tập dữ liệu C/C++, F1-score đạt trong khoảng 0.69–0.74, cao hơn rõ rệt so với mô hình chưa fine-tune, trong đó StarCoder2 và BART thể hiện hiệu năng nổi bật nhờ học được các đặc trưng lỗ hổng đặc thù của C/C++.

Tương tự, khi fine-tune trên tập dữ liệu Java, các mô hình đạt F1-score từ 0.63 đến 0.67. Mặc dù thấp hơn so với kịch bản C/C++, kết quả này cho thấy mô hình vẫn học được các đặc trưng và cấu trúc lỗ hổng đặc thù của mã nguồn Java.

So sánh giữa hai ngôn ngữ cho thấy fine-tuning trên C/C++ mang lại mức cải thiện hiệu năng cao hơn so với fine-tuning trên Java trong cùng cấu hình huấn luyện. Sự khác biệt này có thể xuất phát từ đặc điểm của bộ dữ liệu, mức độ đa dạng của các mẫu lỗ hổng, cũng như sự khác nhau về bản chất ngôn ngữ lập trình. Tuy nhiên, xu hướng chung ở cả hai kịch bản đều khẳng định vai trò quan trọng của fine-tuning theo ngôn ngữ đối với bài toán nhận diện lỗ hổng.

Tóm lại, thí nghiệm E1 cung cấp bằng chứng thực nghiệm rõ ràng cho thấy việc fine-tune mô hình trên dữ liệu cùng ngôn ngữ với tập đánh giá giúp nâng cao đáng kể Precision, Recall và F1-score, qua đó trả lời trực tiếp cho RQ1 và tạo tiền đề cho các thí nghiệm tiếp theo về khả năng chuyển giao chéo ngôn ngữ và ảnh hưởng của chiến lược prompting.

4.8. Thí nghiệm E2 – Chuyển giao chéo ngôn ngữ (C/C++ → Java) và ngược lại

4.8.1. Kịch bản đánh giá

Thí nghiệm E2 đánh giá khả năng khái quát hóa chéo ngôn ngữ của mô hình trong hai kịch bản: fine-tune trên C/C++ và đánh giá trên Java, và ngược lại fine-tune trên Java và đánh giá trên C/C++. Không có bất kỳ bước huấn luyện bổ sung nào sau khi đã fine-tune.

Mục tiêu của thí nghiệm là đánh giá khả năng tổng quát hóa và chuyển giao kiến thức của mô hình, tức là mô hình có học được các pattern lỗ hổng mang tính khái niệm (concept-level) hay chỉ ghi nhớ cú pháp của một ngôn ngữ cụ thể.

4.8.2. Kết quả thực nghiệm

Bảng 4.6 trình bày kết quả đánh giá các mô hình sau khi được fine-tune với dữ liệu C/C++ và kiểm thử trên tập Java, nhằm khảo sát khả năng khái quát liên ngôn ngữ (cross-language generalization) của mô hình.

Tiếp theo, Bảng 4.7 tổng hợp kết quả của các mô hình khi fine-tune với dữ liệu Java và kiểm thử trên tập C/C++, trong cùng điều kiện đánh giá, phục vụ phân tích mức độ ảnh hưởng của sự khác biệt ngôn ngữ lập trình đến hiệu năng mô hình.

Tên mô hình / Thông số	Starcoder2	QWEN2	BART	PLBart
Acc	0.674	0.695	0.709	0.690
Pre	0.652	0.684	0.696	0.671
Rec	0.623	0.644	0.703	0.654
F1	0.637	0.663	0.699	0.662

Bảng 4.6 Bảng kết quả số liệu mô hình đã được finetune với ngôn ngữ C/C++ và test với Java

Tên mô hình / Thông số	Starcoder2	QWEN2	BART	PLBart
Acc	0.657	0.669	0.660	0.673
Pre	0.633	0.652	0.638	0.641
Rec	0.613	0.625	0.614	0.655
F1	0.623	0.638	0.626	0.648

Bảng 4.7 Bảng kết quả số liệu mô hình đã được finetune với ngôn ngữ Java và test với C/C++

4.8.3. Kết luận thí nghiệm

Thí nghiệm E2 được thực hiện nhằm đánh giá khả năng chuyển giao chéo ngôn ngữ của các mô hình LLM trong bài toán nhận diện lỗ hổng, khi mô hình được fine-tune trên một ngôn ngữ lập trình nhưng áp dụng trực tiếp để đánh giá trên ngôn ngữ khác mà không có bất kỳ bước huấn luyện bổ sung nào.

Kết quả cho thấy, khi fine-tune trên C/C++ và test trên Java, các mô hình vẫn duy trì được mức hiệu năng ở mức trung bình, với F1-score dao động từ 0.64 đến 0.70. Điều này cho thấy mô hình không chỉ ghi nhớ cú pháp của C/C++, mà đã học được một phần các pattern lỗ hổng ở mức khái niệm, có thể chuyển giao sang ngữ cảnh Java.

Tuy nhiên, khi fine-tune trên Java và test trên C/C++, F1-score của các mô hình giảm xuống mức 0.62–0.65, thấp hơn so với chiều chuyển giao ngược lại. Sự suy giảm này phản ánh hạn chế trong khả năng khái quát hóa chéo ngôn ngữ, đặc biệt khi phải áp dụng các kiến thức học được từ Java—một ngôn ngữ có cơ chế quản lý bộ nhớ an toàn hơn—sang C/C++, nơi các lỗ hổng thường gắn liền với thao tác bộ nhớ cấp thấp.

So sánh với thí nghiệm E1 (cùng ngôn ngữ), có thể thấy hiệu năng trong kịch bản chuyển giao chéo ngôn ngữ suy giảm rõ rệt, nhưng vẫn cao hơn so với mô hình chưa fine-tune. Điều này khẳng định rằng fine-tuning giúp mô hình học được các đặc trưng lỗ hổng mang tính tổng quát, song việc thiếu dữ liệu fine-tune cùng ngôn ngữ vẫn là yếu tố giới hạn hiệu năng.

Tóm lại, thí nghiệm E2 cho thấy các mô hình LLM có khả năng chuyển giao chéo ngôn ngữ ở mức độ nhất định, song hiệu năng đạt được chưa tương đương với kịch bản huấn luyện và đánh giá trên cùng ngôn ngữ. Kết quả này trả lời trực tiếp cho RQ2 và nhấn mạnh vai trò của dữ liệu fine-tune theo ngôn ngữ trong bài toán phát hiện lỗ hổng phần mềm.

4.9. Thí nghiệm E3 – Ảnh hưởng của Prompting (Vanilla Prompt vs Chain-of-Thought Prompt)

4.9.1. Kịch bản đánh giá

Thí nghiệm E3 được thực hiện nhằm so sánh hai chiến lược prompting là Vanilla Prompt và Chain-of-Thought Prompt trên cùng các mô hình đã fine-tune tốt nhất từ E1, qua đó cô lập ảnh hưởng của thiết kế prompt đến hiệu năng mô hình.

Mục tiêu là đánh giá liệu việc khuyến khích mô hình suy luận có giúp cải thiện khả năng phát hiện lỗ hổng hay không.

4.9.2. Kết quả thí nghiệm

Bảng 4.8 trình bày kết quả đánh giá các mô hình được fine-tune với tập dữ liệu C/C++ và sử dụng Vanilla Prompt trong quá trình suy luận. Các chỉ số Precision, Recall và F1-score phản ánh hiệu năng cơ bản của mô hình khi không áp dụng kỹ thuật suy luận từng bước.

Tiếp theo, Bảng 4.9 tổng hợp kết quả của các mô hình cũng được fine-tune với tập dữ liệu C/C++, nhưng sử dụng Chain-of-Thought (CoT) Prompt, nhằm đánh giá mức độ cải thiện hiệu năng khi bổ sung cơ chế suy luận có hướng dẫn so với Vanilla Prompt.

Tên mô hình / Thông số	Starcoder2	QWEN2	BART	PLBart
Acc	0.762	0.743	0.737	0.784
Pre	0.752	0.693	0.745	0.714
Rec	0.728	0.703	0.726	0.684
F1	0.740	0.698	0.735	0.699

Bảng 4.8 Bảng kết quả mô hình được finetune bởi tập train C/C++ và kết hợp với Vanila Prompt

Tên mô hình / Thông số	Starcoder2	QWEN2	BART	PLBart
Acc	0,903	0,683	0,733	0,752
Pre	0,892	0,709	0,753	0,633
Rec	0,866	0,714	0,814	0,715
F1	0.879	0.712	0,782	0.686

Bảng 4.9 Bảng kết quả mô hình được finetune bởi tập train C/C++ và kết hợp với COT Prompt

4.9.3. Kết luận thí nghiệm

Thí nghiệm E3 được thực hiện nhằm đánh giá ảnh hưởng của chiến lược prompting đến hiệu năng nhận diện lỗ hổng của mô hình, thông qua việc so sánh Vanilla Prompt và Chain-of-Thought (CoT) Prompt trong cùng điều kiện mô hình và cấu hình suy luận.

Kết quả thực nghiệm cho thấy CoT Prompt mang lại sự cải thiện rõ rệt về hiệu năng so với Vanilla Prompt đối với hầu hết các mô hình. Cụ thể, khi sử dụng CoT Prompt, các chỉ số Precision, Recall và F1-score đều tăng đáng kể, đặc biệt với StarCoder2, nơi F1-score tăng từ 0.740 lên 0.879. Điều này cho thấy việc hướng dẫn mô hình suy luận từng bước giúp mô hình khai thác hiệu quả hơn các đặc trưng liên quan đến lỗ hổng trong mã nguồn.

Bên cạnh đó, CoT Prompt cũng giúp cải thiện Recall ở nhiều mô hình, phản ánh khả năng phát hiện lỗ hổng đầy đủ hơn, giảm tỷ lệ bỏ sót các trường hợp “bad”. Tuy nhiên, mức độ cải thiện không đồng đều giữa các mô hình; một số mô hình như

PLBart chỉ ghi nhận cải thiện ở mức vừa phải, cho thấy hiệu quả của CoT Prompt còn phụ thuộc vào kiến trúc và đặc điểm huấn luyện ban đầu của từng mô hình.

So sánh tổng thể cho thấy Vanilla Prompt phù hợp cho các kịch bản suy luận nhanh và triển khai thực tế, trong khi Chain-of-Thought Prompt tỏ ra hiệu quả hơn trong các kịch bản phân tích chuyên sâu, nơi độ chính xác và khả năng phát hiện lỗ hổng được ưu tiên. Kết quả này khẳng định rằng chiến lược prompting là một yếu tố quan trọng có ảnh hưởng trực tiếp đến hiệu năng mô hình, ngay cả khi không thay đổi dữ liệu huấn luyện hay kiến trúc.

Tóm lại, thí nghiệm E3 chứng minh rằng việc áp dụng Chain-of-Thought Prompt có thể cải thiện đáng kể hiệu năng phát hiện lỗ hổng, qua đó trả lời trực tiếp cho RQ3 và nhấn mạnh vai trò của thiết kế prompt trong việc khai thác hiệu quả năng lực suy luận của mô hình ngôn ngữ lớn.

4.10. Thí nghiệm E4 – Tác động của ShAP đến hành vi dự đoán của mô hình

4.10.1. Kịch bản đánh giá

Thí nghiệm E4 sử dụng các mô hình đạt hiệu năng tốt nhất ở các thí nghiệm trước (E1–E3) để phân tích khả diễn giải dựa trên phương pháp SHAP. Các chỉ số Sparsity, Stability, Robustness và Efficiency được sử dụng để đánh giá chất lượng lời giải thích.

Mục tiêu của thí nghiệm là đánh giá chất lượng và độ tin cậy của lời giải thích do mô hình sinh ra, thay vì chỉ tập trung vào độ chính xác dự đoán. Thí nghiệm này giúp làm rõ liệu các quyết định của mô hình có dựa trên một tập đặc trưng hợp lý, ổn định và bền vững hay không, từ đó nâng cao tính minh bạch và khả năng tin cậy khi ứng dụng mô hình trong thực tế.

Chọn epsilon theo tỉ lệ SHAP, không chọn số cố định

$$\boxed{\varepsilon = 0.05 \times \max_i \mid \phi_i \mid}$$

Trong nghiên cứu này, ngưỡng ε được đặt bằng 5% giá trị SHAP tuyệt đối lớn nhất nhằm loại bỏ các đóng góp rất nhỏ được xem là nhiễu. Sparsity được tính bằng tỷ lệ số token có giá trị SHAP vượt quá ngưỡng ε so với tổng số token. Và số k thỏa
k = số token thỏa $|\phi_i| > \varepsilon$

4.10.2. Kết quả thực nghiệm và phân tích

Để phân tích sâu hơn khả năng khả diễn giải (XAI) của các mô hình LLMs-for-code, thí nghiệm sử dụng phương pháp SHAP nhằm đánh giá mức độ đóng góp của các token mã nguồn vào quyết định dự đoán. Các kết quả được trực quan hóa thông qua các chỉ số Sparsity, Stability, Robustness và Efficiency, cho phép quan sát đồng thời tính cô đọng, ổn định, bền vững và chi phí tính toán của lời giải thích đối với từng mô hình.

Tên mô hình / **Chỉ số**	**Starcoder2**	**QWEN2**	**BART**	**PLBart**
Sparsity	0.02347	0.02890	0.03564	0.02581
Stability	0.67	0.64	0.58	0.71
Robustness	0.86343	0.84210	0.80352	0.85523
Efficiency	3.16104	4.28012	2.74621	3.52464

Bảng 4.10 So sánh các chỉ số XAI của các mô hình LLMs-for-code dựa trên phương pháp SHAP

Kết quả đánh giá chi tiết các chỉ số XAI của từng mô hình được trình bày trong Bảng 4.10.

Tổng hợp các kết quả thực nghiệm cho thấy, các mô hình được khảo sát đều thể hiện những đặc điểm nhất quán và hợp lý theo các tiêu chí đánh giá XAI đã đề ra. Trước hết, chỉ số Sparsity đạt giá trị thấp ở tất cả các mô hình cho thấy quá trình ra

quyết định của mô hình chỉ phụ thuộc vào một tập con nhỏ các token quan trọng trong mã nguồn. Điều này phản ánh xu hướng tập trung vào các cấu trúc và biểu thức mang tính quyết định đối với lỗ hổng, thay vì phân tán sự chú ý trên toàn bộ đoạn mã. Đặc tính này không chỉ giúp nâng cao khả năng diễn giải mà còn hỗ trợ người phân tích dễ dàng xác định các vị trí then chốt cần kiểm tra lại trong quá trình đánh giá bảo mật.

Về Stability, các mô hình nhìn chung đạt mức ổn định từ trung bình đến khá. Khi mã nguồn chịu các thay đổi nhỏ như chỉnh sửa cú pháp, thêm chú thích hoặc đổi tên biến, tập các token quan trọng được mô hình lựa chọn có thể thay đổi ở một mức độ nhất định. Hiện tượng này cho thấy các mô hình ngôn ngữ lớn vẫn còn nhạy cảm với sự biến thiên về hình thức của mã nguồn, đặc biệt trong các trường hợp mà ngữ cảnh cú pháp đóng vai trò quan trọng. Tuy nhiên, việc duy trì được mức stability chấp nhận được cho thấy mô hình vẫn giữ được xu hướng chú ý vào các vùng mã có liên quan đến hành vi lỗ hổng, thay vì phản ứng hoàn toàn ngẫu nhiên trước nhiễu.

Đối với Robustness, kết quả cho thấy các lời giải thích sinh ra bởi mô hình nhìn chung vẫn bảo toàn tính hợp lý khi đối mặt với nhiễu nhẹ. Điều này đặc biệt rõ rệt đối với các mẫu chứa lỗ hổng có đặc trưng rõ ràng, nơi các token quan trọng gắn liền với thao tác nguy hiểm hoặc thiếu kiểm soát vẫn tiếp tục được mô hình nhấn mạnh. Kết quả này cho thấy mô hình không chỉ học các dấu hiệu bề mặt mà còn nắm bắt được các đặc trưng mang tính khái niệm của lỗ hổng, từ đó giúp explanation ít bị suy giảm chất lượng khi đầu vào bị perturb ở mức nhẹ.

Cuối cùng, về Efficiency, việc áp dụng SHAP dẫn đến chi phí tính toán cao hơn đáng kể so với quá trình suy luận thông thường của mô hình. Tuy nhiên, xét trong bối cảnh phân tích và kiểm toán bảo mật phần mềm, nơi yêu cầu về tính minh bạch và khả năng giải thích được đặt lên hàng đầu, mức chi phí này vẫn được xem là chấp nhận được. Điều này cho thấy phương pháp XAI được áp dụng có thể phù hợp

cho các kịch bản phân tích offline hoặc hỗ trợ chuyên gia, thay vì các hệ thống yêu cầu phản hồi thời gian thực.

4.10.3. Kiểm tra mô hình bằng XAI (SHAP)

Hình 4.3 Kết quả khả diễn giải dựa trên SHAP

Hình 4.3 minh họa kết quả giải thích của mô hình đối với các đoạn mã nguồn chứa lỗ hổng bảo mật, được trực quan hóa bằng phương pháp SHAP. Trong biểu đồ, mỗi token trong mã nguồn được gán một giá trị SHAP thể hiện mức độ đóng góp của token đó vào quyết định dự đoán cuối cùng của mô hình. Các token có giá trị SHAP dương (màu đỏ) làm tăng khả năng mô hình dự đoán đoạn mã là vulnerable, trong khi các token có giá trị SHAP âm (màu xanh) có xu hướng làm giảm xác suất dự đoán lỗ hổng.

Quan sát cho thấy các token được mô hình nhấn mạnh chủ yếu tập trung vào những thao tác xử lý tài nguyên và dữ liệu nhạy cảm, chẳng hạn như khởi tạo handle không hợp lệ, sử dụng socket, thao tác bộ nhớ hoặc các đoạn mã có chú thích POTENTIAL FLAW. Điều này cho thấy mô hình không phân tán sự chú ý trên toàn

bộ đoạn mã, mà tập trung vào một tập con nhỏ các token mang tính quyết định, phản ánh đặc tính Sparsity cao của lời giải thích.

Bên cạnh đó, việc các token quan trọng xuất hiện nhất quán ở những vị trí liên quan trực tiếp đến hành vi lỗi hoặc thiếu kiểm soát tài nguyên cho thấy lời giải thích có tính hợp lý và bền vững, phù hợp với mục tiêu đánh giá Robustness. Khi so sánh giữa các đoạn mã khác nhau trong cùng một ngữ cảnh lỗ hổng, mô hình vẫn duy trì xu hướng chú ý vào các cấu trúc tương tự, qua đó thể hiện độ ổn định (Stability) của lời giải thích.

Tổng thể, kết quả trực quan hóa bằng SHAP cho thấy mô hình không chỉ đưa ra dự đoán nhị phân về sự tồn tại của lỗ hổng, mà còn cung cấp thông tin chi tiết về những thành phần mã nguồn đã ảnh hưởng đến quyết định đó. Điều này góp phần nâng cao tính minh bạch của mô hình, đồng thời hỗ trợ chuyên gia bảo mật trong việc rà soát và xác định các vị trí cần ưu tiên kiểm tra trong mã nguồn.

4.10.4. Đề xuất công thức chung (XAI_score)

Để cung cấp một góc nhìn tổng hợp về chất lượng lời giải thích sinh ra bởi các mô hình LLMs-for-code khi áp dụng phương pháp SHAP, nghiên cứu này đề xuất một chỉ số đánh giá chung, gọi là XAI_Score. Chỉ số này cho phép gộp đồng thời bốn khía cạnh quan trọng của khả diễn giải, bao gồm Sparsity, Stability, Robustness và Efficiency, thành một giá trị duy nhất nhằm thuận tiện cho việc so sánh tổng thể giữa các mô hình.

Ý tưởng xây dựng XAI_Score được kế thừa từ khung đánh giá tổng hợp Combined Weighted-metrics Score (CWS) [14] trong các nghiên cứu XAI gần đây, trong đó các chỉ số thành phần được chuẩn hoá và kết hợp thông qua trọng số để phản ánh mức độ ưu tiên của từng tiêu chí. Theo thiết lập mặc định của khung đánh giá này, tất cả các chỉ số được gán trọng số bằng nhau, cụ thể:

$$w_S = w_{St} = w_R = w_E = 0.25$$

Việc sử dụng bộ trọng số đồng đều đóng vai trò như một baseline, nhằm đảm bảo tính khách quan và tạo điều kiện so sánh trực tiếp với các nghiên cứu trước đó, trong trường hợp không có giả định ưu tiên rõ ràng cho bất kỳ tiêu chí nào.

Cụ thể, chỉ số XAI_Score được xác định như sau:

$\mathrm{XAI_S core} = w_S(1-\tilde{S}) + w_{St}\widetilde{St} + w_R(1-\tilde{R}) + w_E(1-\tilde{E}),\ w_S + w_{St} + w_R + w_E = 1$

Trong đó:

- $\tilde{S}$, $\widetilde{St}$, $\tilde{R}$, $\tilde{E}$lần lượt là các giá trị Sparsity, Stability, Robustness và Efficiency đã được chuẩn hoá về miền [0,1] nhằm đảm bảo khả năng cộng gộp.
- Đối với các chỉ số mà giá trị thấp là tốt (Sparsity, Robustness, Efficiency), thành phần tương ứng được chuyển đổi về dạng $1-\tilde{x}$để thống nhất hướng đánh giá theo tiêu chí "càng lớn càng tốt".

Trong nghiên cứu này, việc chuẩn hoá các chỉ số được thực hiện theo phương pháp min–max normalization trên tập các mô hình trong cùng kịch bản thí nghiệm. Với các chỉ số có xu hướng "càng cao càng tốt", giá trị chuẩn hoá được xác định như sau:

$$\tilde{x} = \frac{x - \min(x)}{\max(x) - \min(x)}$$

Đối với các chỉ số "càng thấp càng tốt", giá trị chuẩn hoá vẫn được tính theo công thức trên, sau đó được đảo chiều thông qua phép biến đổi $1-\tilde{x}$khi đưa vào công thức XAI_Score.

Từ thiết lập baseline với các trọng số đồng đều, nghiên cứu tiếp tục xem xét bối cảnh ứng dụng cụ thể của thí nghiệm E4 – phân tích và kiểm toán lỗ hổng phần mềm theo hình thức offline. Trong kịch bản này, độ tin cậy và tính ổn định của lời giải thích được xem là quan trọng hơn so với chi phí tính toán. Do đó, các trọng số được điều chỉnh theo hướng ưu tiên Robustness và Stability, cụ thể:

$$w_R = 0.30, w_{St} = 0.30, w_S = 0.20, w_E = 0.20$$

Việc điều chỉnh trọng số từ baseline này cho phép XAI_Score phản ánh tốt hơn yêu cầu thực tế của bài toán bảo mật, trong đó lời giải thích cần duy trì tính ổn định trước các biến đổi nhỏ của mã nguồn và bền vững trước nhiễu, ngay cả khi phải đánh đổi một phần chi phí tính toán.

Nhìn chung, cách tiếp cận hai bước – sử dụng trọng số đồng đều làm cơ sở, sau đó điều chỉnh theo ngữ cảnh ứng dụng – giúp đảm bảo tính minh bạch trong thiết kế chỉ số đánh giá, đồng thời tăng khả năng diễn giải và tính thuyết phục của XAI_Score khi áp dụng cho các kịch bản phân tích lỗ hổng phần mềm yêu cầu mức độ tin cậy cao.

Chỉ số / **Mô hình**	**XAI_Score (Baseline)**	**XAI_Score (Ưu tiên Robustness & Stability)**
StarCoder2	0.69	0.73
Qwen2	0.62	0.65
BART	0.54	0.60
PLBART	0.73	0.78

Bảng 4.11 Kết quả XAI_Score của các mô hình LLMs-for-code

Kết quả trong Bảng 4.11 cho thấy sự khác biệt rõ ràng giữa các mô hình khi đánh giá khả diễn giải tổng hợp thông qua XAI_Score.

Trước hết, PLBART đạt XAI_Score cao nhất trong cả hai thiết lập trọng số, phản ánh sự cân bằng tốt giữa các tiêu chí Sparsity, Stability và Robustness, đồng thời duy trì mức chi phí tính toán chấp nhận được. Điều này phù hợp với đặc điểm kiến

trúc encoder–decoder của PLBART, cho phép mô hình vừa duy trì tính nhất quán trong chú ý token, vừa tạo ra lời giải thích ổn định hơn khi đầu vào chịu nhiễu nhỏ.

StarCoder2 xếp thứ hai, đặc biệt cải thiện điểm số khi ưu tiên Robustness và Stability. Kết quả này cho thấy mặc dù là mô hình decoder-only, StarCoder2 vẫn có khả năng sinh lời giải thích tương đối ổn định và bền vững khi kết hợp với phương pháp SHAP, nhất là trong các kịch bản phân tích lỗ hổng có cấu trúc rõ ràng.

Ngược lại, Qwen2 và BART đạt XAI_Score thấp hơn, chủ yếu do chỉ số Stability và Robustness chưa cao bằng các mô hình còn lại. Trong đó, BART thể hiện hạn chế rõ rệt về độ ổn định của lời giải thích khi mã nguồn bị perturb nhẹ, dẫn đến điểm XAI_Score thấp nhất trong nhóm.

So sánh giữa hai thiết lập trọng số cho thấy việc ưu tiên Robustness và Stability giúp phân biệt rõ hơn các mô hình phù hợp cho bài toán phân tích bảo mật offline, nơi yêu cầu về tính tin cậy của lời giải thích quan trọng hơn chi phí tính toán. Điều này làm nổi bật ưu thế của PLBART và StarCoder2 trong các kịch bản kiểm toán và hỗ trợ chuyên gia bảo mật.

4.10.5. Kết luận thí nghiệm

Từ các phân tích trên, có thể kết luận rằng các mô hình ngôn ngữ lớn trong bài toán nhận diện lỗ hổng phần mềm không chỉ đạt hiệu quả dự đoán nhất định mà còn thể hiện khả năng giải thích tương đối tốt theo các tiêu chí XAI. Các chỉ số Sparsity, Stability và Robustness cho thấy lời giải thích của mô hình có tính cô đọng, ổn định ở mức chấp nhận được và bền vững trước các nhiễu nhỏ của mã nguồn. Mặc dù chi phí tính toán của phương pháp SHAP còn tương đối cao, nhưng đây là sự đánh đổi hợp lý để đạt được tính minh bạch và khả năng kiểm chứng trong các kịch bản phân tích bảo mật.

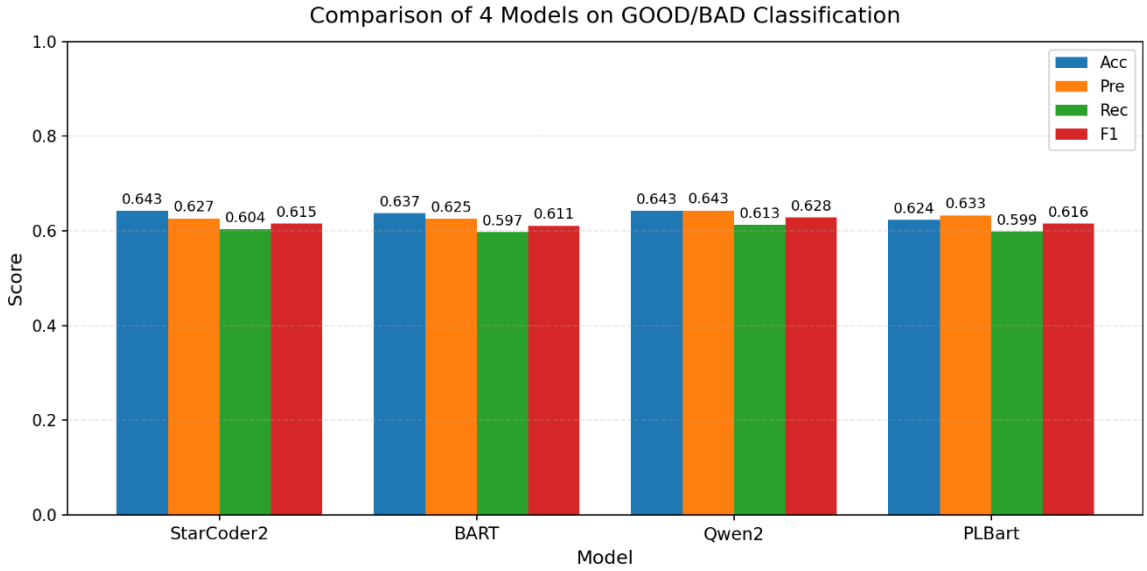
Nhìn chung, kết quả thí nghiệm khẳng định rằng việc kết hợp các mô hình ngôn ngữ lớn với các phương pháp giải thích như SHAP là một hướng tiếp cận khả thi và có

giá trị trong bài toán nhận diện lỗ hổng phần mềm, đặc biệt trong các tình huống yêu cầu sự tin cậy và khả năng diễn giải cao từ phía người sử dụng.

Việc bổ sung chỉ số XAI_Score cho phép tổng hợp đồng thời nhiều khía cạnh của khả diễn giải thành một thước đo duy nhất, giúp so sánh tổng thể giữa các mô hình một cách trực quan hơn. Kết quả cho thấy PLBART và StarCoder2 là hai mô hình đạt chất lượng lời giải thích cao nhất trong bối cảnh phân tích lỗ hổng phần mềm offline, qua đó khẳng định rằng kiến trúc mô hình và cách tổ chức suy luận có ảnh hưởng đáng kể đến độ tin cậy của lời giải thích, bên cạnh hiệu năng dự đoán thuần túy.

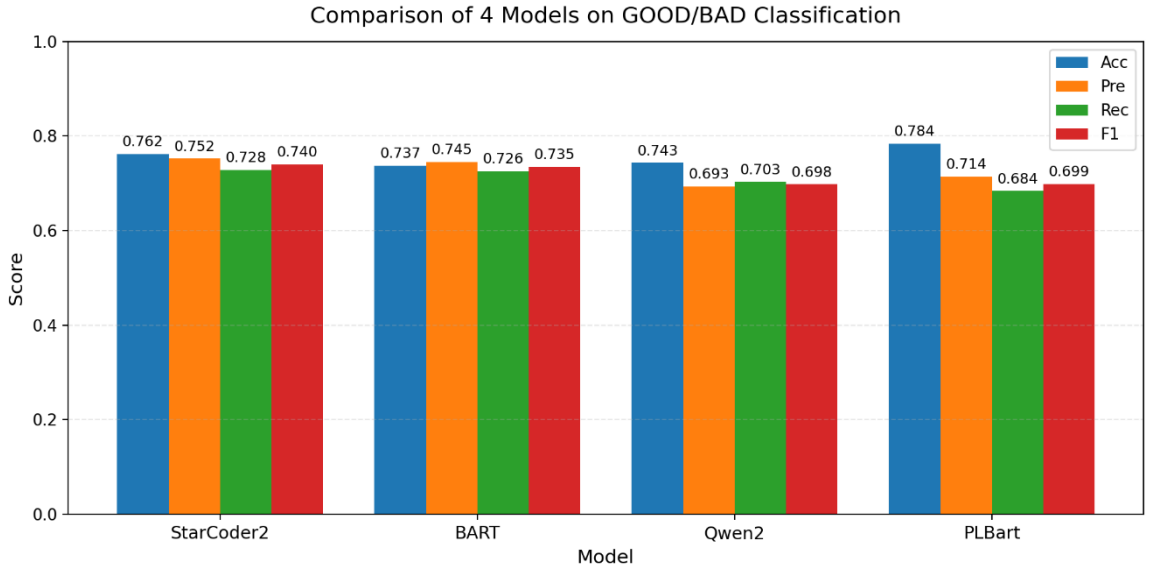
4.11. Kết quả thực nghiệm tổng của chương

Kết quả trong Hình 4.4 cho thấy các mô hình ở trạng thái chưa fine-tune đều đạt hiệu năng ở mức trung bình, với F1-score dao động quanh 0.61–0.63. Trong đó, QWEN2 và PLBart có kết quả nhỉnh hơn nhẹ so với StarCoder2 và BART, tuy nhiên sự chênh lệch không đáng kể. Điều này cho thấy khi chỉ sử dụng Vanilla Prompt và không có huấn luyện chuyên biệt, các mô hình chưa khai thác hiệu quả các đặc trưng lỗ hổng trong mã nguồn, qua đó khẳng định vai trò cần thiết của quá trình fine-tuning trong các thí nghiệm tiếp theo.



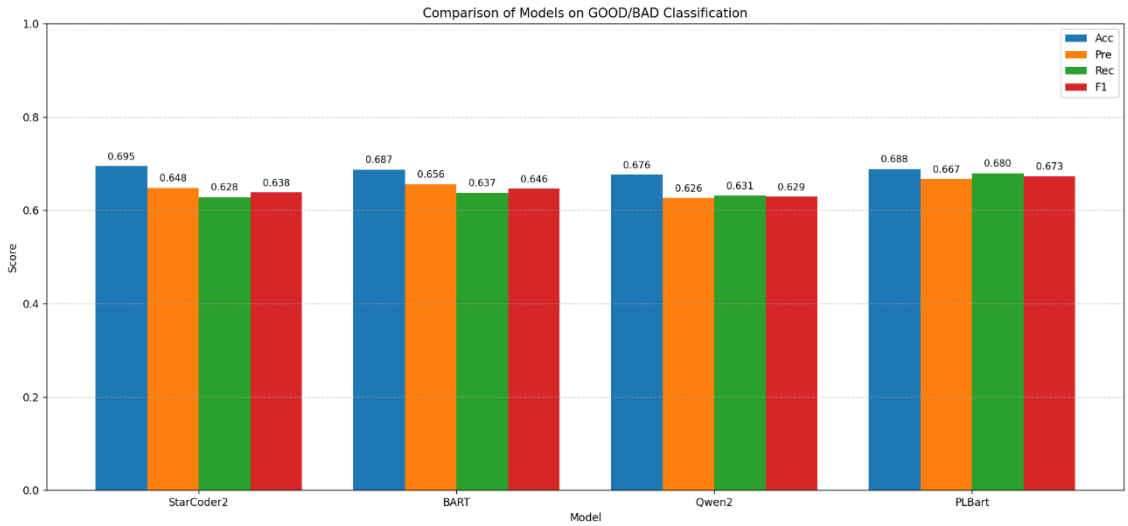
Hình 4.4 Mô hình không được train với dataset và sử dụng Vanila Prompt

Kết quả trong Hình 4.5 cho thấy sau khi được fine-tune với tập dữ liệu Juliet C/C++, hiệu năng của tất cả các mô hình đều được cải thiện rõ rệt so với trạng thái chưa huấn luyện. Các mô hình đạt F1-score trong khoảng 0.69–0.74, trong đó StarCoder2 và BART cho kết quả cao nhất. Điều này cho thấy việc huấn luyện với dữ liệu chuyên biệt giúp mô hình học được các đặc trưng lỗ hổng hiệu quả hơn ngay cả khi chỉ sử dụng Vanilla Prompt.



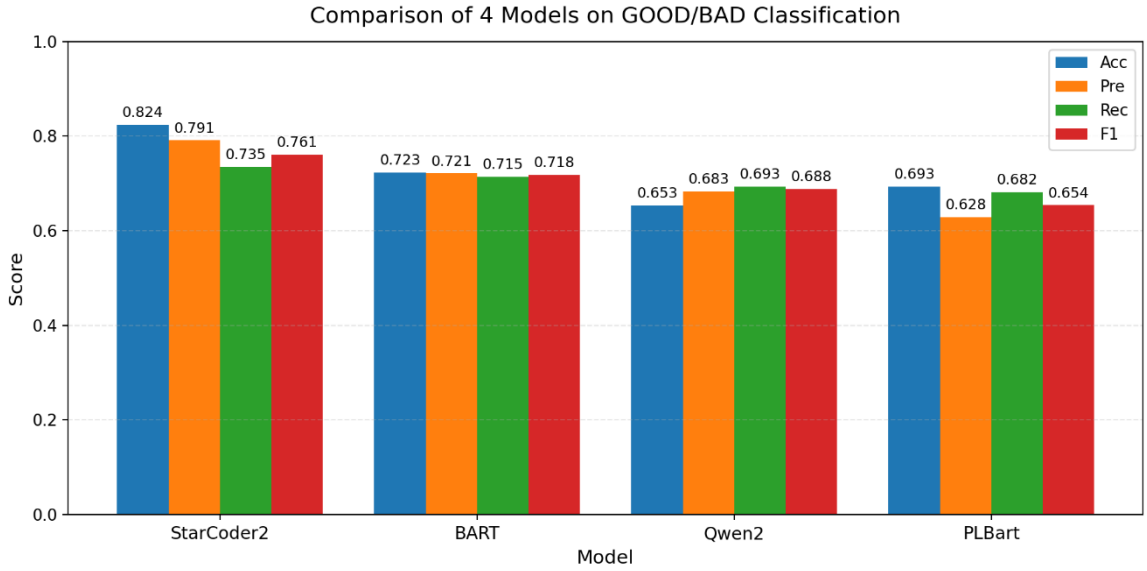
Hình 4.5 Mô hình đã được train với dataset Juliet c/c++ và sử dụng Vanila Prompt

Kết quả trong Hình 4.6 cho thấy các mô hình sau khi được fine-tune với tập dữ liệu Juliet Java đều đạt hiệu năng ổn định, với F1-score dao động trong khoảng 0.63–0.67. So với trường hợp huấn luyện trên C/C++, hiệu năng có xu hướng giảm nhẹ, phản ánh sự khác biệt về đặc trưng ngôn ngữ và cấu trúc mã Java. Tuy nhiên, các mô hình vẫn duy trì khả năng phân biệt GOOD/BAD ở mức chấp nhận được khi sử dụng Vanilla Prompt.



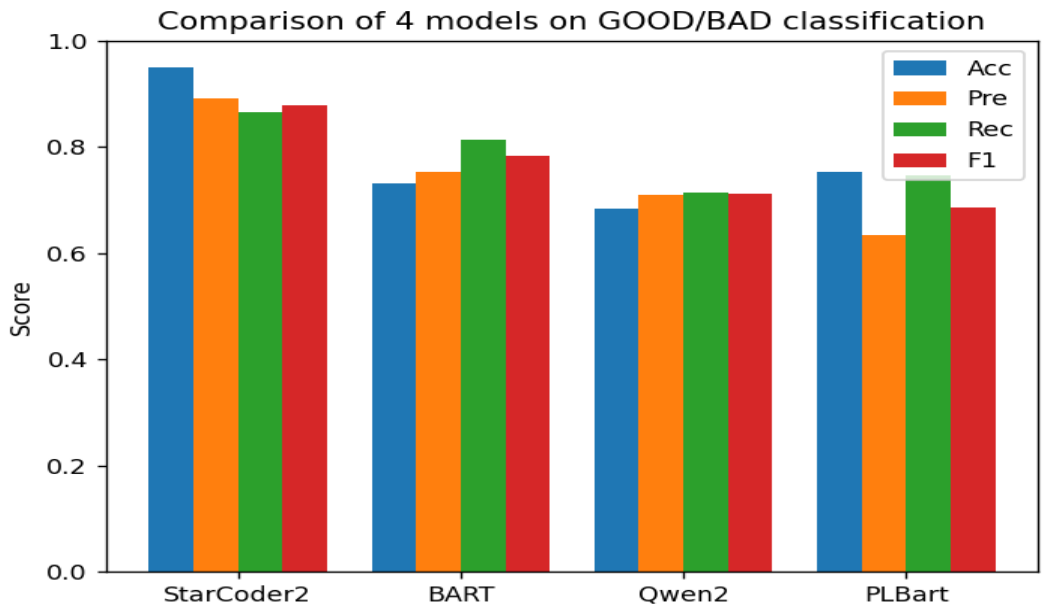
Hình 4.6 Mô hình đã được train với dataset Juliet Java và sử dụng Vanila Prompt

Kết quả trong Hình 4.7 cho thấy việc sử dụng Chain-of-Thought (CoT) Prompt giúp cải thiện hiệu năng của các mô hình ngay cả khi chưa được fine-tune với dữ liệu huấn luyện. Các mô hình đạt F1-score trong khoảng 0.65–0.76, cao hơn so với trường hợp sử dụng Vanilla Prompt, đặc biệt là StarCoder2. Điều này cho thấy CoT Prompt đóng vai trò hỗ trợ mô hình suy luận tốt hơn trong bài toán phân loại GOOD/BAD, ngay cả khi không có huấn luyện chuyên biệt.



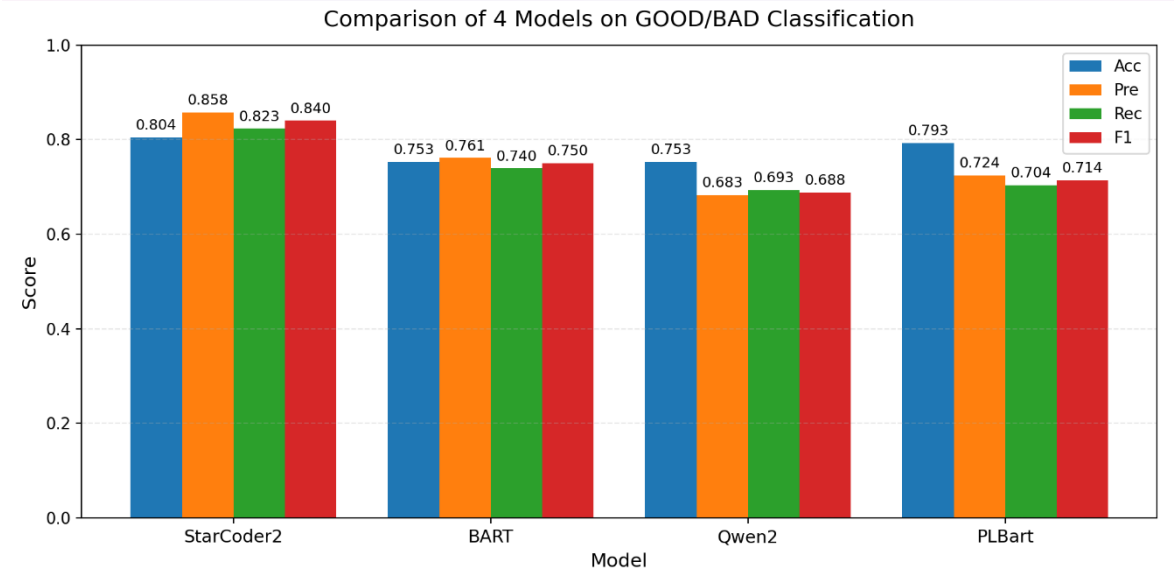
Hình 4.7 Mô hình không train với dataset và sử dụng COT Prompt

Kết quả trong Hình 4.8 cho thấy khi kết hợp fine-tuning với tập dữ liệu Juliet C/C++ và Chain-of-Thought (CoT) Prompt, hiệu năng của các mô hình được cải thiện rõ rệt. Các mô hình đạt F1-score cao, trong đó StarCoder2 và BART cho kết quả nổi bật nhất. Điều này cho thấy sự kết hợp giữa dữ liệu huấn luyện chuyên biệt và chiến lược prompting có hướng dẫn giúp mô hình khai thác tốt hơn các đặc trưng lỗ hổng trong mã nguồn.



Hình 4.8 Mô hình đã được train với dataset Juliet C/C++ và sử dụng COT Prompt

Kết quả trong Hình 4.9 cho thấy khi fine-tune với tập dữ liệu Juliet Java và sử dụng Chain-of-Thought (CoT) Prompt, các mô hình tiếp tục đạt được hiệu năng cao và ổn định, với F1-score dao động trong khoảng 0.69–0.84. So với Vanilla Prompt, CoT Prompt giúp cải thiện rõ rệt khả năng suy luận của mô hình, đặc biệt đối với StarCoder2 và BART. Điều này cho thấy chiến lược prompting có hướng dẫn đóng vai trò quan trọng trong việc nâng cao hiệu quả phát hiện lỗ hổng, ngay cả khi dữ liệu huấn luyện thay đổi theo ngôn ngữ lập trình.



Hình 4.9 Mô hình đã được train với dataset Juliet Java và sử dụng COT Prompt

Chương 5. KẾT LUẬN

Trong khóa luận này, nhóm đã tiến hành khảo sát một cách hệ thống khả năng phát hiện lỗ hổng phần mềm của các mô hình ngôn ngữ lớn cho mã nguồn (LLMs-for-code) thông qua nhiều kịch bản thực nghiệm khác nhau. Kết quả cho thấy quá trình fine-tuning với dữ liệu huấn luyện chuyên biệt (Juliet C/C++ và Juliet Java) đóng vai trò then chốt trong việc nâng cao hiệu năng của mô hình, giúp cải thiện rõ rệt các chỉ số Precision, Recall và F1-score so với mô hình ở trạng thái pre-trained. Bên cạnh đó, việc so sánh giữa các chiến lược prompting cho thấy Chain-of-Thought (CoT) Prompt mang lại hiệu quả vượt trội so với Vanilla Prompt, đặc biệt trong các trường hợp mô hình chưa được huấn luyện hoặc khi dữ liệu có độ phức tạp cao. Các thí nghiệm cross-language cũng chỉ ra rằng mặc dù hiệu năng có suy giảm khi huấn luyện và kiểm thử trên các ngôn ngữ khác nhau, các mô hình vẫn duy trì được khả năng khái quát ở mức chấp nhận được.

Ngoài hiệu năng dự đoán, đề tài còn tập trung đánh giá khả năng khả diễn giải của mô hình thông qua phương pháp SHAP. Kết quả phân tích cho thấy các mô hình có xu hướng đưa ra quyết định dựa trên một tập con nhỏ các token quan trọng, đạt mức Sparsity tương đối cao, đồng thời duy trì mức độ ổn định và bền vững hợp lý trước các nhiễu nhẹ. Mặc dù việc áp dụng SHAP làm gia tăng chi phí tính toán, mức chi phí này được xem là chấp nhận được trong các kịch bản phân tích và kiểm toán bảo mật, nơi tính minh bạch và khả năng giải thích đóng vai trò ưu tiên.

Tuy nhiên, từ góc độ ứng dụng thực tế, các kết quả của khóa luận cũng cho thấy một số thách thức cần được xem xét. Hiệu năng và hành vi suy luận của các mô hình LLMs-for-code phụ thuộc mạnh vào đặc thù dữ liệu huấn luyện và kịch bản sử dụng. Các bộ dữ liệu chuẩn hóa như Juliet Test Suite thuận lợi cho việc đánh giá học thuật, nhưng chưa phản ánh đầy đủ sự đa dạng, quy mô và phong cách lập trình của mã nguồn trong môi trường phát triển phần mềm thực tế. Do đó, kết quả thực nghiệm đạt được trong phạm vi luận văn chưa thể khẳng định rằng các mô hình

LLMs-for-code đã sẵn sàng để triển khai trực tiếp như một giải pháp tự động hoàn chỉnh trong các hệ thống bảo mật thực tế.

Bên cạnh đó, mặc dù các chiến lược prompting và phương pháp XAI như SHAP góp phần nâng cao tính minh bạch của mô hình, các lời giải thích sinh ra vẫn mang tính xấp xỉ và chưa đảm bảo phản ánh đầy đủ cơ chế nội tại của mô hình. Điều này đặc biệt quan trọng trong các kịch bản an toàn thông tin, nơi các quyết định phát hiện lỗ hổng cần được kiểm chứng và xác nhận bởi chuyên gia trước khi đưa ra các hành động bảo mật.

Từ những phân tích trên, khóa luận nhận định rằng các mô hình LLMs-for-code hiện nay phù hợp hơn với vai trò công cụ hỗ trợ chuyên gia bảo mật, chẳng hạn như hỗ trợ rà soát mã nguồn, gợi ý vị trí tiềm ẩn rủi ro hoặc cung cấp phân tích sơ bộ, thay vì thay thế hoàn toàn các phương pháp kiểm tra và đánh giá bảo mật truyền thống. Trên cơ sở đó, đề tài mở ra các hướng nghiên cứu tiếp theo như mở rộng thực nghiệm trên dữ liệu mã nguồn thực tế, kết hợp với các kỹ thuật phân tích tĩnh và động, tối ưu chi phí tính toán của phương pháp giải thích, cũng như phát triển các cơ chế XAI chuyên biệt hơn cho miền mã nguồn nhằm nâng cao độ tin cậy khi ứng dụng trong thực tế.

TÀI LIỆU THAM KHẢO

[1] W. Ahmad et al., "Unified pre-training for program understanding and generation," in Proc. 2021 Conf. of the North American Chapter of the Association for Computational Linguistics (NAACL-HLT), 2021, pp. 2655–2668, doi: 10.18653/v1/2021.naacl-main.211.

[2] T. Boland and P. E. Black, "Juliet 1.1 C/C++ and Java Test Suite," National Institute of Standards and Technology (NIST), 2012.

[3] H. Deters, J. Droste, M. Obaidi, and K. Schneider, "Exploring the means to measure explainability: Metrics, heuristics and questionnaires," Information and Software Technology, 2025, doi: 10.1016/j.infsof.2025.107682.

[4] M. Feng et al., "CodeBERT: A pre-trained model for programming and natural languages," in Proc. 2020 Conf. on Empirical Methods in Natural Language Processing (EMNLP), 2020, pp. 1536–1547, doi: 10.18653/v1/2020.emnlp-main.150.

[5] E. Linardatos, V. Papastefanopoulos, and S. Kotsiantis, "Explainable AI: A Review of Machine Learning Interpretability Methods," Entropy, vol. 23, no. 1, p. 18, 2021, doi: 10.3390/e23010018.

[6] M. Lewis et al., "BART: Denoising Sequence-to-Sequence Pre-training for Natural Language Generation, Translation, and Comprehension," in Proc. 58th Annual Meeting of the Association for Computational Linguistics (ACL), 2020, pp. 7871–7880.

[7] P. Liu et al., "Pre-train, Prompt, and Predict: A Systematic Survey of Prompting Methods in Natural Language Processing," ACM Computing Surveys, vol. 55, no. 9, pp. 1–35, 2023.

[8] T.-T. Nguyen, T. T. Vu, H. D. Vo, and S. Nguyen, "An empirical study on capability of Large Language Models in understanding code semantics," Information and Software Technology, 2025, doi: 10.1016/j.infsof.2025.107780.

[9] M. T. Ribeiro, S. Singh, and C. Guestrin, “Why Should I Trust You?: Explaining the Predictions of Any Classifier,” in Proc. 22nd ACM SIGKDD Int. Conf. on Knowledge Discovery and Data Mining (KDD), 2016, pp. 1135–1144.

[10] H. Tufano et al., “An empirical investigation into learning bug-fixing patches in the wild via neural machine translation,” in Proc. 44th Int. Conf. on Software Engineering (ICSE), 2022, pp. 364–375, doi: 10.1145/3510003.3510147.

[11] J. Wei et al., “Chain-of-Thought Prompting Elicits Reasoning in Large Language Models,” in Proc. Advances in Neural Information Processing Systems (NeurIPS), 2022.

[12] S. M. Lundberg and S.-I. Lee, “A Unified Approach to Interpreting Model Predictions,” in Proc. Advances in Neural Information Processing Systems (NeurIPS), 2017, pp. 4765–4774.

[13] H. Zhao et al., “Explainability for large language models: A survey,” ACM Transactions on Intelligent Systems and Technology, vol. 15, no. 2, pp. 1–38, 2024, doi: 10.1145/3639372.

[14] M. A. Mersha, M. G. Yigezu, H. Shakil, A. K. AlShami, S. Byun, and J. Kalita, “A unified framework with novel metrics for evaluating the effectiveness of XAI techniques in LLMs,” arXiv preprint arXiv:2503.05050, Apr. 2025